



GO: от первой строки до интернет-магазина

JUNIOR · MIDDLE · SENIOR

Практическая книга: один пример,
который вырастает в работающий магазин



SHANRAQ.ORG

Баймурзин Даулет Абузарович

Go: от первой строки до интернет-магазина

Ознакомительный фрагмент · предисловие и главы 1–12

Основы Go · пакеты · HTTP · HTML-шаблоны

Ознакомительный фрагмент 0.21.0-sample

shanraq.org

2026

Выходные сведения

Баймурзин Даулет Абузарович

Go: от первой строки до интернет-магазина: практическая книга. — shanraq.org, 2026. — Электронное издание. Ознакомительный фрагмент 0.21.0-sample.

Практическая книга о Go. Читатель шаг за шагом создаёт магазин цифровых книг: от первой программы и каталога до заказов, выдачи файлов и эксплуатации приложения. Материал организован в три уровня и использует опорные образы, проверяемые примеры и самостоятельные задания.

Дополнительный учебный материал для студентов бакалавриата ИТ-университетов, технических и многопрофильных вузов по направлениям «Программная инженерия», «Компьютерные науки», «Информационные системы», «Вычислительная техника и программное обеспечение». Отдельные разделы полезны студентам информационной безопасности и прикладной информатики, преподавателям практических дисциплин и разработчикам, изучающим Go самостоятельно.

Автор: Баймурзин Даулет Абузарович.

По образованию — инженер-технолог по ОПИ, по призванию — инженер-программист.

Издательский проект: shanraq.org.

Дата этой редакции: 17 сентября 2026 года.

Язык: русский.

Форматы: PDF, EPUB, HTML.

Этот файл содержит предисловие и главы 1–12. Все 42 главы написаны; полный черновик проходит редакционную проверку. Дата относится к рабочей редакции, а не к завершённому коммерческому выпуску. ISBN и библиотечные классификационные индексы в этой редакции не указаны.

© 2026 Баймурзин Даулет Абузарович.

Исходная обложка предоставлена автором; новая версия адаптирована с помощью ИИ.

Подготовка рукописи и учебных материалов ведётся с использованием инструментов ИИ.

Персонаж Go gopher создан Renee French: [источник](#), [CC BY 4.0](#). В обложке, логотипе и иконках использована адаптация с помощью ИИ: добавлены тележка и книга, изменены композиция и оформление. Издание не является официальным продуктом проекта Go.

Код, перенесённый из исходного репозитория, сохраняет GNU AGPL-3.0; [текст лицензии](#).

Шрифты Noto распространяются по [SIL Open Font License 1.1](#); копия лицензии включена в электронный комплект. Этот фрагмент опубликован бесплатно для ознакомления. Главы 13–42 и полные PDF, EPUB, HTML предназначены для продажи через shanraq.org. Бесплатное чтение фрагмента не изменяет условия лицензирования материалов.

Сайт: [shanraq.org](#).

Код книги: [бесплатный архив примеров глав 2–12](#). Покупка для запуска этих примеров не требуется.

Оглавление

Предисловие. Магазин начинается с одной книги

1. Рабочее место, которое можно проверить
2. Книга появляется в терминале
3. Название, цена и состояние книги
4. Какие издания можно показать
5. Правило цены и первый тест
6. Сколько букв в названии книги
7. Каталог: список, массив и словарь
8. Книга как значение: структуры, методы и указатели
9. Файл не открылся: ошибки и освобождение ресурсов
10. Пакеты: у каждого правила свой адрес
11. Каталог отвечает браузеру
12. HTML-шаблон превращает каталог в страницу

Предисловие. Магазин начинается с одной книги

Ознакомительный фрагмент. Здесь доступны предисловие и главы 1–12. Вы начнёте с первой программы и дойдёте до HTTP-каталога с HTML-шаблонами. Полная книга содержит 42 главы; заказы, доступ к файлам, административная часть и эксплуатация рассматриваются в продолжении. [Карточка полной книги на shanraq.org](http://shanraq.org). В учебном проекте используется тестовая оплата: реальные деньги не списываются.

Вы открыли книгу о Go. К её последней главе у вас будет магазин электронных книг. Первым товаром станет та самая книга, которую вы сейчас читаете.

Это удобно для учёбы: у каждой новой возможности есть понятная причина. Строка хранит название. Срез собирает каталог. Функция считает стоимость. Сервер показывает страницу. База помнит заказ после перезапуска. Проверка доступа решает, кому можно отдать файл.

Но магазин начинается раньше сервера. Он начинается с обещания: человек выбирает книгу, оплачивает её и получает возможность читать. Наша работа — написать программу, которая выполняет это обещание даже тогда, когда что-то идёт не по плану.

Ориентир на экране

Снимок работающей контрольной точки 42 показывает итоговый интерфейс. В ранних главах мы будем приходить к нему постепенно; адреса, заказ и цены здесь учебные.

[Перейти к содержимому](#)



eGopher

Корзина

Войти

Каталог

Поиск по названию

Найти

Go: от первой строки до интернет-магазина

Открытый фрагмент

4900.00 KZT

Количество (от 1 до 10)

1

В корзину

Страница 1

© 2026 eGopher

Учебный интернет-магазин.

Итог проекта: каталог интернет-магазина с книгой на Go. Цена на снимке демонстрационная.

Как пройти основы без пропущенных ступеней

Сначала выполните небольшие опыты, затем читайте применение в магазине. Команды `go run ./labs/...` запускаются из корня контрольной точки главы, где лежит её `go.mod`; это отдельные программы, а не запуск всего магазина. До запуска предскажите вывод, затем сравните его с объяснением, измените одну деталь и объясните разницу. Не переходите дальше только потому, что скопированный код скомпилировался.

Где	Что нужно научиться объяснять
1–2	Текущая директория, файл программы, модуль; чем <code>go run</code> отличается от <code>go build</code> .
3–5	Переменные, типы, нулевые значения, условия, циклы, вызовы функций и проверка результата.
6–7	Байты и руны; массив, срез и <code>map</code> ; копирование значения и общее хранилище.

Где	Что нужно научиться объяснять
8–9	Структуры, указатели и методы; функции как значения, замыкания, <code>defer</code> и ошибки.
10–12	Пакеты, HTTP-запрос и обработчик, HTML-шаблон, <code>layout</code> и <code>partial</code> , оформление страницы.

В полной книге главы 15–18 отдельно разбирают JSON, SQL, транзакции и собственные интерфейсы. Стандартный интерфейс `error` нужен раньше — его минимальный контракт объясняется в главе 9, а общий механизм и ловушки интерфейсов возвращаются в главе 18. Горутины, каналы и синхронизация вводятся в главе 30; отмена работы — в 31; параметры типов и итераторы — в 34. До соответствующей главы от читателя не требуется знать эти механизмы целиком.

Если опыт не получается, сначала проверьте директорию, версию Go и точный текст ошибки. Вернитесь к указанной предпосылке; перепечатать решение без объяснения результата — недостаточный критерий готовности. Книга обучает языку через проект и не заменяет полный справочник по всем возможностям Go.

Зачем учиться, если ИИ уже пишет код

Этот вопрос заслуживает прямого ответа. Если ваша цель — получить несколько строк программы, знание языка уже не единственный путь. ИИ может предложить реализацию, объяснить чужой код, подготовить тесты и помочь с исправлением ошибки. Учиться Go сегодня имеет смысл ради более сильной цели: уметь превращать свою задачу в работающий продукт, понимать основания решений и сохранять возможность изменить их.

Не стоит строить мотивацию на мысли, что ИИ «никогда не научится» делать какую-то часть работы. Возможности инструментов меняются. В феврале 2026 года исследователи METR прямо предупредили: их прежний результат о замедлении опытных разработчиков нельзя просто переносить на новые условия. Новое исследование столкнулось с отбором участников и задач, а величину современного ускорения надёжно оценить не удалось. Это аргумент в пользу измерения результата в своей работе, а не лозунга за или против ИИ. [Обновление исследования METR, 24 февраля 2026 года.](#)

Что же дают знания, если помощник способен написать и код, и проверку к нему?

Вы можете задать проверяемую цель. «Сделай интернет-магазин» оставляет много решений неявными. Можно ли покупать снятое с продажи издание? Сохраняется ли старая цена заказа? Что делать, если деньги списаны, а ответ не пришёл? Программирование учит превращать такие вопросы в точные правила. Здесь важна не длина запроса к ИИ, а способность заметить недосказанность и выбрать поведение, которое действительно нужно покупателю.

Вы можете отличить доказательство от убедительного объяснения. Зелёный тест полезен лишь настолько, насколько верно его ожидание. Если код разрешает чужому пользователю скачать файл, а тест проверяет только владельца, они могут выглядеть согласованными и всё же пропускать ошибку. Знание HTTP, сессий и хранения данных позволяет самому сформулировать отрицательный сценарий: войти другим пользователем, подставить чужой ID и проверить отказ. Проверка становится независимой от предположений автора реализации — человека или модели.

Вы сохраняете управление после первой удачной демонстрации. У магазина появятся новые издания, изменится цена, закончится место на диске, придёт повторное уведомление об оплате. Чтобы исправить проблему, нужно связать наблюдаемый симптом с запросом, состоянием базы и конкретным изменением. Навык чтения кода помогает оценить предложенную правку до её применения, а тесты и история Git — проверить результат и вернуться к рабочей версии. Вы можете поручить значительную часть работы инструменту, сохраняя понимание того, что меняете и почему.

Вы получаете больше самостоятельности в выборе задач. Когда вы понимаете основы, помощник становится полезен не только на знакомом примере. Можно сравнить два варианта, проверить их на своих данных и отказаться от лишней сложности. Это позволяет быстрее пробовать идеи и делать небольшие полезные продукты. Но качество идеи, интерес покупателей и доход не следуют автоматически из умения писать код. Книга не обещает ни гарантированной работы, ни успеха бизнеса; она даёт практику создания и проверки системы, за которую вы можете отвечать.

Распространённость инструментов тоже не означает безусловного доверия к результату. В опросе Stack Overflow за 2025 год недоверие к точности ИИ-инструментов выражали 46% ответивших на соответствующий вопрос, доверие — 33%. Это ответы участников опроса, а не измерение доли ошибочных программ. Практический вывод для нашей книги: полезность помощника и необходимость проверки вполне совместимы. [Stack Overflow Developer Survey 2025: AI](#).

Поэтому мы не будем соревноваться с ИИ в скорости набора текста или требовать заучивания всей стандартной библиотеки. Сначала вы освоите достаточно языка, чтобы читать небольшую программу, объяснять её состояние и проверять результат. Затем последовательно разберёте данные, права доступа, транзакции, повторные события и остановку сервера. Эти знания связывают отдельные файлы в понятную вам систему.

Используйте помощника так, чтобы после работы с ним оставался собственный навык:

1. До подсказки запишите ожидаемое поведение и один случай, когда операция должна завершиться отказом.
2. Попробуйте выполнить небольшой шаг сами. Если застряли, попросите подсказку или объяснение конкретного места, затем проверьте его запуском.
3. Для предложенной правки прочитайте diff и объясните, какое правило она сохраняет. Попросите ИИ поискать контрпример, но сами проверьте, действительно ли он проверяет нужное условие.
4. На следующий день измените условие задачи и повторите ключевую часть без готового ответа. Если не получается, вернитесь к маленькому примеру: работающая вчера программа ещё не означает освоенный сегодня приём.

Уже в этой книге вы сможете проверить пользу такого подхода на собственном магазине: не только получить страницу покупки, но и объяснить, почему чужой заказ закрыт, повтор оплаты не выдаёт лишнего права, а снятая с продажи книга остаётся у прежнего покупателя. Именно эта способность — поставить задачу, проверить решение и осознанно его развивать — делает обучение осмысленным при развитии ИИ.

Образ, который можно унести с собой

Закройте на секунду текст и представьте небольшой книжный стол: на нём лежит книга, рядом стоит ценник, под столом хранится тетрадь заказов. Покупатель выбирает книгу; продавец проверяет оплату и отдаёт экземпляр. В этой картине уже есть будущие данные, действия и правила нашего магазина.

Мы будем возвращаться к знакомым предметам: ценнику, карточке, полке, записанному адресу. Это опорные образы. Их задача — помочь вам вспомнить связь между частями программы. Адрес на бумаге, например, напоминает, что копирование адреса не создаёт вторую квартиру. В главе об указателях мы проверим похожее различие на данных книги.

Образ всегда имеет границу. Компьютерная память не устроена как квартира, а срез Go не является деревянной полкой. После знакомства с образом мы назовём точное правило, запустим пример и найдём случай, в котором бытовая подсказка уже недостаточна.

Так мы используем приёмы опорных сигналов В. Ф. Шаталова: сначала видим целое, потом разбираем детали, сжимаем их до понятной опоры и восстанавливаем смысл самостоятельно. Маленький рисунок на полях полезен только тогда, когда по нему вы можете объяснить действие. Художественные способности для этого не нужны: прямоугольник, стрелка и два слова вполне подойдут.

Что мы построим

Посетитель увидит каталог и откроет описание. Покупатель создаст заказ и перейдёт к оплате. После подтверждения книга появится в его библиотеке. Он сможет вернуться завтра и скачать её снова.

У автора будет возможность добавить книгу, исправить описание и выпустить новую версию файла. При этом изменение сегодняшней цены не должно переписать вчерашнюю покупку. А человек, который подобрал адрес чужого заказа, не должен получить чужую библиотеку.

Сначала всё будет происходить на вашем компьютере. Для первых опытов нам не нужны настоящие деньги. Позже появится тестовая платёжная система: мы научимся обрабатывать успех, отказ и ситуацию, когда ответа нет. Подключение реальной оплаты требует отдельной настройки выбранного провайдера; учебный ответ «успешно» не означает, что деньги поступили.

Три ступени одной работы

На первой ступени вы освоите основы языка и соберёте каталог. Мы будем останавливаться на вещах, которые опытный разработчик делает автоматически: где находится файл, что означает сообщение компилятора, почему изменение значения не всегда меняет исходные данные.

На второй ступени приложение начнёт хранить данные и различать пользователей. Здесь появятся заказы, сессии, платежи и файлы. Вам потребуется объяснять не только удачный путь, но и отказ: что произойдёт, если запрос пришёл второй раз или база не ответила.

На третьей ступени мы будем проверять решения под нагрузкой и при сбоях. Научимся находить причину задержки, завершать работу без брошенных задач, восстанавливать данные и выпускать изменения. Сложность здесь определяется последствиями решений, а не

количеством установленных технологий.

Эти ступени называются Junior, Middle и Senior. Это ориентиры глубины работы. Прочитанная глава сама по себе не присваивает квалификацию. Более полезная проверка — сумеете ли вы объяснить решение и повторить его в задаче, которую ещё не видели.

Кому адресована книга

Книга предназначена для самостоятельного изучения Go и использования в качестве дополнительного учебного материала на практических занятиях и в учебных проектах вуза. Начать можно без опыта программирования: первая ступень объясняет инструменты, запись языка и работу небольших программ. Если вы уже пишете на другом языке, первые главы помогут связать знакомые задачи с правилами Go.

В высшем образовании книга прежде всего адресована студентам бакалавриата ИТ-университетов, технических университетов и ИТ-факультетов многопрофильных вузов. Особенно близки нашему проекту следующие направления:

Направление или специальность	Для каких учебных задач пригодится книга
Программная инженерия / Software Engineering	Разработка приложения, разделение ответственности, тестирование, совместная работа с кодом
Компьютерные науки / Computer Science	Практика языка, структуры данных, работа с памятью и конкурентное выполнение
Информационные системы	Проектирование каталога и заказов, базы данных, серверные интерфейсы и права доступа
Вычислительная техника и программное обеспечение	Создание и сопровождение серверных программ, сетевое взаимодействие, инструменты сборки
Прикладная информатика	Перевод требований пользователя в данные и поведение работающего продукта
Информационная безопасность / кибербезопасность	Практика проверки входных данных, управления доступом и безопасной разработки веб-приложений

Таблица описывает назначение полной программы книги. В ознакомительном фрагменте доступны основы языка, пакеты, HTTP и шаблоны; база данных, доступ и эксплуатация входят в полную книгу. Книга не заменяет весь учебный план: математические основы, общие алгоритмы, теория вычислений и специальные дисциплины изучаются также по профильным источникам.

Примеры образовательных программ в Казахстане

Для ориентира сопоставим проект книги с реально существующими программами. Названия и коды проверены по официальным страницам вузов 12 сентября 2026 года; оценка учебной пользы в последнем столбце — авторская.

Вуз	Программа	Возможное применение книги
Astana IT University	6B06102 Software Engineering	Практикум по разработке ПО и веб-приложений; сквозной проект магазина

Вуз	Программа	Возможное применение книги
Международный университет информационных технологий — МУИТ / ITU	6B06105 «Информационные системы»	Проектирование и реализация информационной системы с каталогом, заказами и хранением данных
Satbayev University	Computer Science	Практика программирования на Go и инженерная часть учебного проекта

Этот перечень можно продолжить другими вузами с близкими направлениями. Включение книги в конкретную дисциплину и требования к работе определяет преподаватель или кафедра.

Для курса основ программирования подойдут первые контрольные точки и обязательные задания. Для проектного семинара — развитие магазина с объяснением принятых решений. В дипломной работе учебную основу потребуется расширить собственными требованиями, реализацией и проверками, явно указав использованные материалы.

Если вы учитесь на другой специальности или уже работаете инженером, путь для вас тоже открыт. Начните с первых глав и собственной задачи. Здесь важнее готовность проверить своё объяснение запуском программы, чем название диплома.

Как читать и практиковаться

Перед запуском попробуйте угадать результат. Запишите ответ одной строкой. Затем запустите программу и сравните. Если ответы разошлись, у вас появилось точное место для разбора.

После примера измените одно условие. Уберите книгу из каталога. Поставьте нулевое количество. Передайте неизвестный идентификатор. Умение пользоваться программой начинается с понимания того, где она должна отказать.

Затем выполните обязательное задание без готового решения. Если застряли, прочитайте первую подсказку. Решение стоит открывать после собственной попытки: оно нужно для сравнения подходов и разбора ошибок.

Вопросы «Скажите своими словами» тоже часть работы. Можно ответить вслух, в заметках или объяснить товарищу. Если для объяснения приходится перечитать абзац, вернитесь к небольшому примеру и проверьте его ещё раз.

Как устроены файлы и переходы

Примеры глав 2–12 доступны бесплатно в [открытом репозитории](#) и в архиве [go-book-preview-code.zip](#) ознакомительного выпуска. Распакуйте архив в отдельную рабочую папку. Короткий путь `examples/07-catalog` в главе означает `book/examples/07-catalog` внутри комплекта. Там же находятся `book/book.json` с составом выпуска и `book/TRANSITIONS.md` с изменениями между примерами.

Не запускайте файлы внутри архива. Откройте папку нужной контрольной точки, содержащую её `go.mod`, и запустите терминал там. Покупка и доступ к приватному репозиторию для этих глав не требуются. Git, Python, npm и Docker для запуска примеров не нужны; Python

используется издателем для сборки книги.

Есть два способа работы. В готовой контрольной точке сначала запустите неизменённый пример, затем делайте задание в её копии. При самостоятельном наборе используйте отдельную папку для каждой главы, перенесите свой `go.mod` и только перечисленные в главе файлы. Так старый тест не потеряет свою функцию, а две функции `main` не окажутся в одном пакете. Сохраняйте предыдущую папку: это ваша точка возврата. Не запускайте `go mod init` повторно поверх готового `go.mod`. В архиве кода файл `book/TRANSITIONS.md` перечисляет точные новые, изменённые и удалённые файлы каждой контрольной точки. Список строится из самих исходников, чтобы перенос не зависел от догадки о пропущенном имени. Он дополняет объяснение главы; применяйте изменения в копии, сохраняя предыдущую работу.

После изменения сохраняйте файлы, выполняйте `gofmt -w .`, `go run .` в главах 2–9, `go run ./cmd/shop` начиная с главы 10, а начиная с главы 5 — `go test ./. . . go run main.go` может пропустить другие файлы пакета; в книге запускается пакет целиком. Сообщение `[no test files]` означает, что тестов не найдено, а не что ваше задание проверено. Примеры вывода основной программы относятся к исходной контрольной точке; после задания вывод может намеренно измениться.

Когда переходить дальше

Одна удачная попытка сразу после чтения ещё не показывает, что материал удержался в памяти. В начале следующего занятия восстановите вчерашнее правило без текста, затем сверяйтесь. Через несколько занятий вернитесь к нему в другой задаче. Интервалы подстраивайте под себя; обязательного числа минут и обещания одинакового темпа для всех здесь нет.

После главы	Проверка без копирования	Если не получается, куда вернуться
1	Найти папку проекта и показать её <code>go.mod</code>	«Подготовьте папку» и «Дайте проекту имя»
2	Написать программу из двух строк вывода	«Читаем запись по символам»
3	Объяснить вывод цены <code>105</code> и отличие <code>:</code> от <code>=</code>	«Как получилась точка и два нуля»
4	Предсказать обход четырёх изданий с пропуском третьего	«Цикл: начало, проверка, шаг»
5	Посчитать скидку для <code>105</code> , <code>10%</code> и написать тест отказа для <code>101%</code>	«Сначала договоримся о результате»
6	Объяснить, почему 80 букв <code>Ә</code> допустимы	«Байт, руна и то, что видит читатель»
7	Добавить карточку, сохранив порядок и проверку названия	«Почему витрина не обходит <code>map</code> »
8	Объяснить, почему публикация копии не меняет элемент среза	«Ловушка цикла: меняем копию»
9	Проследить результат при удачном чтении и отказе закрытия	«Записка <code>defer</code> : что и когда выполняется»

Переходите дальше, когда можете предсказать результат, объяснить причину и самостоятельно внести небольшое изменение. При затруднении вернитесь к указанному разделу, измените один вход и снова проверьте объяснение. Подсказки и ответы ниже заданий предназначены для обратной связи после попытки, а не для оценки скорости чтения.

Такой порядок сочетает разобранные примеры, самостоятельные задачи, извлечение из памяти и распределённое повторение. Это адаптация общих рекомендаций [IES по организации обучения](#), а не доказательство эффективности именно этой книги. Для её проверки нужны наблюдения за самостоятельными читателями.

Первое упражнение: обещание покупателю

Пока без кода. Рассмотрим три события:

1. Человек нажал кнопку «Купить».
2. В браузере открылась страница «Спасибо».
3. Сервер получил и проверил подтверждение оплаты.

После какого события можно считать оплату подтверждённой?

Не спешите выбирать ответ только по названию страницы. Браузер показывает то, что ему отдали, а переход по ссылке сам по себе ничего не говорит о движении денег.

Ответ. Основанием служит проверенное подтверждение платёжной системы, связанное с нужным заказом, суммой и валютой. Нажатие кнопки выражает намерение. Страница благодарности сообщает результат пользователю, но не доказывает его серверу.

Теперь продолжение: подтверждение пришло дважды. Должен ли магазин создать два заказа или выдать две независимые покупки? Нет. Повтор одного события должен сохранять уже достигнутый результат. Позже мы назовём это идемпотентностью и проверим тестом.

Пока достаточно заметить: одно короткое требование к поведению приводит нас к устройству данных, обработке ошибок и проверкам. Так будет построена вся книга.

Как понять, что вы продвигаетесь

В конце главы должно измениться что-то наблюдаемое. Появилась карточка. Неверная форма получила объяснение. Заказ сохранился после перезапуска. Чужой файл остался недоступен. Программа завершилась и не потеряла принятую работу.

Если пример запускается только после необъяснённых исправлений, это недостаток материала, а не ваша обязанность угадывать. Сохраняйте команду, сообщение, версию Go и номер контрольной точки. Эти сведения позволяют воспроизвести проблему.

Наш следующий шаг маленький: подготовить рабочую папку и вывести название книги в терминал. У этого вывода ещё не будет покупателей. Но уже будет важное свойство — вы сможете запустить его сами и объяснить каждую строку.

1. Рабочее место, которое можно проверить

В конце этой главы у вас будет папка проекта и терминал, в котором работает команда `go version`. Пока не нужно выбирать базу данных или оплачивать сервер. Сначала убедимся, что можем сохранять файлы и запускать инструменты.

Ориентир на экране

Снимок работающей контрольной точки 42 показывает итоговый интерфейс. В ранних главах мы будем приходить к нему постепенно; адреса, заказ и цены здесь учебные.

[Перейти к содержимому](#)



eGopher [Корзина](#) [Войти](#)

Каталог

Поиск по названию

Найти

Go: от первой строки до интернет-магазина

[Открытый фрагмент](#)

4900.00 KZT

Количество (от 1 до 10)

1

В корзину

Страница 1

© 2026 eGopher

Учебный интернет-магазин.

Тот же итоговый каталог на смартфоне: навигация и форма помещаются по ширине экрана.

Образ: свой рабочий стол

Представьте стол, на котором вы собираетесь написать письмо. На нём лежит папка, внутри — листы; у папки есть название. Если оставить письмо на соседнем столе, открытая перед вами папка его не получит. С программой происходит похожая путаница: вы исправили один `main.go`, а терминал запускает другой.

Наша опора — **стол, папка, сохранённый лист**. Рабочая папка объединяет файлы проекта. Текущая папка терминала определяет, где команда начинает работу. Сохранение переносит набранный в редакторе текст в файл, который прочитает инструмент Go. Редактор может показывать несколько проектов, поэтому взгляд на вкладку ещё не доказывает, что терминал

открыт там же.

У образа есть граница: на компьютере команды могут принимать полный путь к чужой папке. Пока мы намеренно работаем из папки проекта. Нарисуйте прямоугольник с названием [bookshop](#), а внутри — `go.mod`. Перед запуском показывайте пальцем: «Команду выполняю здесь». Это маленькая привычка, которая избавляет от долгих поисков не той ошибки.

Сначала целое

Наша работа будет устроена так:

Где	Что вы делаете	Что получаете
Редактор	Пишете и сохраняете файл <code>main.go</code>	Текст программы на диске
Терминал	Вводите <code>go run</code>	Go собирает и запускает программу
Терминал	Читаете ответ	Проверяете, совпал ли результат с ожиданием

Сохранённый файл и открытая вкладка редактора — не одно и то же. Если вы изменили текст, но не сохранили его, команда может запустить предыдущую версию. Эту причину стоит проверять прежде, чем переписывать программу.

Терминал — программа, принимающая команды. Оболочка внутри него разбирает эти команды. В macOS вы обычно встретите `zsh`, в Linux — `bash`, в Windows будем использовать PowerShell. Там, где команды различаются, книга покажет оба варианта.

Установите Go

Откройте [официальную страницу установки](#). Выберите свою операционную систему и процессор. Для Windows используется установщик MSI, для macOS — PKG; на Linux следуйте инструкции для архива вашей архитектуры. Не вставляйте в терминал команды удаления старой установки, если не понимаете, какой каталог они затронут.

Материалы этого рабочего выпуска проверяются на Go 1.27.1. Ваша версия указана в выводе команды, а не в названии скачанного файла. После установки закройте терминал и откройте новый: так он получит обновлённые настройки поиска программ.

Введите только команду, без символов приглашения вроде `$` или `>`:

```
go version
```

Форма ответа:

```
go version go1.27.1 darwin/arm64
```

Последняя часть зависит от компьютера. [darwin](#) означает macOS, [arm64](#) — архитектуру процессора. На Windows или Linux она будет другой. Здесь важно, что вы получили ответ программы Go, а не сообщение «команда не найдена».

Если Go установлен раньше, не удаляйте его наугад. Сначала посмотрите версию и сверьтесь с инструкцией обновления. Работающий чужой проект может зависеть от своего набора

инструментов.

Подготовьте папку

Выберите место для учебных проектов, к которому у вас есть доступ. Не создавайте новый магазин внутри другого проекта. Откройте терминал в выбранной папке и выполните:

```
mkdir bookshop  
cd bookshop
```

Эти две команды подходят и для PowerShell, и для bash/zsh. Первая создаёт папку, вторая делает её текущей: следующие команды будут работать относительно неё.

Если `bookshop` уже существует, сначала посмотрите, что внутри. Для чистого начала можно выбрать другое имя, например `bookshop-study`; тогда используйте его и в команде `cd`.

Проверьте, где вы находитесь. В macOS/Linux:

```
pwd  
ls
```

В PowerShell:

```
Get-Location  
Get-ChildItem
```

Первая команда показывает путь, вторая — содержимое. Не надо добиваться точного совпадения пути с книгой: у каждого своя домашняя папка. Важно увидеть выбранную вами папку `bookshop`.

Откройте её в редакторе через меню открытия папки. Подойдёт редактор обычного текста, который сохраняет UTF-8. Если используете VS Code, открывайте всю папку, а не случайный файл из соседнего проекта. Для первых глав дополнительные расширения необязательны.

Дайте проекту имя

В терминале внутри `bookshop` выполните один раз:

```
go mod init example.com/bookshop
```

Появится `go.mod`. Модуль — группа пакетов Go с общим описанием версии и зависимостей. Пока достаточно понимать: этот файл обозначает корень нашего проекта. Запись `example.com/bookshop` служит именем учебного модуля; для этого упражнения не нужно покупать домен или создавать сайт по такому адресу.

Код книги связан с репозиторием `dake-edu/gopher-shop`. Контрольные точки находятся в `book/examples/`; их имена модулей отражают путь в репозитории. Для собственного проекта мы используем учебное имя `example.com/bookshop`, чтобы отличать его от готовых примеров.

Не редактируйте `go.mod` ради буквального совпадения с изображением в чужом уроке: версия в нём зависит от инструмента, который его создал. В комплекте книги лежат самостоятельные контрольные точки со своими готовыми `go.mod`; для них повторный `go mod init` не нужен.

Если не получилось

Симптом	Что проверить
<code>go</code> не найден	Установка завершена? Открыт новый терминал? Найден ли каталог Go через PATH?
<code>go.mod already exists</code>	Модуль уже создан; повторная инициализация не нужна
Отказ в создании папки	Выберите доступную папку пользователя
Редактор показывает другие файлы	Сравните открытую папку с выводом <code>pwd</code> или <code>Get-Location</code>

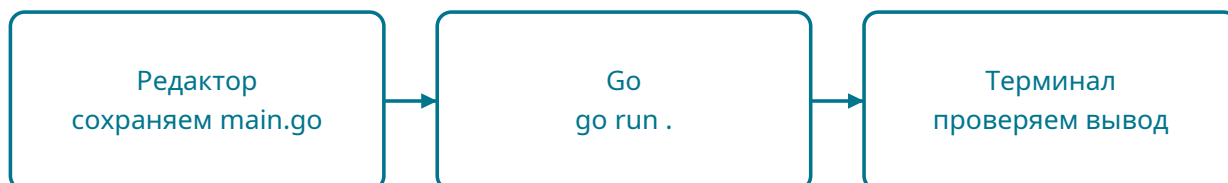
PATH — список папок, где оболочка ищет исполняемые программы. Он похож на список адресов доставки, но саму программу не устанавливает. Если адрес записан верно, а файла по нему нет, поиск всё равно не работает.

Опорные пункты: команда состоит из частей

В `go mod init example.com/bookshop` пробелы отделяют имя программы `go`, подкоманду `mod`, действие `init` и имя модуля. Слеш `/` здесь является частью имени модуля, а не командой перехода в папку. В `go run .` последняя точка — отдельный аргумент «текущий пакет».

Символы приглашения терминала (`$`, `>`, иногда `%`) перед командой не набирают. Они принадлежат оболочке. Круглых или фигурных скобок в этих командах нет: это команды инструменту, а не исходный код Go.

Файл, запуск, наблюдение



Не сохранённый текст не меняет файл на диске

Опора: сохранённый файл, команда Go и наблюдаемый результат

Восстановите по памяти. Закройте пример и назовите три места: где редактируем, что запускаем, где проверяем результат. Если путаются путь и имя модуля, вернитесь к выводу текущей папки и откройте `go.mod`.

Разминка

Предскажите. Если создать файл в папке A, а команду запустить из папки B, обязана ли команда увидеть этот файл?

Заполните. Какая команда меняет текущую папку на `bookshop`?

Почините. Редактор сохранил файл в `bookshop-old`, а терминал открыт в `bookshop`. Как проверить расхождение, не переустанавливая Go?

Задание

Обязательное. Создайте рабочую папку и модуль. Сохраните в `environment.txt` вывод `go version` и путь к папке. Проверьте в редакторе, что рядом существует `go.mod`.

На своих данных. Назовите папку по-своему и повторите проверку пути. Имя папки и имя модуля не обязаны совпадать.

По желанию. Найдите путь к установленной команде Go: `command -v go` в `bash/zsh` или `Get-Command go` в `PowerShell`.

Ответы и самопроверка

Команда работает в своей текущей папке и не обязана искать файл в соседней. Для перехода используется `cd bookshop`. Чтобы найти расхождение, сравните путь терминала с местом сохранения файла в редакторе.

Вы готовы двигаться дальше, если можете объяснить, где лежит проект, показать `go.mod` и получить версию Go. Следующая глава даст этой папке первую программу.

2. Книга появляется в терминале

У нас есть рабочая папка и файл `go.mod`. Теперь напишем программу, которая выводит название книги и три формата её будущего издания. Результат маленький, но проверяемый: две строки в нужном порядке.

Контрольная точка: `examples/02-first-program`. Если начинаете с неё, откройте эту папку и сразу выполняйте команды запуска. Если пишете сами, продолжайте в своей [bookshop](#).

Образ: записка с двумя поручениями

Положите перед собой записку: «Покажи название книги. Затем покажи форматы». Если читать её сверху вниз, порядок понятен. Если поменять поручения местами, сначала появятся форматы. Именно такой небольшой порядок действий мы сейчас зададим программе.

На нашей опорной записке будет три строки: **пакет — нужный инструмент — поручения**. `package main` связывает файл с исполняемым пакетом; `import "fmt"` делает доступным инструмент вывода; внутри `main` стоят два вызова `fmt.Println`. Фигурные скобки можно мысленно обвести рамкой вокруг поручений.

Только не переносите образ дальше нужного: компьютер не понимает просьбы «покажи красиво», пока мы не запишем конкретное действие. И в больших программах порядок может быть сложнее простого чтения сверху вниз. Здесь мы проверяем ровно два последовательных вызова. После запуска спрячьте исходник и восстановите свою записку: что принадлежит пакету, что подключено, что выполняется?

Сразу целиком

Создайте файл `main.go` рядом с `go.mod` и сохраните в нём:

```
package main

import "fmt"

func main() {
    fmt.Println("Go: от первой строки до интернет-магазина")
    fmt.Println("Форматы: PDF, EPUB, HTML")
}
```

Перед запуском запишите предполагаемый вывод. Будут ли видны кавычки? Появится ли пустая строка между названием и форматами?

Для первого запуска используем `go run .` `go` — установленный инструмент, `run` — действие «собрать и сразу выполнить», а отдельная точка `.` — пакет в текущей папке. Пока пакет — наши Go-файлы, лежащие рядом и начинающиеся с `package main`. Сначала сохраните `main.go`, затем в терминале в этой же папке выполните:

```
go run .
```

Результат:

Здесь программа заканчивает работу после двух вызовов печати, и приглашение терминала появляется снова. Это ожидаемое поведение. `go run` не открывает редактор и не следит за изменениями: после каждой сохранённой правки запускайте команду заново. Сравнение с отдельной сборкой приведено ниже в разделе «Запуск и сборка».

Цвет помогает найти знакомое

В книге исходник подсвечен как в редакторе. Та же запись сохраняет свои цвета в предложении: например, `import "fmt"`. Посмотрите сначала на строку программы, затем на её объяснение ниже — вам не придётся заново искать, какая часть относится к языку, а какая содержит данные.

Пример	Цвет и подсказка
<code>package</code>	Оранжевый: объявляем, к какому пакету относится файл
<code>import</code>	Розовый: подключаем другой пакет
<code>func</code>	Синий: объявляем функцию или метод
<code>var, const, type, if, for, return</code>	Фиолетовый: другие ключевые слова; точное действие определяем по самому слову
<code>priceMinor, fmt, Println</code>	Тёмный сине-голубой: имена; их роль определяем по месту в программе
<code>"Книга"</code>	Зелёный: строковые данные внутри кавычек
<code>42</code>	Коричневый: число, записанное прямо в исходнике
<code>// название книги</code>	Серый: комментарий для человека
<code>{, }</code>	Малиново-фиолетовый: границы блока, а позднее — также записи составного типа или значения
<code>(,), [,]</code>	Тёмно-фиолетовый: круглые и квадратные скобки; назначение зависит от места
<code>., ,, :=, +, -</code>	Серый: остальные разделители и операторы

В первой программе можно прочесть цветовую опору так: оранжевый — принадлежность пакету, розовый — подключение, синий — объявление функции, малиново-фиолетовые скобки — границы её тела. Эти цвета повторяются в поясняющих предложениях.

Пока не нужно учить новые слова из таблицы: к каждому мы вернёмся по делу. Зелёная строка `"Книга"` хранит данные. А сине-голубой `priceMinor` — имя, по которому программа обращается к значению. Цвет помогает быстро заметить это различие.

Сине-голубой не означает «это обязательно переменная»: имена пакетов и функций в нашей палитре тоже сине-голубые. Предопределённые имена, например `int64`, `true` и `nil`, тоже сине-голубые: они не являются ключевыми словами Go. Что обозначает каждое из них, разберём при первом использовании. Подсветка разбирает запись, но не доказывает, что программа правильная.

В вашей IDE — редакторе со средствами разработки — цвета могут отличаться: их выбирает тема оформления. У языка Go нет обязательного цвета для переменной или строки. В читалке с собственными настройками и на чёрно-белом экране краски тоже могут измениться или исчезнуть. Кавычки, слова, подписи и наши объяснения сохраняют смысл без цвета.

Попробуйте глазами. Найдите в первой программе слово `import`, затем строку `"fmt"`. Назовите их назначение без слов «розовое» и «зелёное». Если получается, цвет уже работает как подсказка, а не как единственный способ понять запись.

Разбор по строкам

`package main` сообщает, что файл относится к пакету `main`. Из такого пакета можно собрать исполняемую программу. Одного объявления пакета недостаточно: программе нужна функция `main`.

`import "fmt"` подключает пакет стандартной библиотеки для форматирования и вывода. Нам не пришлось отдельно скачивать библиотеку: она поставляется вместе с Go.

`func main()` объявляет функцию с именем `main`. Скобки после имени относятся к параметрам; у нашей функции их нет. Фигурные скобки ограничивают тело функции — команды, которые она выполняет. Подробно параметры разберём в главе 5. Перед `main` Go выполняет инициализацию программы, поэтому запоминать правило «`main` всегда первая строка исполнения» не нужно.

В записи `fmt.Println("Книга")` точка выбирает функцию `Println` из пакета `fmt`, а круглые скобки выполняют вызов с указанным аргументом. Само имя `fmt.Println` без скобок ещё не вызывает функцию. Текст в двойных кавычках — строковое значение. Кавычки отделяют текст программы от её данных, поэтому сами в вывод не попадают. `Println` добавляет перевод строки после вывода.

Внутри `main` команды выполняются последовательно. Поменяете вызовы местами — поменяется и порядок строк. Go пока не знает ни о книге как товаре, ни о форматах как файлах. Мы передали функции печати обычный текст.

Оформление без споров

Введите:

```
gofmt -w main.go
```

`gofmt` приводит код к стандартному оформлению; `-w` записывает результат в файл. Это может изменить отступы. Форматирование не проверяет, правильную ли книгу вы назвали, и не заменяет запуск программы.

Наберите лишние пробелы перед одним вызовом, сохраните и повторите `gofmt`. Затем снова запустите программу. Оформление изменится, результат останется прежним.

Запуск и сборка

У нас два способа проверить один и тот же исходник. **Во время обучения после правки используйте** `go run .`: инструмент соберёт временную программу и сразу запустит её. Это работает одинаково в PowerShell, bash и zsh. Отдельный готовый файл `bookshop` в папке проекта эта команда не оставляет. Предварительно выполнять `go build` не требуется.

Когда нужен сохраняемый исполняемый файл, используется `go build`. Само слово `build` означает сборку, без запуска. Флаг `-o` задаёт имя результата; следующий за ним `bookshop` — выбранное нами имя файла. Последняя точка снова обозначает текущий пакет. Для macOS/Linux:

```
go build -o bookshop .  
./bookshop
```

Для Windows PowerShell:

```
go build -o bookshop.exe .  
.\bookshop.exe
```

Первая команда создаёт программу, вторая её запускает. При успешной сборке `go build` обычно ничего не печатает: проверьте появление файла через `ls` или `Get-ChildItem`. Префиксы `./` и `.\` говорят оболочке запускать файл из текущей папки; `.exe` — обычное расширение исполняемой программы Windows. Это пути файла, а не замена аргументу `.` у инструмента Go.

Что нужно	Команда	Что происходит
Проверить сохранённую правку	<code>go run .</code>	Сборка и немедленный запуск
Получить готовый файл	<code>go build -o bookshop .</code>	Только сборка; для Windows имя <code>bookshop.exe</code>
Запустить уже собранный файл	<code>./bookshop</code> или <code>.\bookshop.exe</code>	Исходники заново не читаются

Если после сборки изменить `main.go`, уже собранный файл сам не изменится: его нужно собрать ещё раз. Проверьте это маленьким опытом: соберите файл, измените название в исходнике и сохраните. Готовый файл покажет старое название, а новый `go run .` — новое. Затем повторите сборку: вывод обоих способов совпадёт.

Не подменяйте команду книги на `go run main.go`. Сейчас результат может совпасть, но такой запуск выбирает только названный файл. Когда появится второй рабочий файл, его объявления не попадут в сборку. `go run .` выбирает пакет целиком; тестовые файлы `_test.go` в обычный запуск не входят. В главе 10 исполняемый пакет переедет в `cmd/shop`, и путь в команде изменится вместе с ним.

Этот пример не требует установки Go для запуска готового файла на совместимой системе. Но файл для macOS нельзя просто запускать как программу Windows: операционная система и архитектура сборки имеют значение. Позже мы также будем отдельно учитывать конфигурацию, данные и внешние сервисы приложения. Сопоставление способов запуска есть и в [официальном уроке Go о сборке](#).

Сломайте безопасно

Удалите закрывающую кавычку у названия книги. Сохраните и запустите `go run` . Сборка завершится ошибкой. Прочитайте имя файла и номер строки; конкретное сообщение может различаться между версиями инструмента.

Верните кавычку и повторите запуск. Мы исправили причину в исходнике. Переустановка Go здесь ничего бы не дала.

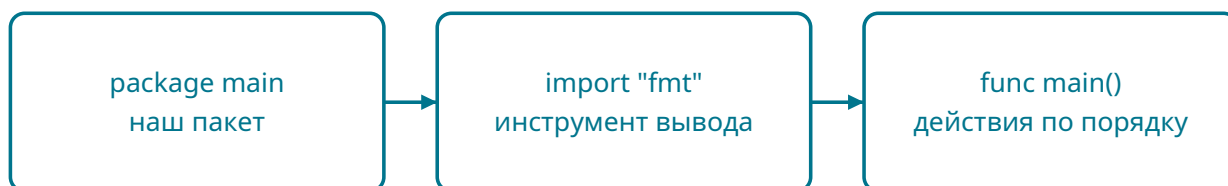
Читаем запись по символам

Запись	Что означает именно здесь
<code>package main</code>	Файл принадлежит пакету с именем <code>main</code>
<code>import "fmt"</code>	Подключаем пакет по пути, записанному строкой
<code>func</code>	Начинается объявление функции
<code>main</code>	Имя функции входа в нашу программу
<code>()</code> после имени в объявлении	Список параметров пуст
<code>{ и }</code>	Начало и конец тела функции
<code>fmt.Println</code>	Точка связывает имя пакета с именем доступной из него функции
<code>("текст")</code> при вызове	Внутри скобок передаётся аргумент — строка
<code>"</code>	Граница строкового литерала; сама кавычка не выводится

Точка в `fmt.Println` и точка в `go run` . выполняют разные роли: первая относится к записи Go, вторая — к аргументам команды терминала. Пробелы и переносы между словами помогают разделить запись, а внутри кавычек являются частью текста.

В Go есть точки с запятой, но в обычном исходнике их чаще не пишут: по правилам языка они автоматически вставляются после определённых токенов в конце строки. Поэтому пока ставьте открывающую `{` функции на той же строке, что и `func main()`. Перенос этой скобки на следующую строку может изменить разбор программы; `gofmt` не исправит любой синтаксический дефект вместо вас.

Программа: принадлежность, инструменты, действия



Скобки ограничивают список параметров и тело функции

Опора первой программы: пакет, подключение и тело `main`

Восстановите по памяти. Нарисуйте три блока, затем напишите программу заново без копирования. Если не помните скобку, найдите её пару и объясните, что она ограничивает.

Разминка

Предскажите. Что изменится, если два вызова `Println` поменять местами?

Заполните. Какое имя пакета должно стоять перед точкой в вызове `____.Println("Книга")`?

Почините. Сообщение говорит `undefined: Fmt`. Найдите разницу между написанным именем и `fmt`.

Задание

Обязательное. Добавьте третью строку `Статус: готовится`. Первые две строки должны остаться прежними. Сохраните файл, отформатируйте и запустите.

На своих данных. Замените название на название собственной будущей книги. Не меняйте список форматов: наш итоговый продукт должен поддерживать PDF, EPUB и HTML.

По желанию. Соберите программу, затем поменяйте название в исходнике. Сравните запуск готового файла и `go run ..` Объясните различие до повторной сборки.

Скажите своими словами

Почему сохранение файла важно перед запуском? Что делает `import`? Почему `go build` не показывает название книги в терминале?

Ответы

Порядок вызовов задаёт порядок строк. Нужен пакет `fmt`, с маленькой буквы: регистр символов различается. Дополнительная строка — ещё один вызов `fmt.Println("Статус: готовится")` внутри `main`, после двух существующих.

Сборка читает сохранённые файлы. `import` делает доступным указанный пакет. `go build` создаёт исполняемый файл и сам его не запускает.

В магазине появилась первая публичная информация — название и форматы. Пока она записана прямо в вызовах печати. В следующей главе отделим данные от действий над ними.

3. Название, цена и состояние книги

Мы уже умеем выводить две строки. Но цена будет участвовать в расчётах, а состояние публикации — определять доступность товара. Поэтому данные должны иметь подходящие типы.

Контрольная точка: `examples/03-values`. В своей рабочей папке замените `main.go` полным примером ниже. Цена учебная: это не объявленная стоимость будущей книги.

Образ: ценник с подписанными местами

Возьмите воображаемый ценник. Сверху оставьте место для названия, ниже — для цены, сбоку — для отметки «показывать покупателю». Не нужно каждый раз переписывать весь ценник, чтобы поменять сумму. Достаточно найти нужное место по подписи.

Имя переменной помогает найти значение в программе. Тип задаёт, какие значения и действия допустимы. Строка названия и целочисленная цена решают разные задачи: мы не станем складывать название с процентом скидки. Изменение переменной меняет её текущее значение; `const` объявляет константу, которой новое значение присвоить нельзя.

На бумаге можно зачеркнуть всё что угодно. Правила языка строже: компилятор проверяет допустимость операций. Поэтому образ ценника помогает связать имя с назначением, а точный ответ о типе мы получаем из Go. Нарисуйте три подписанных места и, не глядя в код, впишите в каждое пример значения. Где можно поставить `false`, а где оно нарушит вашу задумку?

Перед началом: значение, переменная и тип

После главы 2 вы умеете запустить программу и вывести текст. Теперь научимся хранить данные, менять их и отличать ошибку типа от ошибки в расчёте. До цены магазина разберём эти действия по отдельности.

Значение — конкретные данные: число `150`, строка `"Go"`, логическое значение `false`.

Переменная — именованное место хранения значения определённого типа. **Тип** задаёт допустимые значения и операции. Это три разных понятия: `priceMinor` — имя, `int64` — тип, `150` — текущее значение. Изменение значения не меняет тип переменной.

Литерал — запись значения прямо в исходнике. `150` и `"150"` различаются: первый — число, второй — строка из трёх цифр. Компилятор не угадывает, хотите ли вы считать деньги или напечатать номер. Преобразование пользовательского текста в число с возможной ошибкой появится при изучении форм; сейчас работаем с числовыми значениями.

Три способа объявить переменную

Запись внутри <code>main</code>	Что появляется
<code>var quantity int</code>	Переменная типа <code>int</code> с нулевым значением <code>0</code>
<code>var quantity int = 2</code>	Тот же тип с явно указанным начальным значением
<code>quantity := 2</code>	Новая переменная; по этому литералу Go выбирает тип <code>int</code>

Это **альтернативные** записи. Не вставляйте все три объявления одного имени подряд в один блок. В коротком объявлении `:=` двоеточие — часть оператора. Для изменения уже объявленной переменной используйте `quantity = 3`. Запись `quantity := 3` в том же блоке без нового имени приведёт к сообщению `no new variables on left side of :=`.

Блок — группа команд в `{ }`. Пока наши переменные объявлены внутри блока `main` и доступны от своего объявления до конца этого блока. Перед объявлением обратиться к имени нельзя. В главе 4 появятся вложенные блоки условий, а в главе 5 — отдельные функции с собственными параметрами.

Какие типы нужны в начале

Тип	Какие данные хранит	Наше применение
<code>bool</code>	<code>true</code> или <code>false</code>	Опубликована ли книга
<code>string</code>	Последовательность байтов; подробности текста — в главе 6	Название и валюта
<code>int</code>	Знаковое целое; 32 или 64 бита в зависимости от платформы	Количество и индексы
<code>int64</code>	Знаковое целое из 64 бит	Сумма в минимальных денежных единицах
<code>float64</code>	Число с плавающей точкой, обычно приближённое	Измерения, но не цена нашей модели

Есть также `int8`, `int16`, `int32`, беззнаковые `uint8`, `uint16`, `uint32`, `uint64`, `uint` и `float32`. Размер ограничивает диапазон: например, `int8` допускает `-128...127`, а `uint8` — `0...255`. Отсутствие отрицательных значений у беззнакового типа не делает арифметику защищённой от переполнения. `byte` и `rune` — другие имена `uint8` и `int32`; их назначение для текста разберём в главе 6. `uintptr` и комплексные числа в этих упражнениях не используются и не являются предпосылкой следующей главы.

Литерал `2` в `quantity := 2` даёт `int`; `2.5` даёт `float64`, `"Go"` — `string`, `true` — `bool`. Чтобы увидеть выбранный тип, используйте формат `%T` в `fmt.Printf`. `%t` печатает значение `bool`. Это команды форматирования внутри строки, а не объявления типов.

Нулевые значения и копирование

Объявление без правой части не оставляет «случайный мусор»: Go задаёт нулевое значение. Число начинается с `0`, строка — с `" "`, логическое значение — с `false`. Нулевое значение может быть допустимым для языка и недопустимым для задачи: пустое название потребует отклонить отдельно.

Если написать `otherTitle := title`, вторая переменная получает текущее строковое значение. Она не становится автоматически обновляемым вторым именем первой переменной. Когда позднее `title` получит другую строку, `otherTitle` сохранит прежнюю. Это правило о присваивании строковых значений, а не обещание независимости всех будущих составных данных: общий массив среза рассмотрим в главе 7.

Разные числовые типы не складываются сами

Переменная `quantity` типа `int` и `priceMinor` типа `int64` — разные типы. Знак `*` между числами означает умножение. Для произведения явно запишем `int64(quantity) * priceMinor`. Запись `int64(quantity)` — **преобразование типа**, хотя по круглым скобкам похожа на вызов функции. Для нашего количества `2` значение после преобразования остаётся `2`; исходная переменная по-прежнему имеет тип `int`.

Преобразование не является универсальной проверкой корректности. При переводе большого целого в более узкий целый тип информация может потеряться. Строку с цифрами эта запись также не разбирает. Не добавляйте преобразования наугад ради исчезновения ошибки компилятора: сначала определите единицы и допустимый диапазон.

`const percent = 10` объявляет нетипизированную числовую константу. Её можно присвоить `int64`, если значение представимо этим типом. А `const percent int64 = 10` сразу задаёт тип константы. В отличие от переменной константа не получает новое значение. Константное `300` нельзя присвоить `uint8`: компилятор обнаружит выход за диапазон ещё до запуска.

Маленький опыт до ценника

В готовой контрольной точке откройте `labs/01-values/main.go`. `labs` — обычная папка с небольшими опытами, не специальное слово Go. У каждого опыта собственная папка `package main`, но общий `go.mod` этой главы. Поэтому несколько функций `main` из разных папок не конфликтуют. Основной `main.go` магазина не меняйте.

Если набираете сами, создайте указанные вложенные папки рядом с основным `main.go` и сохраните в них полный листинг. Дополнительный `go mod init` не нужен. Терминал остаётся в корне контрольной точки:

```
go run ./labs/01-values
```

Путь после `run` выбирает пакет опыта, а не меняет текущую папку терминала. Команда `go run .` по-прежнему запускает основной пример главы.

```
package main

import "fmt"

func main() {
    var title string
    var priceMinor int64
    var published bool
    fmt.Printf("title=[%s], price=%d, published=%t\n", title, priceMinor, published)
    title = "Go"
    otherTitle := title
    title = "Go shop"
    fmt.Println(title)
    fmt.Println(otherTitle)
}
```

```

quantity := 2
priceMinor = 150
total := int64(quantity) * priceMinor
fmt.Printf("%T %T %d\n", quantity, priceMinor, total)
const percent = 10
var discount int64 = percent
fmt.Println(discount)
}

```

Перед запуском запишите значения обеих переменных с названием после каждого присваивания. Ожидаемый вывод:

```

title=[], price=0, published=false
Go shop
Go
int int64 300
10

```

В первой строке квадратные скобки — просто напечатанные символы, между которыми видна пустая строка. Затем видны две разные строки, два числовых типа и сумма **300**. Последняя строка показывает присваивание константы переменной подходящего типа.

Измените и объясните. Замените количество **2** на **3**: сумма станет **450**, а строка типов не изменится. В копии опыта уберите `int64(...)` вокруг количества: компиляция должна отказать из-за разных типов. Восстановите преобразование. Затем попробуйте присвоить цене строку `"150"` и объясните, почему это другая ошибка того же правила типов.

Проверка перед магазином. Без просмотра листинга объявите нулевую переменную `bool`, присвойте ей `true` и напечатайте. Затем объявите две независимые переменные с одним начальным строковым значением и измените только одну. Если не получается, повторите разделы об объявлении и копировании; исправлять цену магазина пока рано.

Сразу целиком

```

package main

import "fmt"

func main() {
    const currency = "KZT"
    const minorPerUnit = 100
    var priceMinor int64 = 249900
    title := "Go: от первой строки до интернет-магазина"
    published := false

    fmt.Println(title)
    fmt.Printf("Цена: %d.%02d %s\n", priceMinor/minorPerUnit, priceMinor%minorPerUnit, currency)
    fmt.Println("Опубликована:", published)
    published = true
    fmt.Println("Опубликована:", published)
}

```

Запустите `go run` . после того, как попробуете предсказать две последние строки.

```

Go: от первой строки до интернет-магазина
Цена: 2499.00 KZT
Опубликована: false

```

Опубликована: true

Одна переменная участвовала в выводе дважды, и результат различается. Между вызовами мы поменяли её значение.

Значение и его имя

`title := "..."` создаёт переменную `title` и помещает в неё строку. Go определяет тип по правой части. Короткое объявление `:=` применяется внутри функций.

`published := false` создаёт переменную типа `bool`. У логического типа два значения: `true` и `false`. Это удобно для вопроса с двумя ответами: опубликована книга или нет.

`published = true` меняет значение существующей переменной. Здесь стоит `=`, а не `:=`. Для повторного присваивания одному уже объявленному имени новое объявление не требуется.

`var priceMinor int64 = 249900` — более подробная форма: мы явно выбрали тип `int64`. Это знаковое целое фиксированной разрядности. Тип `int`, который часто используют для счётчиков и индексов, зависит от архитектуры. Для представления сумм в модели магазина мы выберем фиксированный целый тип.

Если объявить переменную без начального значения, Go задаст нулевое: для целого числа — `0`, для строки — пустую строку, для `bool` — `false`. Это правило языка, но не бизнес-решение. Не стоит считать нулевую цену автоматически разрешённой, пока магазин не определил правила бесплатных книг.

Константа не меняется

`const currency = "KZT"` задаёт константу. Ей нельзя присвоить другое значение во время работы программы. В этом примере валюта одна, поэтому она неизменна.

Это не означает, что любой магазин обязан иметь одну валюту. Когда появится несколько валют, код валюты станет частью данных, а правила пересчёта и округления придётся определить отдельно.

`minorPerUnit` задаёт число минимальных денежных единиц в основной для нашего учебного представления KZT: сто. Имя помогает увидеть смысл числа в выражении. Число `100` без имени могло бы означать и процент, и ограничение длины, и количество товаров.

Почему цена целая

Сумму `2499.00 KZT` мы храним как `249900` минимальных единиц. Такой способ позволяет выполнять расчёты целыми числами и явно решать, что делать с дробной минимальной единицей.

Тип `float64` полезен для приближённых вычислений, измерений и многих научных задач. Но многие десятичные дроби не представимы в двоичной форме точно. Для денежных правил нам важна воспроизводимая единица учёта, а не приближение, которое случайно выглядит правильно после печати.

Целое число тоже имеет предел. Сам по себе переход на `int64` не защищает от переполнения при умножении. В главе 5 мы ограничим входную цену до расчёта и проверим границы

тестами. Сумму и валюту позже объединим в модель, чтобы не перепутать единицы.

Как получилась точка и два нуля

В выражении `priceMinor / minorPerUnit` целочисленное деление даёт целую часть. В выражении `s %` получаем остаток. Для `249900` это `2499` и `0`.

`fmt.Printf` печатает по шаблону. В нашем шаблоне:

Запись	Значение
<code>%d</code>	Целое десятичное число
<code>%02d</code>	Целое число шириной не менее двух позиций, с ведущим нулём
<code>%s</code>	Строка
<code>\n</code>	Перевод строки

Поэтому остаток `0` становится `00`, а остаток `5` стал бы `05`. В отличие от `Println`, `Printf` не добавляет перевод строки автоматически: он записан в шаблоне.

Этот вывод рассчитан на неотрицательную цену. Он не является готовым форматировщиком отрицательных сумм или всех валют. Мы отделим такие правила, когда появится необходимость.

Читаем новую запись по символам

Запись	Как прочитать
<code>:=</code>	Объявить переменную и дать начальное значение; двоеточие вместе с равно — один оператор
<code>=</code>	Присвоить новое значение уже существующей переменной
<code>const</code>	Объявить константу
<code>var</code>	Объявить переменную
<code>int64, string, bool</code>	Имена типов, то есть допустимых видов значений
<code>249900</code>	Целочисленный литерал без кавычек
<code>false, true</code>	Логические значения без кавычек
<code>,</code> в <code>Printf(...)</code>	Разделитель аргументов одного вызова
<code>/, %</code> между числами	Деление и остаток от целочисленного деления
<code>%d</code> внутри кавычек	Часть формата <code>Printf</code> , а не оператор остатка

Одни и те же видимые знаки читаются по месту. `%` снаружи строки участвует в арифметике Go; `%02d` внутри строки получает смысл, когда `Printf` читает шаблон. Обратный слеш в `\n` начинает управляющую последовательность строкового литерала, обозначающую перевод строки; это не два печатаемых символа.

Данные книги имеют разные типы

<code>title: string</code> название	<code>priceMinor: int64</code> целая сумма	<code>published: bool</code> да или нет
--	---	--

Тип определяет допустимые значения и операции

Опора данных: название, целая сумма и состояние публикации

Проверка понимания. Покажите в примере присваивание, которое изменяет данные, и место, которое только задаёт вид вывода. Если % пока означает для вас одно и то же в обоих местах, перечитайте разбор цены.

Разминка

Предскажите. Как будет выглядеть цена при `priceMinor = 105`?

Заполните. Для изменения опубликованной книги на черновик используйте `published ____ false`.

Почините. Кто-то написал `priceMinor = "249900"`. Почему это не то же самое, что число `249900`?

Задание

Обязательное. Установите цену `350050`. Программа должна напечатать `Цена: 3500.50 KZT`. Вторая строка состояния должна оставаться `Опубликована: true`.

На своих данных. Возьмите положительную цену, у которой последние две цифры — `05`, и предскажите вывод перед запуском.

По желанию. Попробуйте присвоить константе `currency` другое значение. Прочитайте ошибку и отмените изменение.

Скажите своими словами

Чем отличается объявление от присваивания? Почему сумма без валюты неполна? Почему целый тип не освобождает нас от проверки диапазона?

Ответы

`105` минимальных единиц даст `1.05 KZT`. Для присваивания нужен `=`. Значение в кавычках имеет строковый тип, а `priceMinor` объявлена целой: это несовместимые типы для прямого присваивания.

В обязательном задании меняется только начальное значение цены. Не исправляйте шаблон вывода под конкретный ответ: он должен работать и для предыдущей цены.

У нашей книги появились данные трёх видов. Следующий шаг — принимать решение по этим данным, а не всегда печатать одно и то же.

4. Какие издания можно показать

Книга может быть черновиком, а одно из старых изданий — снятым с продажи. Напишем небольшую проверку, которая обходит три номера изданий и сообщает, какие доступны.

Контрольная точка: `examples/04-conditions`. Это упражнение над состоянием каталога. Оно не удаляет купленные книги из библиотеки покупателя: правила доступа к уже приобретённому изданию будут отдельными.

Набираете сами: в отдельной папке главы сохраните свой `go.mod` и создайте `main.go` из полного листинга ниже. Это опыт с условиями; цену из главы 3 сохраняем в предыдущей папке, а не удаляем из истории проекта.

Образ: дверь и три карточки

На столе лежат три карточки изданий. Перед витриной — дверь с условием: «Показывать только опубликованное». Сначала проверяем, открыта ли дверь для нашей книги. Затем берём карточки по одной. Второе издание снято с продажи — откладываем его и берём следующее.

В программе `if` выбирает действие по условию, а `for` повторяет шаги. `continue` в нашем обходе означает: закончить работу с текущей карточкой и перейти к следующему шагу цикла. `break` означает другое: завершить этот цикл целиком. Представьте, что после второй карточки вы либо тянетесь за третьей, либо убираете всю стопку. Разницу теперь можно увидеть до запуска.

Дверь не является отдельным объектом программы: это рисунок выбора. Само условие должно дать логическое значение. Нарисуйте дверь и три маленьких прямоугольника. Зачеркните второй, но оставьте третий на витрине. Какой переход нужен для такого результата? Проверьте свой ответ на нашем цикле.

Сначала выбор без повторения

Условие отвечает на вопрос, а ветвление выбирает, какой блок выполнить. В главе 3 мы только последовательно меняли значения; теперь программа может пропустить часть команд. Условие `if` обязано иметь тип `bool`. Число `1` и непустая строка сами по себе не заменяют `true`.

У одной проверки могут быть две ветви: `if` для истинного условия и `else` для ложного. Цепочка `else if` задаёт следующий вопрос, только когда предыдущий дал ложь. Выполняется первый подходящий блок, а не все подходящие блоки подряд. Для независимых действий используют отдельные `if`.

Сравнения `==`, `!=`, `<`, `<=`, `>` и `>=` дают логическое значение. `=` присваивает, поэтому `quantity = 0` не является вопросом «количество равно нулю». Отрицание `!` меняет истину на ложь и наоборот.

A	B	A && B: оба истинны	A B: хотя бы одно истинно
false	false	false	false

A	B	A && B: оба истинны	A B: хотя бы одно истинно
false	true	false	true
true	false	false	true
true	true	true	true

&& не вычисляет правую часть, если левая ложна. || не вычисляет правую часть, если левая истинна. Это сокращённое вычисление: например, проверка ненулевого делителя должна стоять **до** деления, справа от &&. Скобки помогают показать нужную группировку; не меняйте порядок условий, если правой части нужна гарантия левой.

Один вариант из нескольких: switch

`switch status` сравнивает значение `status` с вариантами после `case`. При совпадении выполняется соответствующий блок. `default` — запасная ветвь, если совпадений нет. В нашем примере это неизвестное состояние. После обычного `case` Go сам выходит из переключателя: писать `break` в конце каждой ветви не нужно. Это отличается от выхода из цикла, который рассмотрим ниже.

Сначала используем именно эту форму переключателя значений. Переключатель **типов** интерфейса — другая конструкция; он появится только после объяснения интерфейсов. Редкая команда `fallthrough` здесь не нужна.

Полная маленькая программа, не изменяющая основной пример главы:

```
package main

import "fmt"

func main() {
    published := true
    quantity := 0
    if !published {
        fmt.Println("Черновик")
    } else if quantity == 0 {
        fmt.Println("Нет экземпляров")
    } else {
        fmt.Println("Можно выбрать")
    }
    fmt.Println(published && quantity > 0)
    fmt.Println(published || quantity > 0)
    status := "draft"
    switch status {
    case "draft":
        fmt.Println("Готовится")
    case "published":
        fmt.Println("На витрине")
    default:
        fmt.Println("Неизвестное состояние")
    }
}
```

Папки опытов устроены как в главе 3. Из корня контрольной точки запустите:

```
go run ./labs/01-choices
```

```
Нет экземпляров
false
true
Готовится
```

Здесь `published` истинно, но количество равно нулю. Поэтому выбран второй блок цепочки, выражение с И ложно, выражение с ИЛИ истинно. `switch` работает с отдельной строкой `status`, а не с результатом прежнего условия.

Проверьте без угадывания. Сделайте количество равным `2`: первая строка изменится на `Можно выбрать`, следующие две будут `true`. Затем установите `published = false`: первой станет строка `Черновик`. Для `switch` задайте неизвестное состояние и получите `Неизвестное состояние`. Эти изменения проверяют разные решения, а не заменяют одно другим.

Где видна переменная

Переменная, объявленная внутри фигурных скобок ветви, недоступна после этой ветви. Если результат нужен после `if`, объявите его до ветвления и используйте внутри `=`. Новое `:=` во вложенном блоке может создать другое имя и **скрыть** внешнюю переменную: внешнее значение тогда не изменится. Это называется затенением.

Не путайте область видимости с временем жизни данных. Здесь мы говорим только о том, в какой части исходника разрешено использовать имя. Возвращение адресов и замыкания разберём после функций и указателей.

Затем повторение без вложенных решений

Go использует одно ключевое слово `for` для нескольких форм цикла:

Форма	Когда удобно
<code>for i := 1; i <= 3; i++ { ... }</code>	Есть счётчик, граница и регулярный шаг
<code>for remaining > 0 { ... }</code>	Действие повторяется, пока условие истинно
<code>for { ... }</code>	Условие выхода проверяется внутри; без выхода цикл не завершится сам

Многоточие в этой таблице обозначает место тела, а не код для копирования. В полном листинге ниже его нет. Цикл `for` с условием проверяет его **перед** телом: возможны ноль проходов. Изменение счётчика должно приближать остановку. Запись `remaining = remaining - 1` вычитает единицу и сохраняет результат в той же переменной. В форме с одним условием отдельного автоматического шага нет — в опыте `remaining` меняет само тело.

```
package main

import "fmt"

func main() {
    for edition := 1; edition <= 3; edition++ {
        fmt.Println("Издание", edition)
    }
}
```

```

    }
    remaining := 2
    for remaining > 0 {
        fmt.Println("Осталось", remaining)
        remaining = remaining - 1
    }
    number := 0
    for {
        number++
        if number == 2 {
            continue
        }
        if number == 4 {
            break
        }
        fmt.Println("Выбрано", number)
    }
}

```

```
go run ./labs/02-loops
```

```

Издание 1
Издание 2
Издание 3
Осталось 2
Осталось 1
Выбрано 1
Выбрано 3

```

Сначала выполняются три прохода первого цикла. Во втором после печати **1** значение становится нулём, следующая проверка ложна. В третьем увеличение стоит перед проверками: номер **2** пропускается через **continue**, номер **4** завершает цикл через **break**. Поэтому остаются строки с **1** и **3**. Программа не содержит бесконечно работающего варианта.

Трассировка на бумаге. Для второго цикла запишите три проверки: **2 > 0**, **1 > 0**, **0 > 0**. Последняя не запускает тело. Поменяйте начальное значение на **0** и предскажите, какие две строки исчезнут. Затем верните **2**.

Проверка перед соединением. Напишите отдельно выбор «черновик / опубликована» и отдельно цикл, печатающий **1**, **2**, **3**. Объясните, почему **continue** оставляет возможность следующего прохода, а **break** её прекращает. Только затем соединяйте эти действия в обходе изданий магазина.

Сразу целиком

```
package main

import "fmt"

func main() {
    published := true
    for edition := 1; edition <= 3; edition++ {
        if !published {
            fmt.Println("Книга готовится")
            break
        }
        if edition == 2 {
            fmt.Println("Издание", edition, "снято с продажи")
            continue
        }
        fmt.Println("Доступно издание", edition)
    }
}
```

Сначала отметьте на бумаге, какие строки сработают для номера 2. Затем запустите `go run ..`

```
Доступно издание 1
Издание 2 снято с продажи
Доступно издание 3
```

Число 2 не потерялось: у него другой результат. Мы явно сообщили, что издание снято с продажи.

Условие — вопрос с логическим ответом

`if edition == 2` проверяет равенство. Если условие истинно, выполняется блок в фигурных скобках. Один знак `=` присваивает значение; два знака `==` сравнивают значения. Перепутать их легко, поэтому читайте выражение словами: «номер издания равен двум?».

`!published` означает «не опубликована». Знак `!` меняет логическое значение на противоположное. Если книга ещё не опубликована, нет смысла показывать доступные издания этого учебного каталога.

Условия могут использовать `<`, `<=`, `>`, `>=` и `!=`. У равенства и сравнения тоже есть ограничения типов: не все значения языка сравниваются любым оператором. Сейчас мы работаем с целыми числами и логическим значением.

Цикл: начало, проверка, шаг

В заголовке `for edition := 1; edition <= 3; edition++` три части.

Первая создаёт счётчик со значением 1 перед обходом. Вторая проверяется перед каждым выполнением тела: можно ли продолжать? Третья увеличивает счётчик на единицу после очередного завершённого прохода.

Получается последовательность: 1, 2, 3. Когда счётчик станет равен 4, условие `4 <= 3` окажется ложным, и цикл закончится. Переменная `edition` объявлена в заголовке цикла и недоступна после него. Это пример области видимости: имя существует не во всех местах

файла.

`edition++` — отдельная команда увеличения, а не выражение, которое надо вкладывать в печать. Здесь она управляет переходом к следующему изданию.

Два разных способа прекратить работу

`continue` завершает текущий проход цикла. В нашем случае при номере **2** строка «Доступно издание» пропускается. Для такого `for` затем выполняется шаг `edition++`, проверяется условие и начинается следующий проход.

`break` выходит из цикла целиком. Если изменить `published` на `false`, программа один раз напечатает **Книга готовится** и прекратит обход.

Представьте проверку полки. `continue` — перейти к следующей книге на полке. `break` — прекратить проверку этой полки. Но в программе границу определяет именно цикл, а не любое место, которое кажется нам «следующей задачей».

Проверим границы

Измените `edition <= 3` на `edition < 3`. Третье издание исчезнет из вывода. Это не сбой инструмента: условие больше не разрешает проход с номером **3**.

Верните `<=`. Затем установите начальное значение **4**. Тело не выполнится ни разу, потому что условие проверяется до первого прохода.

Наконец, верните исходник и попробуйте убрать `edition++`, сохранив обе точки с запятой. Счётчик перестанет меняться, и при опубликованной книге программа будет повторять сообщение. Остановите её `Ctrl+C` и восстановите шаг. Не оставляйте такой вариант работающим: пользы от повторного вывода нет.

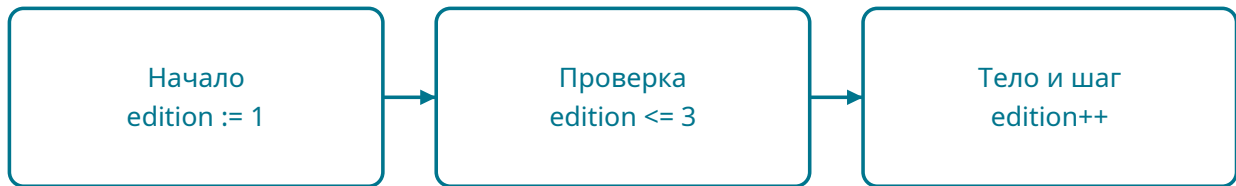
Читаем условие и цикл по символам

Запись	Роль
<code>for</code>	Начало цикла
<code>;</code> в заголовке <code>for</code>	Отделяет начало, условие и шаг
<code><=</code>	Меньше или равно; два знака образуют один оператор
<code>++</code>	Увеличить переменную на единицу; отдельная команда
<code>if</code>	Выполнить блок, если условие истинно
<code>!</code>	Логическое отрицание
<code>==</code>	Проверка равенства; это не присваивание
<code>break</code>	Закончить этот цикл
<code>continue</code>	Закончить текущий проход и перейти к следующему

Фигурные скобки вложенных блоков закрываются в обратном порядке: сначала внутренний `if`, потом внешний `for`, потом `main`. Отступы помогают это видеть; если потерялись,

временно подпишите закрывающие скобки в заметках. Круглые скобки вокруг условия `if` Go не требует.

Маршрут цикла for



После шага снова проверяем условие; false завершает цикл

Опора цикла: начало, проверка, тело и шаг

Пройдите маршрут пальцем. Для номеров 1, 2 и 3 укажите, какие условия сработают и где заканчивается проход. Сначала без запуска, затем сравните с выводом.

Разминка

Предскажите. Сколько строк появится при `published := false`?

Заполните. Какой оператор оставляет номера до трёх включительно: `<` или `<=`?

Почините. У снятого издания удалили `continue`, и теперь оно одновременно «снято» и «доступно». Объясните, почему выполняются обе строки.

Задание

Обязательное. Обойдите номера от 1 до 4 включительно. Снятым должно быть издание 3.

Ожидаемый вывод:

```
Доступно издание 1
Доступно издание 2
Издание 3 снято с продажи
Доступно издание 4
```

На своих данных. Выберите другой снятый номер внутри диапазона. Перед запуском перечислите ожидаемые строки.

По желанию. Снимите два издания с помощью двух отдельных условий. Затем используйте разобранный в этой главе оператор `||` и объедините проверку. Сравните читаемость.

Ответы

Для черновика будет одна строка. Включённая правая граница записывается через `<=`. Без `continue` программа после сообщения о снятии идёт дальше и печатает обычную строку этого же прохода.

В обязательном задании верхняя граница меняется на 4, а проверка снятого номера — на `edition == 3`. Начальное значение остаётся 1.

Мы научились повторять действие и выбирать его по данным. Теперь вынесем самостоятельное правило — расчёт скидки — в функцию и закрепим ожидаемое поведение тестами.

5. Правило цены и первый тест

Цена из главы 3 пока только выводилась на экран. Добавим скидку. Мы хотим один раз записать правило расчёта и вызывать его из разных мест, а не копировать арифметику по всему магазину.

Контрольная точка: `examples/05-functions`. Для этой главы нужны условия, целочисленное деление и понимание переменной. Структуры и коллекции пока не требуются.

Набираете сами: подготовьте отдельную папку с вашим `go.mod`, `main.go` ниже и `main_test.go` из раздела о первом тесте. В готовой контрольной точке `main_test.go` уже существует: добавляйте новый тест туда, не дублируя имена функций.

Образ: карточка расчёта

Представьте карточку кассира с двумя пустыми местами: «цена» и «скидка». Ниже записаны шаги расчёта, а внизу оставлено место для результата. Карточку можно применить к разным книгам; переписывать правило каждый раз не требуется.

У нашей функции два входных значения — параметры. Из вызова в неё поступают аргументы: конкретная цена и конкретный процент. Функция возвращает новую цену и признак того, что исходные данные допустимы. Если процент больше ста, нельзя молча заполнить нижнюю строку случайным числом и назвать расчёт удачным.

Опора: **два входа — проверка — два выхода**. Это описание именно нашей функции. Вообще функция Go может иметь другое число параметров и результатов, а некоторые функции меняют внешнее состояние. Не всякая функция похожа на спокойный расчёт по карточке. Сейчас нам удобно начать с правила, которое легко проверить отдельно. Закройте код и пройдите по карточке с ценой **100** и скидкой **10**: что окажется на каждом выходе?

Что происходит при вызове функции

Именованная функция объявляет участок программы с определёнными параметрами и результатами. Функции без имени тоже существуют; их введём отдельно в главе 9.

Объявление записывает, что будет происходить. **Вызов** запускает эти действия. Наличие объявления выше `main` не означает, что его тело выполнится раньше `main`: обычная функция ждёт вызова.

Начнём с двух более простых случаев, прежде чем одновременно вводить скидку, отказ и тест:

Объявление	Параметры	Результат	Вызов
<code>func announce(title string)</code>	Одна строка	Нет возвращаемого значения	<code>announce("Go")</code>
<code>func total(priceMinor int64, quantity int64) int64</code>	Два целых числа	Одно целое число	<code>total(150, quantity)</code>

Тело каждой функции заключено в фигурные скобки; полные тела приведены ниже. Тип после списка параметров относится к результату. Вызов функции без результата, например

`announce`, нельзя присвоить переменной как число. Печать в терминал и возврат значения — разные действия.

Параметр получает значение аргумента

`priceMinor` и `quantity` в объявлении `total` — параметры. Число `150` и значение переменной `quantity` в её вызове — аргументы. Порядок важен: первый аргумент попадает в первый параметр. Внутри функции типы уже известны из объявления; заново объявлять параметры через `:=` не нужно.

Входные значения копируются. У параметра собственное имя в блоке функции. Если присвоить параметру новое число, переменная вызывающего кода не изменится. Одинаковое написание `quantity` в двух функциях не соединяет их переменные.

`return expression` завершает текущий вызов и отдаёт вычисленное значение вызывающему. Исполнение продолжается после вызова. Команды после выполненного `return` в этой функции уже не выполняются. У функции без результатов возможен просто `return`; у `main` тоже нет возвращаемого значения. Для функции с результатом каждый нормально завершающийся путь должен этот результат предоставить.

Полный опыт: печать, результат и копия параметра

```
package main

import "fmt"

func announce(title string) {
    fmt.Println("Книга:", title)
}

func total(priceMinor int64, quantity int64) int64 {
    return priceMinor * quantity
}

func changeQuantity(quantity int64) {
    quantity = 10
    fmt.Println("Внутри:", quantity)
}

func main() {
    announce("Go")
    var quantity int64 = 2
    amount := total(150, quantity)
    fmt.Println("Сумма:", amount)
    changeQuantity(quantity)
    fmt.Println("Снаружи:", quantity)
}
```

Из корня контрольной точки:

```
go run ./labs/01-functions
```

```
Книга: Go
Сумма: 300
Внутри: 10
Снаружи: 2
```

Проследим один вызов: `main` передаёт `150` и `2` → параметры получают эти значения → `return` отдаёт `300` → переменная `amount` получает результат → `Println` печатает его. Внутри `total`

печати нет.

Другой вызов меняет только свой параметр: `changeQuantity` выводит `10`, но последняя строка показывает прежнее внешнее `2`. Это не ошибка: функция не обещает изменить вызывающего. Позже указатель позволит передать доступ к исходному значению; само правило передачи аргументов по значению не исчезнет.

Измените. Передайте в `total` количество `3`; ожидайте `450`. Затем верните исходник и замените печать `amount` на ещё один вызов `total(150, quantity)`: функция вычислится снова. Она не выполняется автоматически, когда вы лишь упоминаете её имя без `()`.

Самостоятельно. Напишите функцию `double(value int64) int64`, возвращающую удвоенное число без печати. Вызовите её для `7` и напечатайте результат в `main`. Ожидается `14`. Подсказка: умножение должно быть после `return` в теле `double`, а `fmt.Println` — в вызывающей функции. Если функция печатает `14`, но вы не можете присвоить результат переменной, вы перепутали вывод с возвратом.

От одного результата к двум

Скидка может дать нулевую цену, а может получить недопустимые аргументы. Одного числа недостаточно, чтобы различить эти случаи. Поэтому следующая функция возвращает **пару**: число и логический признак. Запись `(int64, bool)` описывает результаты по порядку; `return 0, false` возвращает два значения в том же порядке.

В `price, ok := priceAfterDiscount(...)` правая часть сначала вычисляется целиком, затем результаты присваиваются двум переменным. `ok` не является ключевым словом: это выбранное имя `bool`. Результат, который сознательно не нужен, можно принять в `_`; это пустой идентификатор, к его «значению» потом обратиться нельзя. Отбрасывать признак ошибки при расчёте цены нельзя только ради более короткой строки.

Сейчас не требуются рекурсия, переменное число аргументов или функции, возвращающие другие функции. Их отсутствие в этом опыте не означает запрета языка; они вводятся отдельными шагами, когда для них подготовлены предпосылки.

Сначала договоримся о результате

Наше учебное правило принимает цену в минимальных денежных единицах и целый процент скидки. Цена допустима от `0` до `100000000`, процент — от `0` до `100`, обе границы включены. Ограничение цены выбрано для упражнения и безопасного расчёта, а не как универсальный предел торговли.

Мы сначала считаем размер скидки. Если получается дробная минимальная единица, отбрасываем дробную часть скидки. Например, 10% от `105` — это `10.5`; скидка будет `10`, а покупатель заплатит `95`. Это конкретное правило округления в пользу продавца. Другой магазин может выбрать другое, но обязан описать и проверить его.

При недопустимых аргументах функция возвращает признак отказа. Возвращаемое число в этом случае нельзя использовать как цену.

Сразу целиком

Файл `main.go`:

```

package main

import "fmt"

// priceAfterDiscount принимает цену в минимальных денежных единицах.
// Допустимая цена: от 0 до 100000000 включительно; скидка: от 0 до 100.
// Дробную минимальную единицу в скидке отбрасываем в пользу продавца.
func priceAfterDiscount(priceMinor int64, percent int64) (int64, bool) {
    if priceMinor < 0 || priceMinor > 100000000 {
        return 0, false
    }
    if percent < 0 || percent > 100 {
        return 0, false
    }
    discountMinor := priceMinor * percent / 100
    return priceMinor - discountMinor, true
}

func main() {
    priceMinor, ok := priceAfterDiscount(249900, 10)
    if !ok {
        fmt.Println("Неверная цена или скидка")
        return
    }
    fmt.Println("К оплате:", priceMinor, "минимальных единиц KZT")
}

```

Запустите `go run .:`

К оплате: 224910 минимальных единиц KZT

Две величины на выходе помогают различить бесплатную книгу и ошибку. Для допустимой нулевой цены получим `(0, true)`, для недопустимой — `(0, false)`.

Параметры и аргументы

В объявлении функции `priceMinor` и `percent` — параметры: имена для входных значений. В вызове `priceAfterDiscount(249900, 10)` числа — аргументы, переданные этим параметрам.

Функция не печатает ответ. Она возвращает его вызывающему коду. Благодаря этому одно правило можно будет использовать в терминале, HTTP-обработчике и тесте без захвата вывода с экрана.

`(int64, bool)` после списка параметров описывает два возвращаемых значения. `return 0, false` немедленно завершает вызов. Если проверки прошли, вычисляется скидка и возвращаются цена и `true`.

В строке `priceMinor, ok := ...` оба результата получают имена. `ok` — привычное имя для признака успеха. Сначала проверяем его и только потом используем цену. Позже, когда потребуется объяснить разные причины отказа, заменим логический признак на значение `error`.

Область видимости параметра ограничена функцией. Имя `priceMinor` в `main` — другая переменная, хотя написано одинаково. Передача целого числа копирует его значение; функция не изменяет переменную вызывающего кода.

Почему проверки стоят до умножения

Операторы `||` соединяют условия значением «или». Цена не подходит, если она меньше нуля **или** превышает верхнюю границу. Процент проверяется отдельно.

Максимальное произведение в нашем договоре — `100000000 * 100`, то есть `10000000000`. Оно помещается в `int64`. Произвольную большую цену мы не умножаем: функция откажет раньше.

Переставить проверку после вычисления опасно. Если арифметика уже переполнилась, проверка искажённого результата не восстановит исходный смысл. Ограничения полезны только тогда, когда реально охватывают путь вычисления.

Проверка, которая остаётся

Создайте рядом `main_test.go`. Минимальный первый тест выглядит так:

Прежде чем читать листинг: тестовый инструмент создаёт объект отчёта `testing.T` и передаёт тесту доступ к нему через параметр `t *testing.T`. Звёздочка здесь обозначает указатель, а `t.Fatalf(...)` — вызов метода, то есть функции, связанной с этим объектом. В этом упражнении вы используете готовый объект для сообщения о провале, не создаёте свой тип и не меняете эту сигнатуру. Собственные указатели и методы будем строить в главе 8. Ниже подробно разберём каждую проверку теста.

```
package main

import "testing"

func TestPriceAfterDiscount(t *testing.T) {
    got, ok := priceAfterDiscount(249900, 10)
    if !ok || got != 224910 {
        t.Fatalf("получили (%d, %t), хотели (224910, true)", got, ok)
    }
}
```

`go test` собирает и запускает проверки из файлов `_test.go`. Аргумент `./...` выбирает пакеты в текущей папке и её вложенных папках внутри этого модуля: `./` — относительный путь, `...` — шаблон инструмента Go. Три точки здесь надо набрать; это не пропущенный текст. Пока пакет один, но та же команда сохранится после разделения проекта. Выполняйте её из папки с `go.mod`:

```
go test ./...
```

В успешном отчёте будет строка `ok` с именем модуля. `FAIL` означает провал проверки или сборки; сначала прочитайте сообщение над ним. `[no test files]` означает отсутствие тестов, а не проверенный результат. Обычный `go run` выполняет функцию `main`, но не запускает `TestPriceAfterDiscount`; поэтому нужны обе команды.

Время и отметка кеша могут различаться. `(cached)` означает повторное использование подходящего успешного результата. Для принудительного повторного выполнения используется `go test -count=1 ./...: -count=1` просит выполнить тесты один раз заново.

Имя файла заканчивается на `_test.go`: инструмент Go включает такие файлы при тестировании. `TestPriceAfterDiscount` начинается с `Test`, а параметр `t *testing.T` даёт тесту способы сообщить о провале. Пока используйте эту сигнатуру как договор инструмента;

указатель и методы разберём отдельно. Звёздочка здесь не участвует в формуле скидки.

`got` означает полученный результат. Условие проверяет и число, и признак успеха. `t.Fatalf` помечает тест неуспешным, печатает сообщение и прекращает этот тест. `t.Fatal` тоже прекращает тест, но принимает готовые значения без строки формата; суффикс `f` у `Fatalf` указывает на форматирование. `%d` выводит число, `%t` — логическое значение. В сообщении указано и полученное, и ожидаемое: это помогает разбирать ошибку.

Убедитесь, что тест умеет падать

В функции замените `priceMinor - discountMinor` на `priceMinor + discountMinor`. Это правдоподобная ошибка знака: скидка увеличит цену. Запустите тест и найдите сообщение о несовпадении.

Верните минус и снова выполните тест. Один успешный запуск полезен только вместе с пониманием того, какую ошибку проверка действительно замечает.

Не вычисляйте ожидаемый результат в тесте той же формулой, которую проверяете. Иначе опечатка может переехать в обе половины, и сравнение подтвердит их общую ошибку.

Края важнее одного удачного числа

Цена	Скидка	Результат	Успех
249900	0	249900	true
249900	100	0	true
105	10	95	true
0	10	0	true
-1	10	не использовать	false
100	101	не использовать	false

В полной контрольной точке есть также проверки отрицательной скидки, предельной цены и цены на единицу выше предела. Тесты не доказывают правильность для всех мыслимых программ, но сохраняют важные примеры нашего договора.

Читаем функцию и тест по символам

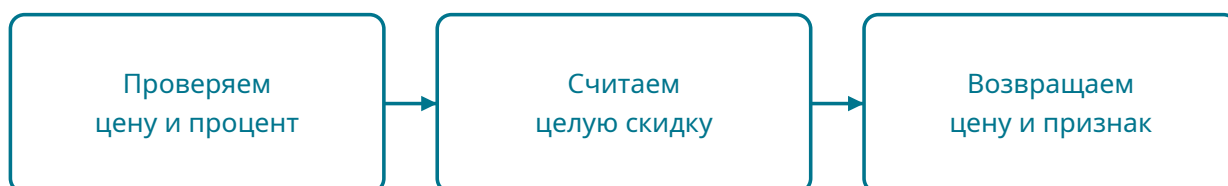
Запись	Как прочитать
//	Однострочный комментарий: текст до конца строки не исполняется
(priceMinor int64, percent int64)	Два параметра; после каждого имени указан его тип
(int64, bool) после параметров	Два возвращаемых значения, в этом порядке
return	Завершить функцию и передать результаты вызывающему коду
	Логическое «или»: достаточно истинности одного условия
*, - в формуле	Умножение и вычитание
!= в тесте	Не равно
_ вместо имени результата	Пустой идентификатор: этот результат сознательно не используем
t *testing.T	Параметр t содержит указатель на объект тестирования типа T из пакета testing
t.Fatalf	Вызов метода объекта тестирования через точку

Звёздочка между числами означает умножение. Звёздочка перед типом `testing.T` означает тип указателя: значение, через которое тест обращается к своему объекту отчёта. Это разные грамматические места. Объект создаёт тестовый инструмент; руками выделять его сейчас не нужно. В главе 8 вы сами создадите значение и проверите, как адрес помогает сохранить изменение.

В `if !ok || got != 224910` сначала читаем «успеха нет» и «число не совпало», затем соединяем через «или». Логическое `||` вычисляет правую часть только если левая ложна. При вычислении арифметики умножение и деление имеют приоритет перед вычитанием; операции одного такого приоритета группируются слева направо. Скобки позволяют явно изменить группировку.

`%t` в сообщении выводит логическое значение, `\n` внутри строки задаёт перевод строки. Комментарий описывает договор, но компилятор не доказывает, что код ему соответствует: для этого мы пишем проверки.

Расчёт с явным договором



При отказе расчёт не продолжается

Опора функции: проверяем аргументы, считаем скидку, возвращаем результат

Проверка понимания. Назовите два разных значения `*` в этой главе и покажите место, где ошибка прекращает вычисление до умножения.

Разминка

Предскажите. Что вернёт `priceAfterDiscount(105, 100)`?

Заполните. После получения `price`, `ok` можно ли печатать цену до проверки `ok`?

Почините. Тест проверяет только, что цена равна нулю. Почему он не отличит бесплатную книгу от отказа?

Задание

Обязательное. Напишите тест `TestSmallDiscount`: цена `105`, скидка `1`; ожидаются `104` и `true`. Сначала вычислите результат на бумаге, затем запустите тест. Не меняйте функцию, чтобы подогнать её под ошибочное ожидание.

На своих данных. Выберите цену и процент, при которых размер скидки получается дробным. Объясните итог по нашему правилу округления.

По желанию. Измените правило: округляйте размер скидки вверх до целой минимальной единицы. Сначала сформулируйте новые ожидания для `105` и `10%`, затем измените отдельную копию функции и тесты. Основную контрольную точку сохраните с прежним договором.

Ответы

При стопроцентной скидке на допустимую цену получим `(0, true)`. До проверки `ok` число не следует считать ценой. В тесте бесплатной книги надо проверить и ноль, и признак успеха.

Для обязательного задания скидка равна $105 * 1 / 100$, то есть `1`, а итог — `104`. По форме тест похож на первый: меняются аргументы, ожидаемое число и имя.

Теперь в проекте есть самостоятельное правило и автоматическая проверка. В следующей главе займёмся названием книги: выясним, почему длина строки в байтах не равна числу видимых букв.

6. Сколько букв в названии книги

Название книги должно одинаково хорошо работать с русским, казахским и английским текстом. Если проверять его длину неподходящим способом, короткое название может неожиданно не пройти ограничение, а обрезанный текст — перестать читаться.

В этой главе мы научимся измерять строку и введём правило названия: убрать пробельные символы по краям, проверить кодировку и ограничить длину. Контрольная точка: `examples/06-strings`.

Набираете сами: подготовьте отдельную папку с вашим `go.mod`, файлами `main.go` и `title.go` ниже; `title_test.go` возьмите из контрольной точки. Тесты скидki оставьте в папке главы 5. Здесь исследуем отдельное правило; в главе 8 соединим название и цену.

Образ: мозаика и её кусочки

Посмотрите на короткое название `ӘGo`. Глаз легко различает три буквы. Теперь представьте, что каждую букву собирают из маленьких кусочков, а размер коробочки измеряют числом кусочков, а не числом букв. В записи UTF-8 латинским `G` и `o` нужен один байт каждой, а `Ә` — два.

Поэтому `len("ӘGo")` даст `4`: он считает байты строки. Обход `range` декодирует руны — кодовые точки Unicode. Для этого названия их три, а начальные байтовые позиции — `0, 2, 3`. Нарисуйте четыре клеточки и объедините первые две одной дугой. Так легче вспомнить, почему следующий символ начинается с позиции два.

Мозаика — только опора различия единиц счёта. Руна тоже не всегда равна одной видимой букве: букву с отдельным комбинируемым знаком можно представить несколькими кодовыми точками. Поэтому не запоминайте «руна — это буква» без оговорки. Каждый раз уточняйте, что требуется посчитать: байты, кодовые точки или воспринимаемые человеком символы.

Сначала строка и отдельная руна

Для этой главы нужны переменные, вызовы функций и цикл из глав 3–5. Новое — представление текста. Прежде чем писать проверку названия, отделим три вопроса: что хранится, как это записано в исходнике и что видит человек.

`"ӘGo"` — строковый литерал в двойных кавычках. `string` хранит последовательность байтов. `'Ә'` — литерал руны в одинарных кавычках: одна кодовая точка Unicode. В `var letter rune = 'Ә'` явно выбран тип `rune`, другое имя `int32`. Это число, которому Unicode сопоставляет символ; сама руна не является строкой длины один.

У символа `Ә` кодовая точка U+04D8, то есть число `1240`. UTF-8 записывает её двумя байтами; значения байтов и номер кодовой точки — разные числа. `%c` показывает символ по номеру руны, `%d` — номер числом. Поэтому результат зависит и от значения, и от формата печати.

Байт состоит из восьми бит и может хранить 256 значений — от 0 до 255. В Go `byte` является другим именем `uint8`. Это единица хранения, не обещание «одна буква». Значение `text[0]` — байт с индексом ноль; полный символ UTF-8 может занимать несколько байтов.

Кавычки и управляющие последовательности

В обычной строке `"Go\nshop"` последовательность `\n` означает один перевод строки, а `\t` — табуляцию. `\` позволяет записать двойную кавычку, `\\` — обратный слеш. В необработанной строке между обратными кавычками обратный слеш сам по себе не начинает такую последовательность. Это полезно, например, для многострочного текста. Кавычки ограничивают литерал и не входят в значение.

Формат `%q` показывает значение в кавычках и делает управляющие символы заметными. Когда на экране видны `\n` или `\\n`, не делайте вывод по числу печатаемых знаков: `%q` специально показывает экранированное представление.

Обход текста: индекс не номер буквы

`range` — форма обхода, связанная с `for`. В записи `for offset, value := range text` на каждом проходе появляются два значения: байтовое смещение начала символа и руна. Типы этих переменных — `int` и `rune`. Для корректной строки UTF-8 переход к следующей руне может пропускать больше одного байта. Если смещение не нужно, вместо его имени используют `_`.

Полный опыт:

```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    text := "ᐅGo"
    var letter rune = 'ᐅ'
    fmt.Println(len(text), utf8.RuneCountInString(text))
    fmt.Printf("%T %c %d\n", letter, letter, letter)
    fmt.Printf("Первый байт: %d\n", text[0])
    for offset, value := range text {
        fmt.Printf("%d %c\n", offset, value)
    }
    fmt.Printf("%q\n", "Go\nshop")
    fmt.Printf("%q\n", `Go\nshop`)
    old := text
    text = "Go" + " shop"
    fmt.Println(old, text)
}

go run ./labs/01-text
```

```
4 3
int32 ᐅ 1240
Первый байт: 211
0 ᐅ
2 G
3 o
"Go\nshop"
"Go\nshop"
ᐅGo Go shop
```

4 3 означает четыре байта и три руны, а не противоречивые ответы на один вопрос. Номер руны 1240 не равен первому байту 211. Обход посещает смещения 0, 2, 3. Затем %q позволяет сравнить настоящий перевод строки с двумя символами «обратный слеш и n» в необработанном литерале.

Последняя строка опыта показывает ещё одно различие. Переменной `text` можно присвоить **новую** строку, например результат `"Go" + " shop"`. Но изменить отдельный байт имеющегося строкового значения через `text[0] = ...` нельзя: строки неизменяемы. Переменная `old` сохраняет прежнее значение.

Измените и объясните. Замените `ᐃGo` на `Go`: получите два байта, две руны и смещения 0, 1. Не заменяйте число ожидаемых рун числом байтов в будущей проверке названия. Пустую строку можно считать и обходить, но обращаться к её элементу `[0]` нельзя. Такой опыт с индексом завершится паникой — аварийным прекращением обычного выполнения; сначала проверяйте `len(text) > 0`.

Проверка перед правилом названия. Объясните четыре разные записи: `"ᐃ"`, `'ᐃ'`, `len("ᐃ")` и `utf8.RuneCountInString("ᐃ")`. Ответ: строка, руна, два байта и одна руна. Если ответом везде получилось «одна буква», повторите опыт: иначе ограничение длины далее будет выглядеть необъяснимым.

Сначала наблюдение

В контрольной точке два рабочих файла: `main.go` запускает опыт, а `title.go` содержит правило. Оба относятся к одному пакету и лежат рядом с `go.mod`. Команда `go run` соберёт их вместе.

Файл `main.go`:

```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    title, ok := normalizeTitle("  Шаңырақ Go ")
    if !ok {
        fmt.Println("Название не подходит")
        return
    }
    fmt.Printf("Название: %q\n", title)
    fmt.Println("Байтов:", len(title))
    fmt.Println("Рун:", utf8.RuneCountInString(title))
    for offset, letter := range "ᐃGo" {
        fmt.Printf("Байт %d: %c\n", offset, letter)
    }
}
```

Пока не запускайте: попробуйте посчитать длину `Шаңырақ Go`. В слове семь букв, затем пробел и ещё две буквы. Почему у программы будет два разных числа?

Файл `title.go`:

```

package main

import (
    "strings"
    "unicode"
    "unicode/utf8"
)

// normalizeTitle возвращает однострочное название без краевых пробелов.
// Лимит измеряется в рунах, а не в графемах или байтах.
func normalizeTitle(title string) (string, bool) {
    if !utf8.ValidString(title) {
        return "", false
    }
    title = strings.TrimSpace(title)
    if title == "" || utf8.RuneCountInString(title) > 80 {
        return "", false
    }
    for _, letter := range title {
        if unicode.IsControl(letter) || letter == '\u2028' || letter == '\u2029' {
            return "", false
        }
    }
    return title, true
}

```

Теперь выполните `go run .`:

```

Название: "Шаңырақ Go"
Байтов: 17
Рун: 10
Байт 0: Ә
Байт 2: G
Байт 3: о

```

И 17, и 10 верны: они отвечают на разные вопросы.

Байт, руна и то, что видит читатель

Байт — единица из восьми бит. В Go `byte` является другим именем типа `uint8`, то есть беззнакового целого от 0 до 255. Строка хранит последовательность байтов. Её длина `len(title)` измеряется именно в байтах.

UTF-8 — способ представить кодовые точки Unicode байтами. ASCII — небольшой набор кодов для латинских букв, цифр, знаков и управляющих символов. Его символы, например `G`, в UTF-8 занимают один байт. Для букв в нашем казахском примере — по два. Поэтому семь букв дают четырнадцать байтов, а пробел и `Go` добавляют ещё три.

Руна в Go — другое имя типа `int32`; при работе с Unicode ею представляют кодовую точку. Функция `utf8.RuneCountInString` считает такие кодовые точки. Не путайте это с обещанием посчитать видимые знаки.

Один воспринимаемый знак может состоять из нескольких кодовых точек: например, буква и отдельно записанное ударение. Такие последовательности называют графемными кластерами. Ограничение в восемьдесят рун — понятное правило нашей учебной модели, но оно не равнозначно ограничению в восемьдесят видимых знаков.

Что возвращает range

В цикле `for offset, letter := range "ӘGo"` первое значение — смещение в байтах, второе — руна. Смещения получились `0, 2, 3`. После `Ә` мы перешли сразу к байту с номером `2`, потому что эта буква занимает два байта.

Если вместо обхода написать `title[0]`, мы получим первый байт. Это может быть только часть многобайтовой последовательности. Для нашей строки такой байт нельзя считать первой буквой.

Пустая строка тоже имеет длину — ноль. Но элемента с индексом `0` у неё нет. Проверка длины до обращения по индексу нужна независимо от языка текста.

Зачем проверять UTF-8

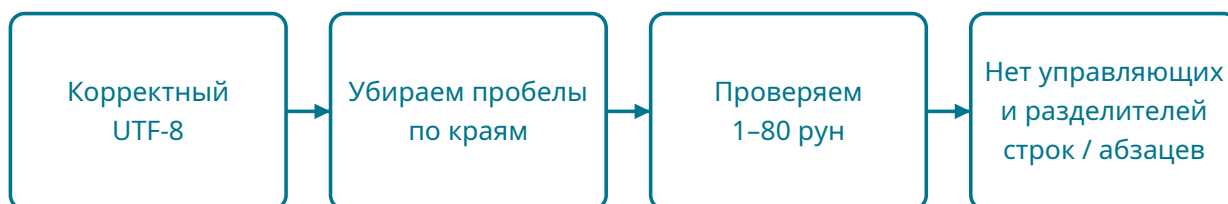
Исходные файлы Go записываются в UTF-8, но строка, полученная во время работы, может содержать произвольные байты. Например, файл мог прийти в другой кодировке или быть повреждён.

`utf8.ValidString` отвечает, является ли последовательность корректным UTF-8. В нашем правиле неверная последовательность приводит к отказу. Мы не пытаемся угадать исходную кодировку и не выдаём заменённые символы за первоначальное название.

Это отдельная проверка от подсчёта рун. Сам по себе успешный вызов счётчика не подтверждает, что исходные байты были корректным текстом.

Четыре шага проверки

Название: проверка до сохранения



Любой отказ — название не принимаем

Карта проверки названия: UTF-8, пробелы, длина и управляющие символы

Сначала проверяем UTF-8. Затем `strings.TrimSpace` убирает пробельные символы с обоих краёв. Пробел между словами остаётся на месте.

После удаления краевых пробелов название не должно быть пустым, а его длина в рунах — превышать восемьдесят. В конце обходим оставшийся текст и отказываем при управляющих символах, например переводе строки внутри названия. Отдельно исключаем U+2028 (разделитель строки) и U+2029 (разделитель абзаца): `unicode.IsControl` проверяет категорию управляющих символов, а эти два разделителя относятся к другим категориям Unicode.

Обратите внимание на порядок. Строка из пробелов и переводов строк превратится в пустую и получит отказ. Название `Go` с переводом строки только в конце станет `Go`: краевые пробелы мы согласились удалять. А `Go`, перевод строки, `магазин` останется многострочным и будет отклонено.

Эта функция не выполняет полную Unicode-нормализацию и не защищает от всех визуально похожих символов. Её имя означает подготовку названия по перечисленным правилам. Когда понадобится поиск, мы отдельно обсудим сравнение и нормализацию Unicode, чтобы не менять авторское написание без объяснения.

Строка не меняется внутри себя

Функция `TrimSpace` возвращает строку. Мы сохраняем её в переменной `title` внутри `normalizeTitle`. Это новое присваивание локальному имени, а не изменение отдельных байтов переданного значения.

Поэтому вызывающий код получает подготовленное название через `return`. Если проигнорировать возвращённую строку, исходная переменная у вызывающего кода не станет автоматически обрезанной.

Проверяем границу, а не впечатление

В `title_test.go` есть проверка строки из восьмидесяти букв `Ә`. Её создаёт `strings.Repeat("Ә", 80)`: первый аргумент — повторяемая строка, второй — число повторов. Она занимает сто шестьдесят байтов, но должна пройти наш лимит в рунах. Следующая проверка добавляет ещё одну такую букву и ожидает отказ.

Запустите `go test ./...`. Затем замените в функции подсчёт рун на `len(title)` и повторите тесты. Граница для казахского текста сломается. Верните исходную проверку и снова запустите тесты.

Это полезнее проверки только короткого английского слова: для него число байтов и рун совпадает, и неправильная реализация могла бы выглядеть правильной.

Читаем новую запись по символам

`import (...)` объединяет несколько импортов в группу; скобки здесь не вызывают функцию. В `for offset, letter := range text` запятая разделяет два имени, а `range` задаёт обход строки. Когда первое значение не нужно, вместо `offset` ставят `_`.

В выводе `%q` показывает строку в кавычках с экранированием при необходимости, `%c` выводит символ по значению руны. Путь `"unicode/utf8"` внутри импорта содержит слеш: это часть пути пакета, а не арифметическое деление.

В тестах `\t` означает табуляцию, `\n` — перевод строки, `\xff` задаёт один байт по шестнадцатеричному коду. Последняя запись позволяет намеренно построить неверный UTF-8. Запись `\u0301` задаёт кодовую точку Unicode; язык сам кодирует её в UTF-8 внутри строки. Такие последовательности полезны для опытов, где обычный вид текста скрыл бы различие.

`'\u2028'` и `'\u2029'` в одинарных кавычках — литералы рун: каждый задаёт одну кодовую точку. В двойных кавычках `"\u2028"` была бы строка. Мы сравниваем `letter` с рунами через

`==`, а не со строками.

Проверка понимания. Не называя просто «длину», произнесите полное условие нашего ограничения: сколько чего мы считаем и что делаем с пробелами по краям.

Разминка

Предскажите. Сколько байтов и рун у строки `Go`? А у `ᐅ`?

Заполните. Индекс в `for offset, letter := range title` измеряется в байтах или в рунах?

Почините. Ограничение названия обещает восемьдесят рун, но функция проверяет `len(title) <= 80`. Какой пример покажет ошибку?

Задание

Обязательное. Допишите тест `TestInternalSpaces: normalizeTitle(" Go дүкені ")` должна вернуть `"Go дүкені"` и `true`. Между словами два пробела; наша функция их не объединяет.

На своих данных. Возьмите название на знакомом вам языке, предскажите число рун и сравните с результатом. Если ожидание разошлось с выводом, сначала уточните, не включает ли видимый знак несколько кодовых точек.

По желанию. Сравните строки `"é"` и `"e\u0301"`: напечатайте их длины в байтах, длины в рунах и результат `==`. Не исправляйте сравнение удалением ударения: это меняет данные.

Ответы

У `Go` два байта и две руны; у `ᐅ` — два байта и одна руна. `range` возвращает байтовое смещение. Восемьдесят букв `ᐅ` обнаружат ошибку проверки через `len`: рун восемьдесят, а байтов сто шестьдесят.

В обязательном тесте сравните оба результата: подготовленный текст и признак успеха. Для дополнительного опыта два варианта `é` занимают соответственно два и три байта, содержат одну и две руны и не равны как строки. Они могут выглядеть одинаково, но последовательности байтов различны.

У магазина появилось правило названия. В следующей главе соберём несколько названий в упорядоченный каталог.

7. Каталог: список, массив и словарь

До сих пор программа работала с одной книгой. Теперь их будет несколько, и порядок на витрине станет частью результата. Для этого нам понадобятся три вида данных: фиксированный набор форматов, список идентификаторов и словарь названий.

Контрольная точка: `examples/07-catalog`. Книга о SQL — учебная карточка для второго элемента каталога, а не обещание готового второго издания в магазине shanraq.org.

Набираете сами: подготовьте отдельную папку с вашим `go.mod`, новыми `main.go` и `catalog.go`, перенесите `title.go` и `title_test.go` из главы 6 и добавьте `catalog_test.go` из контрольной точки. Названия продолжают проходить прежнюю проверку.

Образ: ряд карточек и окно

Разложите три карточки в ряд. Теперь прикройте ряд листом бумаги с прямоугольным окном: видны только две карточки. Передвинуть границу окна и переписать карточку — разные действия. Если два окна открывают одну карточку, исправление на ней будет видно через оба.

Такой образ помогает разобраться со срезом: он описывает участок базового массива. Копия среза сама по себе не создаёт независимые копии элементов. А вот копирование массива копирует его элементы. Для независимого набора элементов среза мы явно выделим новое хранилище и скопируем туда данные.

У окна есть важная граница сходства. `append` может использовать оставшееся место старого массива, а может выделить новый; это зависит от вместимости. Поэтому после добавления нельзя судить об общем хранилище только по похожим названиям переменных. Нарисуйте два окна над одним рядом, затем отдельный ряд для копии. В проверяемом примере измените одну карточку и предскажите, через какое окно изменение станет видно.

А `map` представьте отдельным набором карточек «ключ — название». Этот набор помогает найти название по ключу, но не обещает порядок витрины. Для порядка у нас остаётся собственный список ключей.

Три задачи — три отдельных опыта

Перед соединением каталога проверим каждый контейнер отдельно. Нужны типы, `for`, `range`, функции и проверка названия. На этом этапе не требуется знать структуры или указатели: они появятся в следующей главе. В готовой папке есть три независимые программы в `labs`; основной пример магазина пока не меняйте.

Массив: известное количество элементов

Индекс — позиция элемента, отсчитываемая от нуля. В массиве длины 3 допустимы индексы 0, 1 и 2. `len` массива возвращает число элементов, а не последний индекс. `[3]string` — тип, `[3]string{"PDF", "EPUB", "HTML"}` — значение этого типа, записанное составным литералом.

```
package main

import "fmt"
```

```

func main() {
    formats := [3]string{"PDF", "EPUB", "HTML"}
    other := formats
    other[0] = "TXT"
    fmt.Println(len(formats), formats[0], other[0])
    for index, format := range formats {
        fmt.Println(index, format)
    }
}

```

```
go run ./labs/01-arrays
```

```

3 PDF TXT
0 PDF
1 EPUB
2 HTML

```

Присваивание `other := formats` копирует все три строковых элемента. Замена `other[0]` не заменяет `formats[0]`. `range` для массива даёт индекс и значение, в отличие от строки, где первый результат обозначал байтовое смещение. Мы не увеличивали длину массива: `[3]string` и `[4]string` остаются разными типами.

Опыт с отказом. В копии программы попробуйте обратиться к постоянному индексу `formats[3]`: компилятор отклонит выход за границу. Если индекс приходит из переменной во время исполнения, недопустимое значение приведёт к панике. Проверка должна ограничивать индекс условием `0 <= index && index < len(formats)`. Верните рабочий вариант перед продолжением.

Срез: длина, ёмкость и общий массив

`[]string` — тип среза. Срез описывает участок базового массива: начало, текущую длину и доступную ёмкость. Присваивание среза копирует это описание, а не создаёт новый массив. Два среза могут обращаться к тем же элементам. Это существенно отличается от только что проверенного присваивания массива строк.

`make([]string, length, capacity)` создаёт срез с заданными длиной и ёмкостью. Когда третий аргумент опущен, ёмкость равна длине. Длина определяет доступные индексы сейчас; ёмкость — возможность расширения в этом массиве. Ёмкость 3 при длине 0 не разрешает запись в элемент `[0]`.

`append` возвращает описание увеличенного среза: сохраните его. При достаточной ёмкости используется прежний массив, при недостаточной создаётся другой. Поэтому `append` не гарантирует независимой копии. `copy` копирует элементы в уже существующий срез-приёмник; количество ограничено длинами обоих срезов и возвращается как результат.

```

package main

import "fmt"

func main() {
    titles := []string{"Go", "SQL"}
    same := titles
    same[0] = "Web"
    fmt.Println(titles[0])
}

```

```

independent := make([]string, len(titles))
copied := copy(independent, titles)
independent[0] = "Other"
fmt.Println(copied, titles[0], independent[0])
empty := make([]string, 0, 3)
fmt.Println(len(empty), cap(empty), copy(empty, titles))
empty = append(empty, "Go")
fmt.Println(len(empty), empty[0])
base := [3]string{"Go", "SQL", "Web"}
part := base[:1]
grown := append(part, "New")
fmt.Println(base[1], len(part), len(grown))
}

go run ./labs/02-slices

```

```

Web
2 Web Other
0 3 0
1 Go
New 1 2

```

Разберите результат по действиям:

1. `same[0] = "Web"` меняет общий элемент; через `titles[0]` видим `Web`.
2. Для `independent` выделено новое хранилище, скопированы два элемента; замена его строки не меняет `titles`.
3. Приёмник `empty` имеет ёмкость 3, но длину 0: `copy` переносит ноль элементов.
4. После сохранённого `append` длина равна 1, и индекс 0 доступен.
5. `base[:1]` видит один элемент, но может расширяться в прежнем массиве. `append(part, "New")` заменяет `base[1]`. Длина старого `part` остаётся 1, а возвращённого `grown` равна 2.

Двоеточие в `base[low:high]` задаёт диапазон с включённой левой и **исключённой** правой границей. Пропущенный `low` означает 0, пропущенный `high` — текущую длину. Здесь используем диапазон внутри текущей длины. Возможность расширения среза до ёмкости требует отдельного контроля границ; она не разрешает обычный доступ `[index]` за длиной.

`var titles []string` даёт nil-срез: его длина и ёмкость равны нулю. В него можно добавлять через `titles = append(titles, "Go")`. Пустой созданный срез тоже может иметь длину ноль, но не быть nil. Ни nil-срез, ни пустой срез не содержат элемента с индексом 0.

Самостоятельно. Измените получатель копии на `make([]string, 0, len(titles))`. Предскажите `copy`: вернётся 0, потому что длина равна нулю. Не выполняйте следующую запись `[0]` у пустого среза; исправьте длину приёмника. Это проверка смысла `len`, а не повод ловить панику вместо правильного создания среза.

Словарь: наличие ключа важнее позиции

`map[string]int` связывает строковый ключ с целым значением. Запись `quantities["go"] = 2` создаёт или заменяет значение этого ключа. Запрос отсутствующего ключа возвращает нулевое значение типа элемента — здесь 0. Чтобы отличить отсутствие от действительно записанного нуля, используем два результата: `value, exists := quantities["go"]`.

В словаре нет «первого» элемента витрины. Порядок обхода не гарантирован. Ключ должен поддерживать сравнение на равенство: строка подходит, срез — нет. Один ключ связан с одним текущим значением; повторное присваивание тому же ключу не создаёт вторую строку.

```
package main

import "fmt"

func main() {
    var uninitialized map[string]int
    fmt.Println(uninitialized["go"])
    quantities := make(map[string]int)
    quantities["go"] = 0
    value, exists := quantities["go"]
    missing, found := quantities["sql"]
    fmt.Println(value, exists, missing, found)
    quantities["go"] = 2
    same := quantities
    same["go"] = 3
    fmt.Println(quantities["go"])
    delete(quantities, "go")
    _, exists = quantities["go"]
    fmt.Println(exists, len(quantities))
    quantities["sql"] = 1
    clear(quantities)
    fmt.Println(len(quantities), quantities == nil)
}
```

```
go run ./labs/03-maps
```

```
0
0 true 0 false
3
false 0
0 false
```

Из nil-map можно читать, но записывать в неё нельзя: программа запаникует.

`make(map[string]int)` создаёт рабочий словарь. Копирование значения map, как `same := quantities`, сохраняет доступ к общим записям, поэтому изменение через `same` видно через `quantities`.

`delete(quantities, "go")` удаляет один ключ. Если его нет, ничего не происходит. `clear(quantities)` удаляет все записи, но не превращает созданную map в nil; в неё по-прежнему можно записывать. Обе функции встроены в язык и не требуют импорта. Значений для присваивания они не возвращают.

Проверка перед каталогом. Не глядя в код, объясните, почему две прочитанные величины могут равняться 0, но один признак присутствия истинен, а другой ложен. Затем выберите контейнер для трёх задач: фиксированные три формата; меняющийся упорядоченный список ID; поиск названия по ID. Ответ: массив, срез, map. Это выбор для нашего опыта, не обязательная архитектура любого магазина.

Сразу целиком

Файл `main.go`:

```

package main

import "fmt"

func main() {
    formats := []string{"PDF", "EPUB", "HTML"}
    fmt.Println("Форматы:", formats)

    ids := []string{"go-shop"}
    ids = append(ids, "sql-notes")
    titles := map[string]string{
        "go-shop": "Go: от первой строки до интернет-магазина",
        "sql-notes": "Заметки о SQL",
    }
    ordered, ok := catalogTitles(ids, titles)
    if !ok {
        fmt.Println("Каталог неполон")
        return
    }
    for index, title := range ordered {
        fmt.Println(index+1, title)
    }
    _, found := titles["missing"]
    fmt.Println("Неизвестная книга найдена:", found)
}

```

Файл catalog.go:

```

package main

// catalogTitles сохраняет порядок ids и отказывает при неполных данных.
func catalogTitles(ids []string, titles map[string]string) ([]string, bool) {
    ordered := make([]string, 0, len(ids))
    for _, id := range ids {
        title, ok := titles[id]
        if !ok {
            return nil, false
        }
        title, ok = normalizeTitle(title)
        if !ok {
            return nil, false
        }
        ordered = append(ordered, title)
    }
    return ordered, true
}

```

Запустите `go run .:`

```

Форматы: [PDF EPUB HTML]
1 Go: от первой строки до интернет-магазина
2 Заметки о SQL
Неизвестная книга найдена: false

```

Первая книга в списке — наша книга. Словарь помогает получить название по идентификатору, а список задаёт последовательность показа.

Массив: длина входит в тип

`[3]string` означает массив из трёх строк. Число стоит между квадратными скобками и является частью типа: `[2]string` и `[3]string` — разные типы.

Мы сразу записали три значения: `PDF`, `EPUB`, `HTML`. К первому обращаются по индексу `0`, к последнему — по `2`. Индекса `3` у этого массива нет.

Массив выбран для демонстрации фиксированного набора. Это не утверждение, что вся архитектура магазина навсегда обязана хранить форматы в массиве. Когда у разных книг будет разный набор файлов, понадобится другая модель.

При присваивании массива другому массиву копируются элементы. Для массива строк можно поменять элемент копии, не заменяя строку в исходном массиве. Но если сами элементы содержат ссылки на общие данные, копирование внешнего массива не делает эти данные независимыми автоматически.

Срез: видимая часть массива

`[]string` означает срез строк. Числа между скобками нет. Начальный срез `ids` содержит один идентификатор. Вызов `append(ids, "sql-notes")` возвращает срез с добавленным элементом, и мы сохраняем этот результат обратно в `ids`.

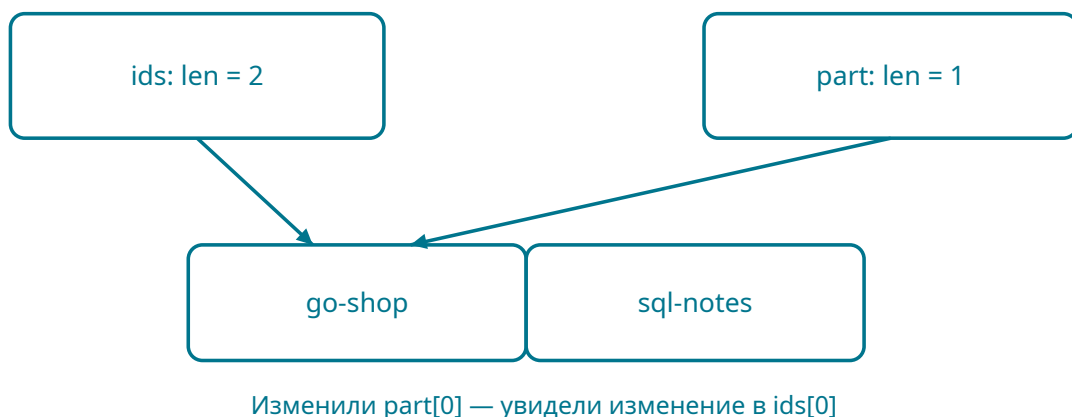
Почему возвращённое значение важно? В срезе есть информация о начале данных, длине и доступной ёмкости. Добавление меняет длину, а иногда требует другого массива. Если не сохранить результат `append`, у вас не будет правильного описания расширенного среза.

`len` отвечает, сколько элементов сейчас видно. `cap` — сколько элементов доступно от начала этого среза до конца доступной ему области массива. Для среза, начинающегося в середине массива, это не обязательно размер всего исходного массива.

`make([]string, 0, len(ids))` создаёт срез с нулевой длиной и заранее достаточной ёмкостью для нашего результата. Длина ноль означает, что обращаться к `ordered[0]` пока нельзя. Сначала нужно добавить элемент.

Общее хранилище: важная граница образа

Два среза могут видеть один элемент



Два среза могут смотреть на один массив; изменение элемента видно обоим

Срез удобно представлять окном в массив. Но два окна могут показывать одни и те же ячейки. Выражение `part := ids[:1]` не обещает независимую копию первого элемента.

Здесь левая граница пропущена и равна нулю, правая не включается: виден только элемент с индексом `0`. Если присвоить `part[0]` новое значение, оно будет видно через `ids[0]`.

Для независимого списка строк можно создать срез нужной длины через `make` и вызвать встроенную функцию `copy`. Она копирует столько элементов, сколько помещается в оба среза, и возвращает число скопированных элементов. Если создать приёмник длиной ноль, ничего не скопируется, даже при большой ёмкости.

Копирование списка строк достаточно для независимой замены его строковых элементов. Для среза структур с вложенными `map` или другими срезами вопрос независимости придётся рассматривать глубже.

Не забывайте конкретный коэффициент роста `cap` после `append`: наш код не должен зависеть от стратегии выделения памяти. Проверять нужно наличие элементов и требуемую независимость данных.

Словарь: ключ вместо позиции

В `map[string]string` первый тип относится к ключу, второй — к значению. Ключ `go-shop` связывается с названием книги.

Операция `title, ok := titles[id]` возвращает значение и признак присутствия ключа. Для неизвестного ключа строка будет пустой, а `ok` — ложным. Если ключ существует и хранит пустую строку, `ok` будет истинным. Поэтому функция сначала проверяет наличие ключа, затем вызывает `normalizeTitle` из главы 6: проверяются UTF-8, краевые пробелы, длина и однострочность. В результат попадает подготовленное название. Полезное правило не должно исчезать при переходе от одной книги к нескольким.

Нулевое значение `map` — `nil`. Из неё можно читать, получая отсутствие ключа. Записывать в `nil` `map` нельзя: нужен созданный словарь, например через `make(map[string]string)` или литерал, как в нашем примере. Это связано с начальным значением, а не со словом `var`: запись `var titles = make(map[string]string)` создаёт рабочий словарь.

При присваивании `map` другой переменной данные словаря не копируются в независимое хранилище. Изменение элемента через одно имя будет видно через другое. Подробнее проверим это вместе со структурами.

Почему витрина не обходит `map`

Порядок обхода `map` не определён. Даже если несколько запусков случайно дали удобную последовательность, полагаться на неё нельзя.

Поэтому мы обходим `ids`. Для каждого идентификатора получаем соответствующее название. Если данных не хватает, возвращаем `nil`, `false` и не показываем частично собранный результат как полноценный каталог.

Два отдельных контейнера требуют согласованности. Можно добавить идентификатор и забыть название. В этой главе мы намеренно делаем такой риск видимым; следующая объединит поля книги в структуру и сократит число мест, которые нужно менять вместе.

Словарь полезен для поиска по ключу, но образ «мгновенного гардероба» не означает нулевого времени или гарантированной одинаковой стоимости любой операции. Производительность измеряют на задаче и данных, а не выводят из метафоры.

Проверяем поведение

`catalog_test.go` проверяет заданный порядок, отсутствующий ключ, пустое название и пустой каталог. Запустите `go test ./...`

Пустой каталог считается допустимым: список результатов пуст, признак успеха истинен. Каталог с обещанным идентификатором, для которого не найдено название, — ошибка. Эти случаи отличаются смыслом, хотя оба могут не дать ни одной строки на витрине.

Читаем коллекции по символам

Запись	Как прочитать
<code>[3]string</code>	Массив ровно из трёх строк
<code>[]string</code>	Срез строк
<code>{"PDF", "EPUB", "HTML"}</code> после типа	Значения составного литерала
<code>[0]</code> после переменной	Обращение к элементу по индексу
<code>[: 1]</code> после среза	Диапазон от начала до индекса 1, не включая его
<code>map[string]string</code>	Словарь: строки в качестве ключей и значений
<code>"go-shop": "Название"</code>	Двоеточие разделяет ключ и значение записи словаря
<code>make, append, copy, len, cap</code>	Встроенные функции языка; отдельный импорт не нужен
<code>nil</code>	Нулевое значение для среза, тар и ряда других типов, но не для строки или целого числа

В многострочном составном литерале после последнего элемента тоже стоит запятая. Она нужна для такой записи с переносом перед закрывающей скобкой. Не переносите правило на список аргументов в одну строку бездумно: здесь мы объясняем конкретный многострочный пример.

`range ordered` даёт индекс и значение элемента среза. Выражение `index+1` только меняет печатаемый номер: индекс самого среза по-прежнему начинается с нуля. Формат `%v` в сообщении теста показывает значение в обычном представлении.

Восстановите по памяти. Нарисуйте словарь как карточки «ключ — название» и отдельно список ключей для витрины. Проведите путь от первого элемента списка к его названию. Если хочется взять «первую запись тар», вернитесь к правилу порядка обхода.

Разминка

Предскажите. Сколько элементов видит срез после `make([]string, 0, 10)`?

Заполните. Как сохранить результат добавления: `ids = ____(ids, "new-book")`?

Почините. `copy` вернул ноль, хотя у приёмника ёмкость десять. Какое второе свойство приёмника нужно проверить?

Задание

Обязательное. Добавьте третью карточку с идентификатором `go-tests` и названием `Тесты на Go`. Она должна появиться третьей. Затем удалите только её название из `map`: программа должна вывести `Каталог неполон`, а не первые две карточки как будто всё получилось.

На своих данных. Поменяйте порядок идентификаторов. Убедитесь, что меняется порядок витрины, а пары в словаре переписывать не нужно.

По желанию. Создайте срез из двух строк, получите `part := original[:1]`, поменяйте `part[0]`. Повторите с отдельным приёмником и `copy`. Сравните результаты.

Ответы

У среза длина ноль: ёмкость не даёт права обратиться к ещё не существующему элементу. Для добавления используется `append`. `copy` учитывает длину приёмника, поэтому её тоже надо проверить.

В обязательной задаче изменяются два места: добавление идентификатора и запись в словаре. Структура объединит поля одной книги; для полного каталога структур всё равно понадобятся список и правило поиска. В следующей контрольной точке начнём с одной такой структуры.

После этой главы у нас есть работающий каталог с явным порядком и отказом при неполных данных. Следующая глава объединит идентификатор, название, цену и состояние публикации в одно значение.

8. Книга как значение: структуры, методы и указатели

Название, цена и признак публикации относятся к одной книге. Если хранить их в отдельных переменных или параллельных словарях, легко перепутать, что к чему относится. Объединим данные и сохраним уже написанные правила.

Контрольная точка: `examples/08-books`. В ней `pricing.go` содержит расчёт скидки из главы 5, а `title.go` — проверку названия из главы 6. Их поведение осталось прежним и проверяется теми же тестами.

Набираете сами: в новой папке нужны ваш `go.mod`, `main.go`, `book.go`, `pricing.go`, `title.go` и тесты `book_test.go`, `pricing_test.go`, `title_test.go`. Готовые файлы есть в контрольной точке. Из главы 5 переносите только функцию расчёта и её тесты (тестовый файл здесь назван `pricing_test.go`), а не вторую функцию `main`. Словарь из главы 7 остаётся учебным опытом; этот пример пока показывает одну карточку `Book`, не полный каталог структур.

Образ: карточка книги и записанный адрес

На карточке книги стоят название, цена и отметка о публикации. Сделайте её копию и исправьте название на копии. Первая карточка сохранит прежний текст. Для нашей структуры `Book` с её нынешними полями это хорошее начало понимания копирования значения.

Теперь вместо карточки перепишите на другой лист её адрес. У вас два листа с адресом, но сама карточка одна. Если оба человека пришли по этому адресу и один поставил отметку о публикации, второй увидит эту же отметку. Так удобнее представить копирование указателя: копируется значение адреса, а новая книга от этого не появляется.

Опора: **копия карточки — другая карточка; копия адреса — путь к прежней карточке**. Вызов метода с получателем-указателем позволяет изменить исходную книгу. Но если внутри структуры появятся срезы или `map`, одной картинкой двух независимых карточек уже мало: их копии могут обращаться к общим внутренним данным. Ниже мы отдельно проверим эту границу. Попробуйте нарисовать два адресных листка и только одну книгу, а затем объяснить, что именно копирует присваивание указателя.

Сначала три маленьких опыта

Не будем одновременно изучать устройство карточки, адрес и новый способ вызова функции. В папке этой главы есть три независимые программы `labs`. Запускайте их из корня `08-books`, рядом с `go.mod`; основной магазин пока не меняйте.

Структура без указателей и методов

`type Book struct { ... }` определяет новый тип: у каждого его значения есть перечисленные поля. Литерал `Book{Title: "Go", PriceMinor: 150}` создаёт одно значение; двоеточие связывает имя поля с начальным значением. Неуказанное поле получает нулевое значение. Точка в `original.Title` выбирает поле, а не функцию пакета.

```

package main

import "fmt"

type Book struct {
    Title      string
    PriceMinor int64
    Published   bool
}

func main() {
    original := Book{Title: "Go", PriceMinor: 150}
    copied := original
    copied.Title = "SQL"
    fmt.Println(original.Title, copied.Title, original.Published)
    var empty Book
    fmt.Printf("[%s] %d %t\n", empty.Title, empty.PriceMinor, empty.Published)
}

go run ./labs/01-structs

```

```

Go SQL false
[] 0 false

```

`copied := original` копирует все поля. Изменение строки в `copied` не меняет строку в `original`. `empty` не является отсутствующей книгой: это существующее значение структуры с пустыми полями. **Проверьте:** задайте `Published: true` только в литерале `original` и предскажите значение этого поля у копии. Копия получает состояние на момент присваивания, а не на момент следующей печати.

Адрес отдельно от карточки

В типе `*int` звёздочка означает указатель на целое. Выражение `&quantity` получает адрес переменной; выражение `*address` обращается к переменной по адресу. Присваивание `*address = 9` меняет эту переменную. Умножение `2 * 9` использует тот же знак в другой роли.

```

package main

import "fmt"

func replaceCopy(value int) {
    value = 9
}

func replaceThroughAddress(address *int) {
    *address = 9
}

func main() {
    quantity := 2
    replaceCopy(quantity)
    fmt.Println(quantity)
    address := &quantity
    sameAddress := address
    replaceThroughAddress(sameAddress)
    fmt.Println(quantity, *address, address == sameAddress)
    var absent *int
    fmt.Println(absent == nil)
}

```

```

}

go run ./labs/02-pointers

2
9 9 true
true

```

Первый вызов меняет собственный параметр и оставляет `quantity` равным 2. Второй тоже получает копию аргумента, но аргумент теперь — адрес `quantity`. По скопированному адресу меняется исходное число. `address == sameAddress` сравнивает указатели, не два независимо созданных числа.

`var absent *int` создаёт nil-указатель. Сравнить его с `nil` можно; читать или записывать `*absent` нельзя — это приведёт к панике. Для изменения через указатель нужно существующее значение и договор, кто вправе его менять. Указатель не является автоматической защитой от ошибки и не требует ручного освобождения памяти как отдельной операции этой программы.

Самостоятельно: создайте другое целое с тем же значением 9, получите его адрес и сравните с `address`. Ожидайте `false`: равенство содержимого не означает один объект.

Метод добавляет получателя, а не новое правило передачи

Обычная функция принимает аргументы в скобках после имени. Метод дополнительно объявляет получателя перед именем: `(book Book)` или `(book *Book)`. Вызов `book.Publish()` передаёт этот объект как получателя. Тип должен быть определён в нашем пакете, чтобы мы объявили для него такой метод.

```

package main

import "fmt"

type Book struct {
    Title    string
    Published bool
}

func (book Book) RenameCopy(title string) {
    book.Title = title
}

func (book *Book) Publish() bool {
    if book == nil {
        return false
    }
    book.Published = true
    return true
}

func main() {
    book := Book{Title: "Go"}
    book.RenameCopy("SQL")
    fmt.Println(book.Title)
    fmt.Println(book.Publish(), book.Published)
    var absent *Book
    fmt.Println(absent.Publish())
    books := []Book{{Title: "Go"}, {Title: "SQL"}}
}

```

```

    for _, copy := range books {
        copy.Publish()
    }
    fmt.Println(books[0].Published, books[1].Published)
    for i := range books {
        books[i].Publish()
    }
    fmt.Println(books[0].Published, books[1].Published)
}

```

```
go run ./labs/03-methods
```

```

Go
true true
false
false false
true true

```

`RenameCopy` меняет копию структуры, поэтому выводится `Go`. Метод `Publish` получает указатель; для адресуемой переменной запись `book.Publish()` здесь равнозначна `(&book).Publish()`. Изменение сохраняется. У `nil`-получателя метод возвращает `false`, потому что **мы написали проверку**, а не потому что Go проверяет каждый метод за нас.

Последние два цикла противопоставляют копию элемента и сам элемент среза. В первом `copy` — новая переменная структуры; во втором `books[i]` обозначает нужный элемент. Уберите один цикл, предскажите результат другого, затем запустите. Не заменяйте все получатели указателями наугад: сначала ответьте, чьё состояние должно измениться.

Критерий перехода: без листинга объясните три разные операции: копирование структуры, копирование указателя и изменение значения по указателю. Затем напишите метод `Unpublish` для указателя с понятным поведением при `nil`. Подсказка: проверка остаётся той же, поле получает `false`. Теперь можно соединить это с проверкой названия и цены.

Сначала работающий результат

Новый файл `book.go`:

```

package main

type Book struct {
    ID          string
    Title       string
    PriceMinor  int64
    Currency    string
    Published   bool
}

// PriceAfterDiscount вычисляет цену, не изменяя книгу.
func (b Book) PriceAfterDiscount(percent int64) (int64, bool) {
    return priceAfterDiscount(b.PriceMinor, percent)
}

// Publish меняет состояние книги только после успешной проверки.
func (b *Book) Publish() bool {
    if b == nil || b.ID == "" || b.Currency != "KZT" {
        return false
    }
}

```

```

    }
    title, ok := normalizeTitle(b.Title)
    if !ok {
        return false
    }
    _, ok = priceAfterDiscount(b.PriceMinor, 0)
    if !ok {
        return false
    }
    b.Title = title
    b.Published = true
    return true
}

```

Файл `main.go`:

```

package main

import "fmt"

func main() {
    book := Book{
        ID:      "go-shop",
        Title:    "  Go: от первой строки до интернет-магазина  ",
        PriceMinor: 249900,
        Currency:  "KZT",
    }
    fmt.Println("До публикации:", book.Published)
    if !book.Publish() {
        fmt.Println("Книга не готова к публикации")
        return
    }
    fmt.Println("После публикации:", book.Published)
    price, ok := book.PriceAfterDiscount(10)
    if !ok {
        fmt.Println("Скидка не подходит")
        return
    }
    fmt.Println(book.Title)
    fmt.Println("К оплате:", price, "минимальных единиц", book.Currency)
}

```

Добавьте из предыдущих контрольных точек `title.go` и функцию `priceAfterDiscount` в файл `pricing.go`. В готовой папке они уже есть. Не копируйте второй `main.go`: внутри пакета нам нужна одна функция `main`.

Запуск: `go run ..` Результат:

```

До публикации: false
После публикации: true
Go: от первой строки до интернет-магазина
К оплате: 224910 минимальных единиц KZT

```

У книги изменилось состояние публикации, название потеряло краевые пробелы, а вычисленная цена использует прежнее правило скидки. Мы не написали магазин заново: соединили уже проверенные части.

Структура связывает поля

`type Book struct` объявляет тип `Book` с перечисленными полями. У каждого поля есть имя и тип. В `main` мы создаём значение литералом `Book{...}` и явно подписываем поля.

Поле `Published` не указано в литерале, поэтому получает нулевое значение `false`. По нашему правилу новая книга начинается как черновик. Это уже осмысленное применение нулевого значения: правило программы согласовано с правилом языка.

`book.Title` обращается к полю конкретного значения. Две разные переменные типа `Book` могут содержать разные названия и цены. Сам тип описывает форму данных, а не единственную книгу на весь магазин.

Имена полей с заглавной буквы экспортируются из пакета. В этой главе весь код ещё внутри `main`; границы пакетов разберём позже. Экспортированное поле не обеспечивает запрета на некорректное присваивание: вызывающий код может записать туда отрицательную цену. Поэтому проверка перед публикацией остаётся необходимой.

Метод — функция с получателем

У `PriceAfterDiscount` перед именем стоит `(b Book)`. Это получатель: значение книги, для которого вызывается метод. Вызов `book.PriceAfterDiscount(10)` передаёт в метод книгу и процент скидки.

Метод с получателем-значением получает копию `Book`. Для нашей структуры копируются строки, целое число и логическое значение. Сам метод просто передаёт цену знакомой функции и ничего не меняет.

Это не новое правило арифметики, а удобный способ связать уже существующее действие с данными книги. Нет необходимости превращать в методы все функции проекта: если связь с конкретным типом не помогает чтению, обычная функция может быть яснее.

Указатель: куда записать изменение

`Publish` объявлен с получателем `(b *Book)`. Тип `*Book` означает указатель на значение `Book`. Такой указатель позволяет обратиться к исходной книге и изменить её поля.

В Go аргументы передаются по значению. Это верно и для указателя: копируется значение указателя, а обе копии указывают на ту же книгу. Поэтому формулировка «со звёздочкой копирования вообще нет» неверна.

`&book` получает адрес переменной. `*pointer` позволяет обратиться к значению по указателю. Для обычного доступа к полю достаточно `pointer.Title`: Go разрешает такую краткую запись без явного `(*pointer).Title`.

В `book.Publish()` компилятор может взять адрес переменной `book` автоматически. Это работает для адресуемого значения. Не делайте из этого правило «любой результат можно сразу менять методом»: временное значение, возвращённое некоторым выражением, может не быть адресуемым. Сначала сохраните его в переменную, если собираетесь изменять.

Нулевой указатель

У указателя есть нулевое значение `nil`: адрес книги не задан. В нашем методе сначала стоит проверка `b == nil`, поэтому такой вызов вернёт отказ, а не попытается читать поле отсутствующей книги.

Это решение именно нашего метода. В Go вызов метода на `nil`-указателе не становится автоматически безопасным: всё зависит от того, что метод делает. `PriceAfterDiscount` имеет получателя-значение, и отсутствие книги не является для него допустимым аргументом.

Изменение после всех проверок

Сначала `Publish` проверяет указатель, идентификатор и валюту. Затем получает подготовленное название в локальную переменную. Потом проверяет цену через существующую функцию скидки с нулевым процентом.

Только после всех успешных проверок два поля получают новые значения. Если цена не подходит, даже удаление краевых пробелов не сохранится в книге. Тест `TestInvalidBookUnchanged` сравнивает её с состоянием до вызова.

Это маленький пример важного правила: отказ не должен оставлять полувыполненное изменение, если договор операции обещает целостность. Для базы данных позже потребуется транзакция; сейчас достаточно правильного порядка присваиваний в одной функции.

Ловушка цикла: меняем копию

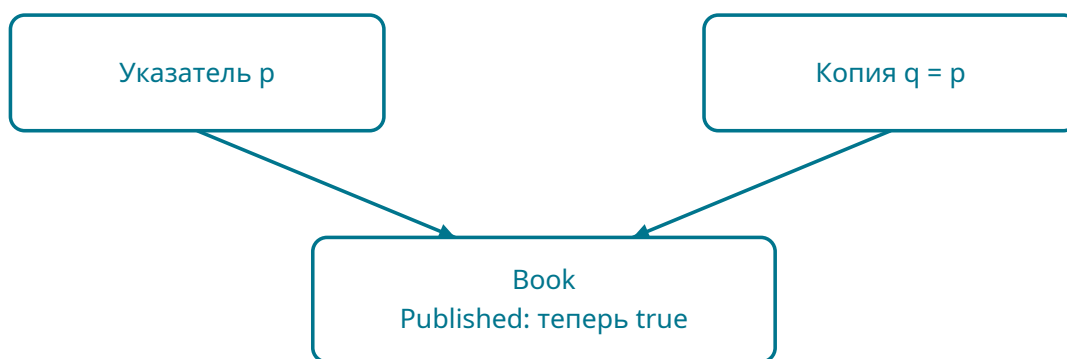
Если написать `for _, book := range books`, переменная `book` получает значение очередного элемента. Для среза структур `Book` это копия структуры. Вызов `book.Publish()` изменит её, а исходный элемент среза останется черновиком.

Для изменения элементов нужен обход по индексам: `for i := range books`, затем `books[i].Publish()`. Элемент среза адресуем, поэтому метод меняет нужную книгу.

В `book_test.go` проверяются оба варианта подряд. Сначала тест подтверждает, что изменение копии не затронуло оригинал, затем — что изменение элемента среза сохранилось. Здесь не требуется угадывать по внешнему виду кода: можно воспроизвести оба поведения.

А если поле — `map` или срез?

Копии указателя ссылаются на одну книгу



Адрес копируется; поле меняется в общей книге

Копирование значения указателя сохраняет адрес общей книги

Наша `Book` пока состоит из простых полей. Если добавить поле `map` или `срез`, копия внешней структуры может продолжать обращаться к общим внутренним данным. Изменение элемента такого контейнера отличается от замены поля целиком.

Копирование структуры с полем `map` сохраняет доступ к общему словарю: запись элемента через копию видна оригиналу, замена самого поля — нет. Например, если `a` и `b` — копии структуры с полем `Titles map[string]string`, то `b.Titles["go"] = "Новое"` меняет общий словарь. А `b.Titles = make(map[string]string)` заменяет только поле `b`. Проверяемая демонстрация есть в комплекте исходников: `book/research/reproductions/map-value`.

Выбор получателя не стоит обосновывать лозунгом «указатели всегда быстрее». Сначала определите семантику: должно ли изменение сохраняться, есть ли общее состояние и разрешено ли копировать сам тип. Измерения производительности — отдельная работа.

Читаем тип и методы по символам

`type` вводит объявление типа. `struct { ... }` перечисляет его поля. В `Book{ID: "go-shop"}` фигурные скобки создают составное значение, а двоеточие связывает имя поля с его значением. Внутри объявления `struct` поля записаны иначе — имя и тип без двоеточия.

Точка в `book.Title` выбирает поле; точка в `book.Publish()` выбирает метод, который затем вызывается круглыми скобками. В `(b Book)` перед именем метода объявлен получатель. В `(b *Book)` звёздочка входит в тип получателя.

`&book` берёт адрес. В выражении `*pointer` звёздочка обращается к значению по адресу, а в типе `*Book` обозначает указатель на книгу. В `%+v` внутри сообщения теста плюс просит `fmt` показать имена полей структуры: это часть формата, не сложение.

Сравнение `book != before` в нашем тесте возможно потому, что все поля текущей `Book` сравнимы. Если добавить `срез` или `map`, структура уже не будет сравниваться таким оператором. Нельзя бездумно переносить тест после изменения формы данных.

Проверка понимания. Прочитайте одну строку создания книги и одну строку вызова метода, вслух называя роль каждого вида скобок. Затем объясните, какой именно объект изменит получатель-указатель.

Разминка

Предскажите. Если `copy := book`, затем `copy.Title = "Другая книга"`, изменится ли `book.Title` для нашей текущей структуры?

Заполните. Какой знак получает адрес переменной: `&` или `*`?

Почините. Цикл вызвал `Publish` для каждой переменной `book`, но элементы `books` остались черновиками. Какой вариант обхода нужен?

Задание

Обязательное. Напишите тест для бесплатной книги: непустые `ID` и `Title`, валюта `KZT`, цена `0`. Метод `Publish` должен вернуть `true` и установить `Published`. Это согласовано с допустимым нулём из главы 5.

На своих данных. Создайте две книги в срезе. Опубликуйте их обходом по индексам и выведите названия опубликованных. Сделайте одну цену недопустимой и убедитесь, что именно эта книга остаётся черновиком.

По желанию. Добавьте метод `Unpublish`, который возвращает книгу в черновик. Определите, что он делает при повторном вызове, и запишите это в тесте. Не связывайте снятие с витрины с удалением уже купленных файлов: это разные решения магазина.

Ответы

Изменение строкового поля копии не заменяет поле оригинала. Адрес получается через `&`. Для изменения структуры в срезе используйте индекс и обращайтесь к `books[i]`.

Тест бесплатной книги должен проверять и результат метода, и состояние поля. Одного `true` недостаточно: ошибочная реализация могла бы согласиться с операцией, ничего не изменив.

Мы получили модель книги и соединили проверку названия с правилом цены. Следующий шаг — научиться возвращать объяснимую ошибку при работе с файлом. Сам метод `Publish` пока сохранит результат `bool`; не все интерфейсы проекта меняются одновременно.

9. Файл не открылся: ошибки и освобождение ресурсов

Название книги пока живёт в исходнике. Чтобы исправить опечатку, приходится менять программу. Сегодня вынесем название в `title.txt`, а прежние правила проверки оставим на месте. После удачного чтения карточка пройдёт тот же метод `Publish`, что и в главе 8.

Контрольная точка: `examples/09-errors`. Вам понадобятся функции, несколько возвращаемых значений, структуры и указатели. Новые вещи — значение ошибки, отложенный вызов и небольшой договор чтения — разберём здесь, до самостоятельного задания.

Набираете сами: скопируйте папку главы 8, замените `main.go` листингом этой главы, добавьте `file.go`, `file_test.go`, `defer_test.go` из контрольной точки и обычный текстовый файл `title.txt` в UTF-8 без BOM. Остальные Go-файлы и `go.mod` сохраните. Расширение должно быть `.txt`, а не `.txt.txt`. Терминал запускайте в этой папке: относительный путь к названию отсчитывается от неё. BOM — необязательная метка кодировки в начале текстового файла. Для опыта выберите в меню кодировки редактора обычную UTF-8 без BOM: лишняя метка не является частью названия.

Образ: ключ и записка у двери

Представьте читальный зал. Вы получили ключ от шкафчика, достали нужный лист и ушли. Лист уже у вас, но ключ всё ещё надо вернуть. Теперь другой случай: шкафчик открылся, а нужного листа нет. Возвращать ключ нужно и тогда. А если ключ вообще не выдали, возвращать нечего.

Для открытого файла есть похожая обязанность: освободить ресурс после работы. Проверка ошибки отвечает на вопрос «получили ли мы ключ?». `defer` позволяет сразу оставить записку: «перед выходом вернуть». Тогда ранний выход из-за плохого названия не забудет о закрытии файла.

Записка — опора, а не устройство операционной системы. Открытый файл не является бумажным ключом. И закрытие файла не означает автоматического сохранения любых данных на физический носитель. Сейчас мы читаем; надёжную запись разберём отдельно.

До файла: функция как значение, ошибка и `defer`

Новые механизмы сначала проверим без файловой системы. Команды выполняются из корня `09-errors`; каждый каталог `labs` содержит самостоятельный пакет `main`.

Передать функцию — не значит вызвать её

В `operation := twice` имя функции стоит без `()`: сохраняется функция. В `operation(3)` скобки запускают её. Тип `func(int) int` означает функцию с одним параметром `int` и одним результатом `int`; имя параметра в этом типе необязательно. Параметр `operation` функции `apply` принимает такое значение и вызывает его позднее. Это основа обратного вызова, который понадобится HTTP-обработчику.

```

package main

import "fmt"

func twice(value int) int {
    return value * 2
}

func apply(value int, operation func(int) int) int {
    return operation(value)
}

func main() {
    operation := twice
    fmt.Println(operation(3), apply(4, operation))
    increment := 1
    next := func(value int) int { return value + increment }
    fmt.Println(next(3))
    increment = 5
    fmt.Println(next(3))
}

go run ./labs/01-functions

6 8
4
8

```

`func(value int) int { ... }` создаёт функцию без имени. Она использует `increment` из внешнего блока: это замыкание. После присваивания `increment = 5` следующий вызов видит 5. Замыкание сохраняет доступ к переменной, а не навсегда запоминает её прежнее число. **Измените:** перед созданием функции сохраните `saved := increment` и используйте в теле `saved`. Второй вызов теперь тоже вернёт 4, поскольку `saved` мы не меняли.

Ошибка — значение с поведением

`error` — готовый интерфейс языка с методом `Error() string`. Интерфейс задаёт доступное поведение, а конкретный тип предоставляет реализацию. Чтобы начать пользоваться этим договором, не требуется уже уметь строить большую архитектуру интерфейсов. В опыте наш маленький тип явно показывает, откуда берётся метод.

```

package main

import (
    "errors"
    "fmt"
)

type titleError struct{ reason string }

func (problem titleError) Error() string {
    return problem.reason
}

func main() {
    var absent error
    fmt.Println(absent == nil)
    var problem error = titleError{reason: "empty title"}
    fmt.Println(problem.Error())
    cause := errors.New("missing file")
}

```

```

    wrapped := fmt.Errorf("read title: %w", cause)
    fmt.Println(wrapped)
    fmt.Println(errors.Is(wrapped, cause))
    fmt.Println(errors.Is(wrapped, errors.New("missing file")))
}

```

```
go run ./labs/03-errors
```

```

true
empty title
read title: missing file
true
false

```

`titleError` — структура из главы 8, `Error` — её метод. Поэтому значение структуры можно присвоить переменной типа `error`; отдельного слова `implements` в Go нет. Через интерфейс вызывающий код использует обещанный метод, не читая внутреннее поле `reason`. У нулевой переменной интерфейса `absent` нет конкретного значения, поэтому она равна `nil`. Нюанс интерфейса, внутри которого лежит `nil`-указатель, разберём отдельно в главе 18; не подменяйте им обычный `return nil`.

`fmt.Errorf` с `%w` добавляет контекст и сохраняет причину. `errors.Is` проверяет цепочку причин, а не совпадение текста. Два отдельных вызова `errors.New("missing file")` создают разные ошибки, поэтому последняя проверка даёт `false`. **Проверка:** замените `%w` на `%v`; сообщение выглядит похоже, но связь с причиной исчезает и первая проверка `errors.Is` становится ложной. Верните `%w`.

Что именно откладывает defer

`defer` регистрирует вызов до выхода из текущей функции. Аргументы обычного вызова вычисляются сразу; само действие выполняется позднее. Несколько действий исполняются в обратном порядке регистрации. Отложенное замыкание может прочитать переменную уже после её изменения.

```

package main

import "fmt"

func observe() {
    value := 1
    defer fmt.Println("argument", value)
    defer func() { fmt.Println("closure", value) }()
    value = 2
    fmt.Println("body", value)
}

func result() (value int) {
    defer func() { value = value + 1 }()
    return 10
}

func main() {
    observe()
    fmt.Println("result", result())
}

go run ./labs/02-defer

```

```
body 2
closure 2
argument 1
result 11
```

В `observe` сначала печатается тело. Затем замыкание читает новое значение 2; последний вызов печатает аргумент 1, сохранённый при регистрации. Закрывающие `()` после тела анонимной функции обозначают вызов, который откладывается; без них это было бы лишь значение функции.

В `result()` (`value int`) результат имеет имя и ведёт себя как переменная функции. `return 10` сначала присваивает ей 10, затем `defer` увеличивает её до 11, и только после этого вызывающий получает 11. Это механизм, который позволит сохранить ошибку закрытия ресурса.

Перед продолжением: переставьте два `defer` и сначала запишите ожидаемый порядок строк. Объясните, почему значение обычного аргумента всё равно остаётся 1. Если это пока не получается, повторите опыт; длинный листинг чтения файла не заменит понимания порядка действий.

Сначала результат

В готовой папке есть прежние `Book`, расчёт скидки, проверка названия и их тесты. Добавился файл `title.txt`:

```
Go: от первой строки до интернет-магазина
```

Два пробела в начале строки здесь намеренные. После запуска `go run` . получим:

```
Go: от первой строки до интернет-магазина
Опубликована: true
```

Файл нужен только для названия. Цену и валюту оставляем в программе, чтобы не изучать одновременно собственный формат каталога. Это промежуточный шаг, а не готовое хранилище магазина.

Вот новый `main.go`:

```
package main

import (
    "errors"
    "fmt"
    "os"
)

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

func run() error {
    title, err := loadTitle("title.txt")
```

```

    if err != nil {
        return err
    }
    book := Book{ID: "go-shop", Title: title, PriceMinor: 249900, Currency: "KZT"}
    if !book.Publish() {
        return errors.New("карточка книги не прошла проверку")
    }
    fmt.Println(book.Title)
    fmt.Println("Опубликована:", book.Published)
    return nil
}

```

Сначала `run` получает название, затем создаёт карточку и проверяет её прежним методом. Печать происходит после успешной проверки. Ошибка не превращается в пустую опубликованную книгу.

Что изменилось по сравнению с `bool`

Раньше второй результат говорил `true` или `false`. Теперь хочется различать причины: файла нет, прочитать его не удалось, название недопустимо. Для этого функция возвращает `error`.

`error` — предопределённый интерфейс Go. Пока достаточно его маленького договора: значение ошибки умеет предоставлять описание через метод `Error() string`. Интерфейс описывает доступное поведение. Какие конкретные типы могут его выполнять и как устроены наборы методов, подробно разберём в главе 18. Здесь нам нужны два состояния: `nil` означает отсутствие ошибки, ненулевое значение — ошибку.

`errors.New("описание")` создаёт ошибку с сообщением. `return nil` в `run` означает успешное завершение. Это результат функции, а не команда печати. В функции с результатами (`string, error`) успешный возврат выглядит как `return title, nil`, а неудачный — как `return "", err`.

Не делайте вывод о неудаче только по пустой строке. Проверять нужно возвращённую ошибку. В наших функциях при ошибке результат специально очищается, но такой договор не распространяется автоматически на все функции стандартной библиотеки.

Где принять решение об ошибке

`main` вызывает `run` и решает, как завершить программу. Запись `if err := run(); err != nil` содержит два шага: сначала вызов и объявление `err`, затем условие после точки с запятой. Здесь `;` написана явно: она отделяет подготовительное действие от проверки. Имя `err` доступно внутри этого `if`.

`fmt.Fprintln(os.Stderr, err)` выводит сообщение в стандартный поток ошибок. Обычный результат программы идёт в другой поток — стандартный вывод. Это позволяет инструментам различать данные и диагностические сообщения.

`os.Exit(1)` завершает процесс с ненулевым кодом: запуск закончился неудачей. При успешном возврате из `main` код завершения будет нулевым. Сам по себе `return` из обычной функции не устанавливает ошибочный код процесса.

Важная граница: `os.Exit` не выполняет отложенные вызовы. Поэтому работа с файлом находится глубже, внутри функции чтения, и заканчивается до возвращения ошибки в `main`. К моменту `os.Exit` наша функция уже попыталась закрыть открытый ресурс.

Три прохода вместо одного большого прыжка

Сначала проследите только `run`: вызов, проверка `err`, ранний `return`, успешный `return nil`. Затем разберите маленькие тесты ниже: у них нет файлов и интерфейса чтения. Только после этого переходите к `file.go`, где эти механизмы работают вместе.

Файл `defer_test.go` из контрольной точки:

```
package main

import (
    "errors"
    "testing"
)

func TestDeferOrder(t *testing.T) {
    order := 0
    f := func() {
        defer func() { order = order*10 + 1 }()
        defer func() { order = order*10 + 2 }()
    }
    f()
    if order != 21 {
        t.Fatalf("получили %d, хотели 21", order)
    }
}

func TestDeferArgumentAndClosure(t *testing.T) {
    argument := 0
    closure := 0
    f := func() {
        n := 1
        defer func(value int) { argument = value }(n)
        defer func() { closure = n }()
        n = 2
    }
    f()
    if argument != 1 || closure != 2 {
        t.Fatalf("получили %d и %d, хотели 1 и 2", argument, closure)
    }
}

func TestDeferNamedResult(t *testing.T) {
    closeErr := errors.New("заккрытие не удалось")
    f := func() (err error) {
        defer func() { err = closeErr }()
        return nil
    }
    if !errors.Is(f(), closeErr) {
        t.Fatal("отложенное действие не изменило результат")
    }
}
```

Запустите `go test -run TestDefer -v ..`. Флаг `-run` выбирает тесты по имени, `-v` показывает их имена и результат. Сначала предскажите значения, не меняя код. Первый тест получает `21`: последнее зарегистрированное действие выполняется первым. Во втором обычный аргумент запоминает `1`, а замыкание позднее читает `2`. В третьем `return nil` сначала записывает `nil` в именованный результат `err`, затем отложенное действие заменяет

его ошибкой.

Запись `f := func() { ... }` сохраняет функцию в переменную, а `f()` вызывает её. Функция может быть значением, как число или строка, но выполняется только при вызове. В записи `func() (err error)` у неё нет входных параметров и один именованный результат. Если этот разбор пока не получается, остановитесь здесь: для следующего шага нужно объяснить все три наблюдения, а не выучить длинный файл наизусть.

Открываем и сохраняем причину отказа

Вся работа с файлом находится в `file.go`:

```
package main

import (
    "errors"
    "fmt"
    "io"
    "os"
)

const maxTitleFileBytes = 1024

var ErrTitleTooLarge = errors.New("файл названия слишком большой")
var ErrInvalidTitle = errors.New("название книги недопустимо")

func loadTitle(path string) (string, error) {
    file, err := os.Open(path)
    if err != nil {
        return "", fmt.Errorf("открыть название %q: %w", path, err)
    }
    title, err := readTitleAndClose(file)
    if err != nil {
        return "", fmt.Errorf("прочитать название %q: %w", path, err)
    }
    return title, nil
}

// readTitleAndClose принимает на себя обязанность закрыть reader.
func readTitleAndClose(reader io.ReadCloser) (title string, err error) {
    defer func() {
        closeErr := reader.Close()
        if closeErr != nil {
            title = ""
            err = errors.Join(err, closeErr)
        }
    }()

    data, err := io.ReadAll(io.LimitReader(reader, maxTitleFileBytes+1))
    if err != nil {
        return "", err
    }
    if len(data) > maxTitleFileBytes {
        return "", ErrTitleTooLarge
    }
    title, ok := normalizeTitle(string(data))
    if !ok {
        return "", ErrInvalidTitle
    }
}
```

```
    return title, nil
}
```

Первый шаг — `os.Open(path)`. Он открывает существующий файл для чтения и возвращает указатель на `os.File` и ошибку. Если ошибка есть, мы сразу возвращаемся. Только после успешного открытия передаём файл в `readTitleAndClose`.

`fmt.Errorf` создаёт ошибку по формату. В "открыть название %q: %w" спецификатор `%q` добавляет путь в кавычках, а `%w` сохраняет исходную ошибку внутри новой. Так мы добавляем контекст, не теряя причину. `%v` тоже мог бы напечатать текст причины, но не создал бы такую связь.

Для программы полезна проверка `errors.Is(err, fs.ErrNotExist)`: присутствует ли среди причин ошибка отсутствия файла? Не сравнивайте `err.Error()` с английской фразой операционной системы. Текст может зависеть от платформы и добавленного контекста.

Наши `ErrTitleTooLarge` и `ErrInvalidTitle` созданы один раз как переменные пакета. Они служат узнаваемыми причинами отказа. Новый вызов `errors.New` с тем же текстом создаст другую ошибку; совпадение сообщения не делает её тем же значением. В тестах используем именно объявленные переменные.

Путь сейчас задан автором программы: `title.txt` в текущей папке запуска. Это не обработчик адресов скачивания из интернета. Прежде чем принимать путь от пользователя, потребуется отдельный договор о доступных файлах и границах каталогов.

Почему чтение выделено в отдельную функцию

`readTitleAndClose` получает `io.ReadCloser`. Это интерфейс из стандартной библиотеки: переданное значение должно предоставлять методы чтения `Read` и закрытия `Close`. Настоящий файл это умеет. В тесте мы передадим маленький объект с теми же методами и сможем специально вызвать отказ.

Интерфейс удовлетворяется неявно: у `*os.File` уже есть нужные методы, отдельного слова `implements` в Go нет. В сигнатуре `Read(p []byte) (int, error)` вызывающий передаёт срез для заполнения; метод записывает в него байты и возвращает их число и ошибку. Именно `io.ReadAll` повторяет чтение и собирает результат.

Функция ожидает действительный открытый ресурс. Передавать в неё `nil` нельзя: вызвать у отсутствующего ресурса `Read` или `Close` не получится.

Это первое практическое знакомство с интерфейсом через готовый договор. Мы не строим иерархию абстракций. Разделение нужно для конкретной проверки: удастся ли сохранить одновременно ошибку чтения и ошибку закрытия? Надёжно вызвать такой случай на настоящем диске во всех трёх ОС неудобно.

Название функции и комментарий задают владение ресурсом: после передачи объекта закрытие становится обязанностью `readTitleAndClose`. Поэтому `loadTitle` не вызывает второй `Close`. Если передавать один объект нескольким функциям без такого договора, легко получить двойное закрытие или забытый ресурс.

Записка defer: что и когда выполняется

В записи (`title string, err error`) результатам даны имена. При входе в функцию это переменные с нулевыми значениями: пустой строкой и `nil`. Именованные результаты нужны здесь, чтобы отложенное действие могло изменить возвращаемую ошибку, если закрытие не удалось.

`defer func() { ... }()` состоит из трёх частей. `func() { ... }` создаёт функцию без имени. Последние `()` обозначают её вызов. `defer` откладывает этот вызов до выхода из окружающей функции. Мы сразу регистрируем действие, но выполняем его позже.

Внутренняя функция обращается к `reader`, `title` и `err` из окружающей функции. Это замыкание: функция сохраняет доступ к нужным переменным своего окружения. В нашем примере важно, что она изменяет именно именованные результаты, а не случайные новые переменные.

При `return title, nil` Go сначала присваивает возвращаемые значения результатам, потом выполняет `defer`, затем отдаёт результат вызывающему коду. Поэтому отложенный вызов ещё успевает заменить `nil` ошибкой закрытия. `data, err := ...` не создаёт новую `err` в этом же блоке: новым является `data`, а именованный результат `err` получает присваивание. Аналогично в `title, ok := ...` новое имя — `ok`, а `title` остаётся результатом функции. Для короткого объявления достаточно хотя бы одного нового имени в том же блоке. Во вложенном блоке новое объявление могло бы скрыть внешнее имя — поэтому здесь важно читать границы скобок.

Везде оставлены явные значения после `return`: читателю легче видеть основной путь.

Если `Close` вернул ошибку, очищаем `title` и вызываем `errors.Join(err, closeErr)`. Эта функция объединяет ненулевые ошибки, сохраняя возможность проверить каждую через `errors.Is`. Если чтение уже отказало, его причина остаётся. Если чтение прошло, но закрытие отказало, наружу всё равно уйдёт ошибка. Это строгий договор нашей функции: успех означает и успешное чтение, и успешное закрытие.

У `defer` есть ещё два правила. Несколько отложенных вызовов выполняются в обратном порядке регистрации. А значения аргументов обычного отложенного вызова вычисляются сразу, при встрече `defer`. Замыкание может обращаться к переменным позже — эти два случая не стоит смешивать. Две короткие проверки ждут вас в задании.

Не регистрируйте закрытие внутри длинного цикла, если рассчитываете освободить каждый файл в конце итерации: `defer` ждёт завершения функции. Наша небольшая функция обработки одного файла как раз задаёт удобную границу.

Ограничиваем то, что читаем

Название не должно заставлять программу загружать огромный файл целиком.

`io.LimitReader(reader, maxTitleFileBytes+1)` ограничивает поток чтения, а `io.ReadAll` собирает прочитанные байты в `[]byte` — срез байтов. Ограничение относится к числу байтов, полученных этим чтением; оно не проверяет заранее размер файла и не закрывает его.

Зачем `+1`? При лимите ровно `1024` мы не отличили бы файл длиной 1024 байта от начала более длинного файла. Разрешив прочитать 1025, получаем возможность обнаружить превышение. Полностью читать оставшиеся мегабайты не требуется.

`len(data)` считает байты. `string(data)` преобразует их в строку, после чего прежняя `normalizeTitle` проверяет UTF-8, убирает краевые пробелы и проверяет название. Лимит файла 1024 байта и лимит названия 80 рун отвечают на разные вопросы. Первый ограничивает объём ввода, второй — правило каталога. Например, большой файл из одних пробелов должен получить отказ ещё до нормализации.

Даже если при ошибке чтения удалось получить часть данных, мы её не используем: программа не должна публиковать обрезанное название. Обычное достижение конца потока само по себе не является ошибкой `io.ReadAll`.

Читаем новые знаки в тестах

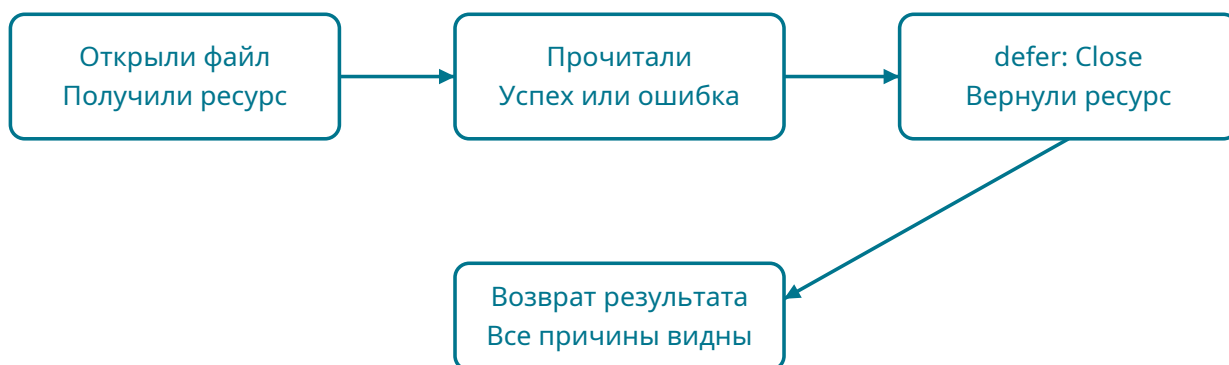
`[]byte("Go")` преобразует строку в срез байтов. `0o600` — целое число в восьмеричной записи: префикс `0o` задаёт основание. Здесь оно передаётся в `os.WriteFile` как права создаваемого тестового файла; интерпретация прав зависит от ОС. Это не настройка прав будущего сервера.

`t.TempDir()` создаёт временную папку, которую тестовая среда затем уберёт. `filepath.Join` соединяет части пути по правилам ОС. `t.Run` запускает именованный подтест; вторым аргументом мы передаём функцию без имени, принимающую `*testing.T`.

`[]struct { ... }` — срез значений структуры, для которой мы не объявили отдельного имени типа. Поля задают вход и ожидаемый результат. В каждой строке таблицы значения идут в порядке полей. `0xff` — число в шестнадцатеричной записи; байт с таким значением позволяет проверить недопустимый UTF-8.

В тестовом `trackedReader` поле `reader` имеет тип `io.Reader`: оно умеет читать. Метод `Read(p []byte) (int, error)` передаёт ему работу. Число в результате — количество прочитанных байтов. Метод `Close` увеличивает счётчик и возвращает заданную тестом ошибку. Так тест наблюдает закрытие без догадок по сообщению программы. `struct{}` у `failingReader` — структура без полей: объекту не нужны данные, чтобы всякий раз вернуть одну проверочную ошибку.

Получили ключ — вернём при любом выходе



Ошибка чтения не отменяет обязанность закрыть файл

Опора: открытый ресурс закрывается перед возвратом при успехе и отказе

Проверяем обещания функции

Запустите `go test ./...`. Тесты проверяют настоящее чтение файла, сохранение причины отсутствия файла, пустое название, недопустимый UTF-8, несколько строк, точную границу размера и превышение на один байт. Отдельно проверяются отказ закрытия и совместный отказ чтения с закрытием. Во всех проверках ресурса ожидается ровно один вызов `Close`.

Полный набор находится в `file_test.go`. Начните с `TestMissingTitleKeepsCause`: обёртка добавляет описание, но `errors.Is` продолжает узнавать отсутствие файла. Затем разберите `TestReadAndCloseErrorsBothSurvive`: одна ошибка не должна скрывать другую.

Задания и опорная карта

Обязательное. Добавьте тест: в файле недопустимое пустое название, а `Close` тоже возвращает ошибку. Ожидаем пустой результат, обе причины через `errors.Is` и ровно одно закрытие. Сначала запишите эти четыре условия словами.

Подсказка. Возьмите `trackedReader` со `strings.NewReader("")` и заданным `closeErr`. Не создавайте новую `errors.New` при сравнении: сохраните одно значение ошибки в переменной.

Разбор. После вызова проверьте `got == "", errors.Is(err, ErrInvalidTitle), errors.Is(err, closeErr) и reader.closes == 1`. Если проверка первой причины не проходит, посмотрите, не заменили ли вы ошибку проверки названия простой записью `err = closeErr`.

Предскажите. В отдельной маленькой функции запишите `defer fmt.Println("первый")`, затем `defer fmt.Println("второй")` и обычную печать `"работа"`. Результат: «работа», «второй», «первый». Отложенные вызовы идут в обратном порядке.

Сравните. Объявите `n := 1`, запишите `defer fmt.Println(n)`, затем `n = 2`. Отложенная печать покажет `1`: аргумент вычислен при регистрации. Если вместо этого написать `defer func() { fmt.Println(n) }()`, увидите `2`: тело замыкания прочитает переменную при выполнении.

Проверьте отказ. В собственной копии контрольной точки временно переименуйте `title.txt`, запустите программу и затем верните имя. Вы увидите сообщение с контекстом открытия файла. Внутренние сообщения ОС могут различаться; сравнивать их побуквенно с книгой не нужно.

Нарисуйте короткую опору: **получили ресурс — зарегистрировали закрытие — прочитали — проверили — вернули результат**. Рядом со словом «закрытие» нарисуйте две стрелки: от удачного пути и от отказа. Закройте исходник и объясните, почему обе стрелки приходят в одно действие.

В магазине появилось первое внешнее содержимое. Мы научились принимать его с ограничением, объяснять отказ и завершать работу с ресурсом. В следующей главе разложим код по пакетам так, чтобы эти правила оставались видимыми и проверяемыми.

Справочные правила: `пакет errors`, `чтение через io`, `файлы os`, `defer в спецификации Go`.

10. Пакеты: у каждого правила свой адрес

К концу главы программа по-прежнему читает название и публикует книгу. Меняется устройство исходников: правила каталога можно использовать из терминала, а затем из HTTP-обработчика. Читатель умеет найти определение по импорту, запустить все тесты и объяснить, почему модуль не равен репозиторию.

Перед началом без подсказки восстановите цепочку главы 9: открыть файл → ограничить чтение → проверить название → закрыть файл → вернуть причину ошибки. Если непонятно, почему закрытие должно произойти и при ошибке чтения, сначала повторите три теста `defer` из предыдущей главы.

Контрольная точка и перенос

Откройте `examples/10-packages` из комплекта кода. Здесь уже есть `go.mod`; выполнять `go mod init` повторно не нужно. Каждая папка главы — самостоятельная версия магазина, а не подкаталог одного огромного модуля. Терминал остаётся в корне контрольной точки: здесь лежат `go.mod` и `title.txt`, а точка входа теперь находится в `cmd/shop/main.go`.

```
10-packages/  
  go.mod  
  cmd/shop/main.go  
  title.txt  
  stdout.txt  
  internal/  
    catalog/  
      book.go          book_test.go  
      pricing.go       pricing_test.go  
      title.go         title_test.go  
  titlefile/  
    file.go           file_test.go
```

Если собираете переход вручную, скопируйте папку главы 9 в отдельную папку. Перенесите файлы `book`, `pricing`, `title` вместе с их тестами в `internal/catalog`, замените первую строку на `package catalog`. Переименуйте `normalizeTitle` в `NormalizeTitle` во всех этих файлах. Затем перенесите `file.go` и `file_test.go` в `internal/titlefile`, замените пакет на `titlefile`; `loadTitle` станет `Load`. В `file.go` добавьте импорт каталога и вызов `catalog.NormalizeTitle`. Оставшийся учебный `defer_test.go` полезен в главе 9; он не часть рабочего пакета каталога. Создайте `cmd/shop` и перенесите туда `main.go`, сохранив `package main`. Старые копии перенесённых файлов в корне новой папки не оставляйте.

Путь модуля в готовом `go.mod`:

```
module github.com/dake-edu/gopher-shop/book/examples/10-packages  
  
go 1.27.1
```

Это адрес пространства имён, а не команда скачивания проекта. Go найдёт собственные пакеты внутри текущего модуля. Учебные пути сохранены для воспроизводимости примеров; при создании своего отдельного продукта выберите свой путь и согласованно замените его в импортах.

Пакет, модуль и репозиторий

Пакет объединяет файлы Go в одной папке. Их объявления доступны друг другу без импортов. Тесты могут находиться в том же пакете; отдельный внешний тестовый пакет с суффиксом `_test` разберём позже. В обычной папке нельзя смешать `package main` и `package catalog`.

Модуль — дерево пакетов с `go.mod` в корне, собственной версией языка и зависимостями.

Репозиторий хранит историю файлов. Один репозиторий может содержать несколько модулей: именно так устроены наши независимые контрольные точки. Запуск `go test ./...` из папки главы не перескакивает в соседние модули.

Опорный образ: пакет — отдел, модуль — организация, репозиторий — архив изменений. Граница образа: каталог на диске не всегда пакет Go, а Git не участвует в исполнении программы.

Папка `internal` имеет специальное правило импорта. Пакет внутри неё доступен коду из дерева родительской папки `internal`. В нашем случае все пакеты магазина могут импортировать `internal/catalog`, а чужой модуль вне этого дерева — нет. Это ограничение компилятора для архитектуры, не защита секретов или файлов от чтения.

Два пакета до переноса магазина

Проверим импорт на программе без файлов, цен и ошибок. В готовой контрольной точке есть `labs/01-package/main.go` и `labs/01-package/greeting/message.go`. Общий `go.mod` остаётся в корне главы. Папка `greeting` — отдельный пакет; `main.go` её родительской папки принадлежит запускаемому пакету `main`.

```
package greeting

func Message(title string) string {
    return "Книга: " + title
}

package main

import (
    "fmt"
    "github.com/dake-edu/gopher-shop/book/examples/10-packages/labs/01-package/greeting"
)

func main() {
    fmt.Println(greeting.Message("Go"))
}

go run ./labs/01-package
```

Книга: Go

Путь импорта складывается из строки `module` и относительного пути папки `labs/01-package/greeting`. Это свой пакет текущего модуля: команда не должна скачивать его с GitHub. В собственном модуле замените начало импорта на **свой** путь из `go.mod`, сохранив путь папки. Имя `greeting` в вызове берётся из `package greeting`; имя файла `message.go` в вызове не участвует.

Опыт с ошибкой: в копии опыта назовите функцию `message` со строчной буквы, сохранив вызов из другого пакета. Компилятор откажет в доступе к нужному экспортированному имени. Затем верните `Message`. Второй опыт: удалите импорт, оставив вызов; ошибка теперь относится к неизвестному имени пакета. Это разные причины, исправлять их нужно в разных местах.

Не переносите этот учебный пакет в `internal/catalog`: он показывает механизм импорта, а не правило магазина. Наличие `labs` не меняет точку входа приложения. Теперь путь `main → titlefile → catalog` ниже можно читать как вызовы между такими же пакетами.

Экспорт и направление зависимостей

Имя, начинающееся с заглавной буквы, экспортируется: `Book`, `Publish`, `NormalizeTitle`, `Load`. Строчная буква делает объявление доступным только внутри своего пакета. Не нужно делать каждую вспомогательную функцию публичной. Например, `readTitleAndClose` остаётся деталью чтения файла.

Направление зависимостей сейчас такое:

```
main → titlefile → catalog
main → catalog
```

Каталог ничего не знает о файлах. Поэтому HTTP-слой позже сможет использовать те же правила. Цикл `catalog → titlefile → catalog` запрещён: Go сообщит `import cycle not allowed`. Лечить его случайным переносом импортов нельзя; нужно пересмотреть, кому принадлежит правило.

Полная точка входа:

```
package main

import (
    "fmt"
    "os"

    "github.com/dake-edu/gopher-shop/book/examples/10-packages/internal/catalog"
    "github.com/dake-edu/gopher-shop/book/examples/10-packages/internal/titlefile"
)

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

func run() error {
    title, err := titlefile.Load("title.txt")
    if err != nil {
        return err
    }
}
```

```

    }
    book := catalog.Book{ID: "go-shop", Title: title, PriceMinor: 249900, Currency: "KZT"}
    if !book.Publish() {
        return fmt.Errorf("карточка книги не прошла проверку")
    }
    fmt.Println(book.Title)
    fmt.Println("Опубликована:", book.Published)
    return nil
}

```

В `catalog.Book` точка означает обращение к экспортированному имени пакета. В `book.Title` та же точка выбирает поле значения. По левой части выражения можно отличить одно от другого. Импорт перечисляет путь, а имя `catalog` берётся из объявления пакета. Неиспользуемый импорт — ошибка компиляции.

Разделите стандартные импорты и импорты проекта пустой строкой. `gofmt` выравнивает оформление, но не принимает архитектурных решений. Комментарий к экспортированной функции начинайте с её имени и объясняйте обещание вызывающему: какие данные допустимы и как представлен отказ.

Проверка без догадок

Исполняемый пакет переехал из корня в `cmd/shop`, поэтому вместо `go run .` теперь используем `go run ./cmd/shop`. Терминал остаётся рядом с `go.mod` и `title.txt`: путь после `run` выбирает пакет, но не меняет рабочую папку программы.

`gofmt -w .` форматирует Go-файлы в текущей папке и её подкаталогах; в главе 2 мы передавали имя одного файла. `go vet ./...` ищет подозрительные конструкции без запуска магазина, например несовпадение формата `Printf` и типа аргумента. Успех обычно не печатает ничего и не заменяет тесты. `go doc ./internal/catalog Book` показывает объявление и комментарий типа `Book` из указанного пакета. Эти команды выполняйте из папки **10-packages**:

```

gofmt -w .
go run ./cmd/shop
go test ./...
go vet ./...
go doc ./internal/catalog Book

```

Вывод программы совпадает с главой 9:

```

Go: от первой строки до интернет-магазина
Опубликована: true

```

`./...` означает текущий пакет и вложенные пакеты текущего модуля. Строка `[no test files]` для `cmd/shop` нормальна: бизнес-тесты переехали вместе с правилами. Но отсутствие тестов во всех пакетах — повод проверить, не потерялись ли файлы при переносе. `go doc` показывает документацию объявления; справка доступна локально без браузера.

Предскажите результат опыта: переименуйте `Load` в `load`, но оставьте `titlefile.Load` в `main.go`. Компиляция завершится ошибкой: требуемого экспортированного имени нет. Верните имя. Второй опыт: запустите `go run .` из корня этой контрольной точки. Go сообщит, что в корне нет файлов Go: запускаемый пакет теперь лежит в `cmd/shop`. Верная команда — `go run ./cmd/shop`; она включает все файлы пакета. Не переходите в `cmd/shop`: относительный

путь `title.txt` считается от текущей папки терминала.

История изменений

Git необязателен для запуска примеров. Если он установлен и вы ведёте собственный учебный репозиторий, после успешных тестов посмотрите `git status` и `git diff`. Добавляйте осознанно выбранные файлы, затем делайте коммит с понятной причиной изменения. Коммит сохраняется локально; `git push` отправляет его на удалённый сервер. Это разные действия.

Не добавляйте пароли, ключи, пользовательские базы и платные материалы без разрешения. В рабочем проекте этой книги полная рукопись хранится отдельно от открытого учебного репозитория. Учебные импорты не означают, что ваши файлы автоматически публикуются по этому адресу.

Самостоятельно: найдите хозяина правила

Нужно запретить пустое название и после чтения файла, и при создании карточки в будущей форме. Где должна находиться проверка? Запишите ответ до просмотра решения.

Подсказка: файл и HTTP — два способа получить данные. Запрет должен работать независимо от способа.

Решение: правило остаётся в `catalog.NormalizeTitle`, а оба входа используют его. Сделайте в копии проекта проверку строже и убедитесь, что тесты чтения файла замечают изменение. Не дублируйте отдельную версию проверки в каждом входе. Вернитесь к исходному правилу и повторите `go test ./...`

Если появляется `no required module provides package`, проверьте соответствие пути в `go.mod` и начале импортов. Если `found packages`, проверьте первые строки файлов одной папки. Если `cannot find main module`, перейдите в папку с `go.mod`; не создавайте новый модуль там, где случайно оказался терминал.

Готовность к следующей главе: вы находите пакет по импорту, объясняете экспорт и `internal`, запускаете все тесты и можете нарисовать зависимости без цикла.

Почему `cmd`, `internal` и `web`, а не обязательный набор папок

`cmd/shop` — исполняемая программа: флаги, соединение зависимостей, запуск и завершение. `internal/catalog` содержит правила товара, `internal/titlefile` — чтение заголовка. При появлении HTTP добавится `internal/web` с обработчиками, а в главе 12 — верхнеуровневый `web` с HTML и CSS. Одинаковое слово не означает одинаковую ответственность: один пакет обрабатывает запросы, другой хранит оформление.

Go ограничивает импорт пакетов внутри `internal` деревом родительской директории. Это полезно для логики приложения, внешний API которой мы не обещаем поддерживать. Каталоги `cmd` и `web` — соглашения проекта. `pkg` не является обязательным стандартом языка; пустую папку ради схемы не создаём. Если появится самостоятельная библиотека для других приложений, отдельно определим её API и границы модуля.

Такое решение соответствует разделу Server project официального руководства: <https://go.dev/doc/modules/layout>. Первые маленькие программы оставались в корне, потому

что делить их на слои ещё не требовалось. Теперь объяснить путь запроса и найти правило важнее, чем сократить число папок.

11. Каталог отвечает браузеру

HTTP — протокол обмена запросами и ответами между клиентом и сервером. Клиентом будет браузер, сервером — наша программа Go. Результат главы — локальный HTTP-сервер с каталогом и отдельным адресом книги. Данные пока не меняются. Мы проверяем поведение запросов автоматическими тестами, не требующими занятого порта или браузера.

Нужны пакеты главы 10, структуры, срезы, указатели и функции. Восстановите по памяти: почему `catalog` не должен импортировать `web`? Ответ понадобится при отделении правил магазина от способа показа.

Как запрос становится ответом



Маршрут — правило программы, а не путь к файлу.

Как запрос становится ответом

Как запустить контрольную точку

Откройте `examples/11-http`. При ручном переносе скопируйте главу 10, замените путь модуля и его начало во всех внутренних импортах на `11-http`. Сохраните пакеты `catalog` и `titlefile` вместе с тестами. Добавьте `internal/catalog/list.go`, папку `internal/web`, замените `cmd/shop/main.go`. Никакой установки веб-фреймворка не требуется.

- `check` — новый флаг нашего магазина: короткая проверка каталога с выходом вместо ожидания браузера. Он передаётся **после пути запускаемого пакета** `./cmd/shop`. Это не стандартная возможность `go run`: её обрабатывает код `cmd/shop/main.go` этой главы. Команда без флага запускает сервер.

```
go run ./cmd/shop -check
go test ./...
go run ./cmd/shop
```

Первая команда печатает `Книг в каталоге: 2` и завершается. Последняя печатает адрес и продолжает работать. Это ожидаемо: сервер ждёт запросов. Откройте в браузере `http://127.0.0.1:8080/`. Для остановки нажмите Ctrl+C в терминале сервера. Для других команд используйте второй терминал в той же папке либо сначала остановите сервер.

URL — адрес ресурса. В `http://127.0.0.1:8080/` часть `http` задаёт протокол, последний `/` — путь главной страницы. `127.0.0.1` — адрес этого компьютера, `8080` — порт, то есть номер точки приёма соединений. Адрес не публикует магазин в интернете. Не заменяйте его на `0.0.0.0` только ради избавления от ошибки: это меняет доступность сервиса для других машин.

Запрос и ответ

Браузер отправляет метод и путь. Метод `GET` просит представление ресурса. Ответ содержит статус, заголовки и тело. Например, `200` означает успешную обработку, `404` — ресурс не найден, `405` — метод не подходит этому адресу.

Запрос	Результат
<code>GET /</code>	Две книги, статус 200
<code>GET /books/go-shop</code>	Название первой книги, статус 200
<code>GET /books/missing</code>	Статус 404
<code>GET /unexpected</code>	Статус 404
<code>POST /books/go-shop</code>	Статус 405

Путь `/books/go-shop` не обязан соответствовать файлу на диске. Это адрес, который программа сопоставляет с обработчиком. Опорный образ — стойка справочной службы: вопрос направляют нужному сотруднику. Граница образа: запросы независимы, и сервер может обслуживать несколько одновременно; нельзя предполагать общий последовательный разговор.

Новые книги создаются в одном месте:

```
package catalog

// Books creates the ordered initial catalog from the validated file title.
func Books(title string) ([]Book, bool) {
    books := []Book{
        {ID: "go-shop", Title: title, PriceMinor: 249900, Currency: "KZT"},
        {ID: "go-tests", Title: "Тесты на Go", PriceMinor: 150000, Currency: "KZT"},
    }
    for i := range books {
        if !books[i].Publish() {
            return nil, false
        }
    }
    return books, true
}
```

Метод `Publish` по-прежнему проверяет название, цену и валюту. `books[i]` позволяет изменить элемент исходного среза; переменная `book` при переборе значений была бы копией. При невалидной карточке возвращаем `false`, и сервер не стартует с частично испорченным каталогом.

Один обработчик без запуска сервера

До большого файла маршрутов проследим один вызов. Функция `home` принимает два аргумента: интерфейс для записи ответа и указатель на структуру запроса. `Header().Set` задаёт заголовок ответа; `fmt.Fprintln` пишет в переданный объект вместо терминала. Обработчик не возвращает строку: результат накапливается в объекте ответа.

```
package main
```

```
import (
    "fmt"
    "net/http"
    "net/http/httptest"
)

func home(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")
    fmt.Fprintln(w, "Go shop")
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /{ $ }", home)
    request := httptest.NewRequest("GET", "/", nil)
    response := httptest.NewRecorder()
    mux.ServeHTTP(response, request)
    fmt.Println(response.Code)
    fmt.Print(response.Body.String())
    missing := httptest.NewRecorder()
    mux.ServeHTTP(missing, httptest.NewRequest("GET", "/missing", nil))
    fmt.Println(missing.Code)
}

go run ./labs/01-handler

200
Go shop
404
```

Команду выполняем из корня главы. Порт не открывается: `httptest` — часть стандартной библиотеки для запросов и ответов в памяти. `NewRequest` создаёт запрос, `NewRecorder` — получателя ответа; его поля `Code` и `Body` позволяют увидеть результат. Пустое тело запроса задаётся `nil`.

В `HandleFunc("GET /{ $ }", home)` передаётся **функция без вызова**, как `operation` в главе 9. Маршрутизатор запоминает её; позже `mux.ServeHTTP(response, request)` выбирает маршрут и вызывает `home`. Первая запись тела выбирает 200, если статус не задан явно. Для `/missing` подходящего маршрута нет, поэтому код ответа 404.

Трассировка: `main` → `ServeHTTP` → `home` → `Fprintln` → **возврат в main**. Напечатайте эту последовательность словами до запуска. Затем замените путь запроса на `/missing`, не меняя маршрут: `home` не будет вызван. Не сравнивайте ответ 404 с успешным текстом магазина. Верните путь и добавьте второй маршрут `/healthz` с отдельной функцией: это подготовка к заданию в конце главы.

`ResponseWriter` и `Handler` — интерфейсные договоры стандартной библиотеки. Здесь первый предоставляет запись ответа, второй — метод `ServeHTTP`. Как и с `error`, конкретное значение удовлетворяет договору своими методами. Собственный минимальный интерфейс и правила наборов методов разберём в главе 18; для чтения следующего листинга уже нужно отличать функцию от её вызова и интерфейс от указателя на структуру.

Обработчик и маршрутизатор

```
package web

import (
    "fmt"
    "net/http"

    "github.com/dake-edu/gopher-shop/book/examples/11-http/internal/catalog"
)

// New copies the catalog; handlers only read this snapshot.
func New(books []catalog.Book) http.Handler {
    snapshot := make([]catalog.Book, len(books))
    copy(snapshot, books)
    mux := http.NewServeMux()
    mux.HandleFunc("GET /{$}", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "text/plain; charset=utf-8")
        for _, book := range snapshot {
            fmt.Fprintln(w, book.ID, book.Title)
        }
    })
    mux.HandleFunc("GET /books/{id}", func(w http.ResponseWriter, r *http.Request) {
        for _, book := range snapshot {
            if book.ID == r.PathValue("id") {
                w.Header().Set("Content-Type", "text/plain; charset=utf-8")
                fmt.Fprintln(w, book.Title)
                return
            }
        }
        http.NotFound(w, r)
    })
    return mux
}
```

`http.NewServeMux()` создаёт маршрутизатор. `HandleFunc` связывает шаблон маршрута с функцией. В записи `GET /{$}` метод входит в шаблон, а `{}` ограничивает совпадение ровно корнем. Просто `/` совпадало бы и с другими путями. В `/books/{id}` фигурные скобки обозначают переменный сегмент; `r.PathValue("id")` возвращает его значение. Это синтаксис маршрута внутри строки, а не блок Go.

У функции обработчика два параметра. Через `w http.ResponseWriter` пишем ответ. Через `r *http.Request` читаем запрос. `http.ResponseWriter` — интерфейс: он описывает доступные методы, а конкретный объект предоставляет сервер. Как с `error` в главе 9, нам достаточно пользоваться контрактом. Указатель перед `http.Request` уже знаком по структурам.

Заголовок `Content-Type` сообщает вид содержимого и кодировку. Его устанавливают до первой записи тела. `fmt.Fprintln(w, ...)` отправляет текст в ответ вместо терминала. Первая запись тела без явного `WriteHeader` выбирает статус 200. `http.NotFound` сам записывает 404 и тело; после его вызова не следует дописывать успешный ответ.

Возвращаемое `http.Handler` обещает метод `ServeHTTP`. Наш `mux` умеет его выполнять. Это позволяет одному обработчику работать и с настоящим сервером, и в тесте.

Функции маршрутов замыкают `snapshot`: используют значение из внешней функции после её завершения. Мы создаём новый срез и копируем в него карточки. Поэтому последующее изменение среза вызывающего кода не изменяет каталог обработчика. Сейчас `Book` содержит

строки, числа и булево поле: копирования значений достаточно. Если позже появится поле-срез или словарь, копия структуры не станет глубокой автоматически.

Обработчики только читают снимок. Это существенное условие: библиотека может вызывать их одновременно. Добавлять общий изменяемый словарь без синхронизации нельзя.

Механизм конкурентного исполнения разберём отдельно; сейчас исключаем общую запись из модели.

Жизненный цикл процесса

```
package main

import (
    "errors"
    "flag"
    "fmt"
    "net/http"
    "os"
    "time"

    "github.com/dake-edu/gopher-shop/book/examples/11-http/internal/catalog"
    "github.com/dake-edu/gopher-shop/book/examples/11-http/internal/titlefile"
    "github.com/dake-edu/gopher-shop/book/examples/11-http/internal/web"
)

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

func run() error {
    check := flag.Bool("check", false, "проверить каталог без запуска сервера")
    flag.Parse()
    title, err := titlefile.Load("title.txt")
    if err != nil {
        return err
    }
    books, ok := catalog.Books(title)
    if !ok {
        return errors.New("каталог не прошёл проверку")
    }
    if *check {
        fmt.Println("Книг в каталоге:", len(books))
        return nil
    }
    server := &http.Server{
        Addr: "127.0.0.1:8080", Handler: web.New(books),
        ReadHeaderTimeout: 5 * time.Second, ReadTimeout: 10 * time.Second,
        WriteTimeout: 15 * time.Second, IdleTimeout: time.Minute,
    }
    fmt.Println("Откройте http://127.0.0.1:8080; остановка: Ctrl+C")
    return server.ListenAndServe()
}
```

`flag.Bool` объявляет логический флаг и возвращает указатель на его значение. Три аргумента — имя, значение по умолчанию и текст справки. `flag.Parse()` читает аргументы

командной строки; после него `*check` получает выбранное значение. В `go run ./cmd/shop -check` путь пакета равен `./cmd/shop`, а всё после него передаётся магазину. `go run ./cmd/shop -h` показывает справку о флагах этой программы. Она может меняться между главами.

`time.Second` и `time.Minute` — длительности. Поля таймаутов ограничивают чтение заголовков, запроса, запись ответа и ожидание следующего запроса в соединении. Эти значения подходят маленьким учебным ответам; они ещё не являются настройкой для многогигабайтных скачиваний.

`ListenAndServe` занимает текущий вызов до ошибки или остановки. В этой главе `Ctrl+C` прерывает процесс. Завершение активных запросов без обрыва появится в главе о контексте и остановке. Не называем нынешнюю остановку плавной.

Тестируем HTTP без сети

```
package web
```

```
import (
    "net/http/httptest"
    "strings"
    "testing"

    "github.com/dake-edu/gopher-shop/book/examples/11-http/internal/catalog"
)

func TestRoutes(t *testing.T) {
    books, ok := catalog.Books("Go")
    if !ok {
        t.Fatal("invalid fixture")
    }
    handler := New(books)
    books[0].Title = "Changed outside"
    for _, tc := range []struct {
        method, path string
        code          int
    }{
        {"GET", "/", 200}, {"GET", "/books/go-shop", 200},
        {"GET", "/books/missing", 404}, {"GET", "/missing", 404},
        {"POST", "/books/go-shop", 405},
    } {
        w := httptest.NewRecorder()
        handler.ServeHTTP(w, httptest.NewRequest(tc.method, tc.path, nil))
        if w.Code != tc.code {
            t.Fatalf("%s %s: %d", tc.method, tc.path, w.Code)
        }
        if strings.Contains(w.Body.String(), "Changed outside") {
            t.Fatal("catalog aliases caller")
        }
    }
}
```

`httptest.NewRequest` создаёт запрос в памяти. `httptest.NewRecorder` записывает ответ. `handler.ServeHTTP` вызывает тот же код, который обслужит браузер. Вложенный литерал `[]struct{...}` описывает таблицу случаев без отдельного имени типа: поля `method`, `path`, `code` объясняют каждую строку.

Этот тест проверяет маршрутизацию, но не доказывает работоспособность порта, TLS или сетевых таймаутов. Поэтому один раз отдельно открываем страницу браузером. После изменений повторяем `go test ./...`

Задание и восстановление

Добавьте адрес `GET /healthz`, возвращающий `ok` и перевод строки со статусом 200. Сначала добавьте случай в таблицу и убедитесь, что тест падает с 404. Подсказка: используйте `mux.HandleFunc` до `return mux`.

Решение — обработчик с `fmt.Fprintln(w, "ok")`. После этого тест должен пройти. Это только проверка доступности процесса, а не готовности будущей базы данных. Проверьте отдельно, что `POST /healthz` получает 405.

Ошибка `address already in use` означает, что порт занят, часто вашим предыдущим запуском. Остановите его Ctrl+C. Если изменяете порт в `Addr`, измените и адрес в браузере. Если браузер показывает старый текст, убедитесь, что перезапустили сервер после редактирования Go: `go run` не следит за файлами автоматически.

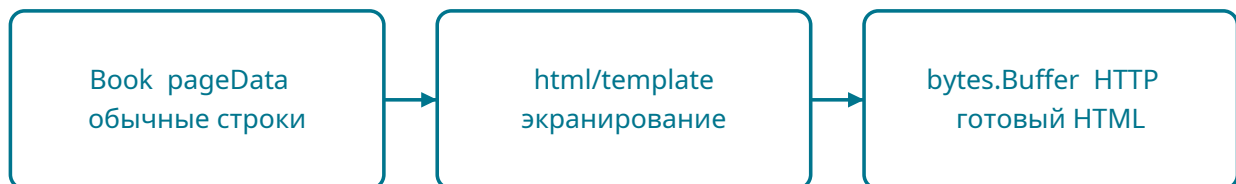
Готовность: вы различаете маршрут и путь файла, объясняете статус 404 и 405, запускаете и останавливаете сервер, пишете тест запроса без открытия сетевого порта.

12. HTML-шаблон превращает каталог в страницу

В этой главе каталог получает заголовки, ссылки, цены и простой стиль. Мы сохраняем сервер и правила книги, меняем представление ответа. Внешний сборщик, JavaScript и CSS-фреймворк не нужны.

Перед началом объясните, почему заголовок ответа устанавливают раньше тела и почему обработчик возвращает 404 для неизвестной книги. Если не получается, повторите тест маршрутов из главы 11.

Данные и представление



Сначала готовим страницу; затем отправляем ответ.

Данные и представление

Файлы и быстрый результат

Откройте `examples/12-templates`. При ручном переходе скопируйте главу 11, согласованно замените `11-http` на `12-templates` в `go.mod` и импортах. В `internal/web` замените `web.go`, добавьте `templates.go`, `template_test.go` и `footer_test.go`; прежний `web_test.go` сохраняется. Скопируйте дерево `web` из контрольной точки: `assets.go`, `templates/layouts`, `templates/pages`, `templates/partials` и `static/css/style.css`. Остальные пакеты не меняют поведения.

```
go test ./...
go run ./cmd/shop
```

Откройте `http://127.0.0.1:8080/`: увидите две карточки. В меню пока только eGopher: корзина появится в главе 13, регистрация — в главе 19, вход — в главе 20. Ссылки добавляем вместе с работающими маршрутами; отсутствие будущего пункта здесь не ошибка. Переход по названию открывает отдельную страницу книги. Адрес отсутствующей книги по-прежнему даёт 404. Сервер останавливается `Ctrl+C`.

Минимум HTML для чтения шаблона

HTML размечает смысл документа. `<h1>` — главный заголовок страницы, `<article>` — самостоятельная карточка, `<a href="...">` — ссылка, `<p>` — абзац. Закрывающий тег начинается с `</`. Атрибуты внутри открывающего тега задают свойства элемента. HTML не является кодом Go.

`lang="ru"` помогает средствам чтения выбрать язык. `charset="utf-8"` задаёт кодировку. `viewport` позволяет странице учитывать ширину экрана. Область `<main>` отделяет основное содержимое, ссылка на `#content` позволяет перейти к нему с клавиатуры. Сначала смысл и порядок чтения, затем оформление.

Сначала одна подстановка

Не будем одновременно разбирать файлы `layout`, встраивание и каталог. Откройте `labs/01-template/main.go` в контрольной точке; команда запускается рядом с `go.mod` этой главы.

```
package main

import (
    "fmt"
    "html/template"
    "os"
)

type Page struct{ Title string }

func run() error {
    page, err := template.New("card").Parse("<h1>{{.Title}}</h1>\n")
    if err != nil {
        return err
    }
    return page.Execute(os.Stdout, Page{Title: "<Go>"})
}

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

go run ./labs/01-template

<h1>&lt;Go&gt;</h1>
```

`template.New("card")` создаёт набор с именем `card`. `Parse` разбирает текст шаблона и может вернуть ошибку; `Execute` исполняет его с конкретными данными и записывает результат в `os.Stdout`. Разбор и исполнение — два разных шага. Запись `{{.Title}}` читает поле текущего значения `Page`, переданного вторым аргументом `Execute`.

Мы передали строку `<Go>`. Результат содержит `<` и `>`; HTML-сущности для знаков `<` и `>`. Браузер покажет название текстом, а не новым HTML-тегом. Терминал печатает сами символы разметки: он не является браузером. **Опыт:** замените название на `Go & SQL`; ожидайте `Go & SQL` между тегам. Верните исходное значение после проверки.

Ошибка неизвестного поля может проявиться при **исполнении**, когда появились конкретные данные: попробуйте `{{.Missing}}`. Ошибка незавершённого действия вроде `{{` обнаружится при **разборе**. Сохраните рабочий вариант: позже обработчик должен различать эти стадии и не отправлять половину успешного ответа.

Затем каркас и повторяемая часть

`define` объявляет именованный шаблон, а `template` исполняет его. Имя в кавычках — ключ в наборе шаблонов, оно не обязано совпадать с именем файла. Последняя точка передаёт текущие данные вложенному шаблону.

```
package main

import (
    "fmt"
    "html/template"
    "os"
)

type Page struct{ Title string }

func run() error {
    source := `{{define "base"}}<main>{{template "content" .}}</main>{{template "footer" .}}
{{end}}{{define "content"}}<h1>{{.Title}}</h1>{{end}}{{define "footer"}}<footer>eGopher</footer>{{end}}`
    page, err := template.New("pages").Parse(source)
    if err != nil {
        return err
    }
    return page.ExecuteTemplate(os.Stdout, "base", Page{Title: "Go"})
}

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

go run ./labs/02-layout
```

```
<main><h1>Go</h1></main><footer>eGopher</footer>
```

Сначала `ExecuteTemplate` выбирает `base`, он вызывает `content` и `footer`. Только `content` читает `Title`; подвал здесь содержит постоянную подпись. Измените `Page.Title`: изменится заголовок, каркас и подвал сохранятся. Затем уберите точку из `{{template "content" .}}` и наблюдайте потерю данных во вложенном шаблоне. Восстановите передачу точки. Так мы проверяем назначение аргумента, а не запоминаем скобки наизусть.

В магазине эти три объявления будут разделены по файлам `layouts`, `pages` и `partials`. `embed` решает доставку этих файлов вместе с программой, а CSS — их отображение в браузере. Они не меняют уже изученный принцип «данные → исполнение шаблона → HTML». Ниже добавляем их по отдельности.

Возвращаемся к каталогу

Содержимое страницы каталога:

```
{{define "title"}}{{.Title}}{{end}}
{{define "content"}}
<h1>{{.Title}}</h1>
{{range .Books}}
    <article>
        <h2>
            <a href="/books/{{.ID}}">{{.Title}}</a>
```

```

        </h2>
        <p>{{money .PriceMinor}} {{.Currency}}</p>
    </article>
    {{else}}
        <p>Каталог пока пуст.</p>
    {{end}}
    {{end}}

```

Двойные фигурные скобки — действия шаблона. `{{.Title}}` берёт поле `Title` текущих данных. `{{range .Books}}` проходит по книгам; внутри цикла точкой становится текущая книга. `{{else}}` показывает текст при пустом срезе. `{{end}}` завершает блок. Это отдельный язык `html/template`; слова похожи на Go, но правила принадлежат шаблонизатору.

`{{money .PriceMinor}}` вызывает зарегистрированную функцию форматирования. Цена хранится целым числом минимальных единиц; деление на 100 даёт целую часть, остаток % 100 — дробную. Формат `%02d` дополняет дробную часть слева нулём. Из 105 получаем `1.05`. Область применения этого помощника — неотрицательные цены нашей валюты с двумя дробными знаками; это не универсальная библиотека денежных расчётов.

Встраивание и подготовка ответа

Публичные ресурсы сразу разделим по назначению, чтобы рост магазина не превратил `static` в общий склад:

<code>web/static/</code>	
<code>brand/</code>	логотип и фирменные знаки
<code>css/style.css</code>	таблица стилей
<code>js/</code>	сценарии браузера, когда понадобятся
<code>images/</code>	общедоступные изображения

Сейчас используются CSS и фирменные изображения. В папках `js` и `images` лежит пустой `.gitkeep`: Git хранит файлы, а не пустые директории, поэтому этот обычный файл сохраняет структуру в репозитории и комплекте примеров. Это соглашение проекта, не специальная команда Git. JavaScript пока не нужен; пустая папка не означает, что нужно писать лишний сценарий.

`brand` содержит постоянную айдентiku магазина, `images` — общедоступные иллюстрации и обложки товаров. Полные платные книги, базы, ключи и пользовательские загрузки сюда не относятся. Добавление файла на диск само по себе не открывает ему HTTP-адрес: сервер явно обслуживает `/assets/css/style.css` и перечисленные ниже фирменные изображения. Путь на диске — `web/static/css/style.css`; URL выбирает маршрутизатор. При добавлении новых типов ресурсов расширяем обработку осознанно, без выдачи списка директорий.

Логотип и маленькая иконка решают разные задачи

В `web/static/brand/gopher-cart.png` лежит логотип: гофер с тележкой по мотивам обложки нашей книги. Это PNG с прозрачным фоном, 256 × 256 пикселей. В шапке он показывается размером 80 × 80; запас разрешения помогает на плотных экранах. Для крошечной вкладки подробная тележка не подходит: отдельная иконка показывает только узнаваемую голову гофера.

`favicon.ico` содержит два размера, 16 × 16 и 32 × 32. Их исходные PNG тоже лежат в `brand`. `apple-touch-icon.png` — квадрат 180 × 180 с белым фоном для сохранения сайта на

домашнем экране совместимого устройства. Это иконка, не установка полноценного приложения; манифеста и работы без сети здесь нет. Исходные большие изображения и описание их подготовки находятся в материалах редакции, а в контрольной точке — готовые небольшие файлы.

В `web/templates/layouts/base.html` элементы `link` с `rel="icon"` и `rel="apple-touch-icon"` сообщают браузеру назначение файлов. В `web/templates/partials/header.html` обычный `img` загружает логотип. Атрибуты `width` и `height` резервируют место до загрузки. Ссылка логотипа получает доступное имя через `aria-label="{{brand_name}} — на главную"`. Поэтому у изображения оставлен пустой `alt=""`: программа чтения экрана использует имя ссылки и не повторяет его. Отдельный элемент `nav` с `aria-label="Главное меню"` объединяет текстовые ссылки меню. Функция `brand_name` возвращает короткое название `eGopher`; шапка, подвал и название вкладки используют одну функцию. При смене бренда достаточно изменить это значение в `internal/web/templates.go`.

Добавьте `internal/web/brand.go` и вызов `registerBrand(mux)` сразу после создания маршрутизатора. Функция регистрирует только три разрешённых адреса; содержимое произвольной папки через HTTP не раскрывается. `serveImage` принимает байты и тип содержимого: `image/png` для PNG и `image/vnd.microsoft.icon` для ICO. `w.Write(data)` передаёт двоичные байты без преобразования в текст. Здесь результат единственной записи игнорируется явно: при оборванном соединении повторная запись ответа не восстановит клиента.

```
package web

import (
    "net/http"

    "github.com/dake-edu/gopher-shop/book/examples/12-templates/web"
)

func registerBrand(mux *http.ServeMux) {
    serveImage(mux, "/assets/brand/gopher-cart.png", "image/png", assets.Logo)
    serveImage(mux, "/favicon.ico", "image/vnd.microsoft.icon", assets.Favicon)
    serveImage(mux, "/assets/brand/apple-touch-icon.png", "image/png", assets.TouchIcon)
}

func serveImage(mux *http.ServeMux, path, contentType string, data []byte) {
    mux.HandleFunc("GET "+path, func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", contentType)
        _, _ = w.Write(data)
    })
}
```

Логотип и иконки встраиваются через отдельные переменные `[]byte` в пакете `assets`; после замены изображения требуется пересборка программы. Проверьте логотип в шапке и откройте `/favicon.ico` напрямую. Если вкладка показывает прежний значок, учтите кеш браузера: перезагрузите страницу или проверьте её в новом профиле. Тест `TestBrandAssetsAndClosedDirectory` сравнивает ответы с встроенными файлами и проверяет 404 для неизвестного файла и каталога.

```
package web

import (
```

```

"bytes"
"fmt"
"net/http"

"github.com/dake-edu/gopher-shop/book/examples/12-templates/internal/catalog"
)

type pageData struct {
    Title string
    Books []catalog.Book
}

func money(minor int64) string { return fmt.Sprintf("%d.%02d", minor/100, minor%100) }

func New(books []catalog.Book) http.Handler {
    snapshot := append([]catalog.Book(nil), books...)
    page := pageTemplate("home")
    render := func(w http.ResponseWriter, data pageData) {
        var out bytes.Buffer
        if err := page.Execute(&out, data); err != nil {
            http.Error(w, "Не удалось показать страницу", http.StatusInternalServerError)
            return
        }
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        _, _ = out.WriteTo(w)
    }
    mux := http.NewServeMux()
    registerBrand(mux)
    mux.HandleFunc("GET /{$}", func(w http.ResponseWriter, r *http.Request) {
        render(w, pageData{Title: "Каталог", Books: snapshot})
    })
    mux.HandleFunc("GET /assets/css/style.css", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "text/css; charset=utf-8")
        fmt.Fprint(w, styleSource)
    })
    mux.HandleFunc("GET /books/{id}", func(w http.ResponseWriter, r *http.Request) {
        for _, book := range snapshot {
            if book.ID == r.PathValue("id") {
                render(w, pageData{Title: book.Title, Books: []catalog.Book{book}})
                return
            }
        }
        http.NotFound(w, r)
    })
    return mux
}

```

В верхнеуровневом пакете `web` объявлено имя `assets`, чтобы его не путать с обработчиками. Директива `//go:embed templates static` включает дерево файлов в `embed.FS`. Это встроенная файловая система только для чтения. Путь считается от файла с директивой; выйти наверх через `..` нельзя. Поэтому `assets.go` лежит рядом с папками ресурсов. Отдельная строковая переменная `Style` встраивает CSS для простого обработчика одного файла.

```

// Package assets contains the public presentation files embedded in the binary.
package assets

import "embed"

```

```
//go:embed templates static
var Files embed.FS

//go:embed static/css/style.css
var Style string

//go:embed static/brand/gopher-cart.png
var Logo []byte

//go:embed static/brand/favicon.ico
var Favicon []byte

//go:embed static/brand/apple-touch-icon.png
var TouchIcon []byte
```

Каркас документа хранится в layout. `{{define "base"}}` объявляет именованный шаблон, `{{template "content" .}}` исполняет другой шаблон и передаёт ему текущие данные. Partials — небольшие повторяющиеся части, например шапка и подвал. Это соглашение об устройстве файлов, поддержанное средствами стандартной библиотеки, а не сторонний движок.

```
{{define "base"}}
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{{template "title" .}} – {{brand_name}}</title>
    <link rel="icon" href="/favicon.ico" sizes="16x16 32x32">
    <link rel="apple-touch-icon" href="/assets/brand/apple-touch-icon.png" sizes="180x180">
    <link rel="stylesheet" href="/assets/css/style.css">
  </head>
  <body>
    <a href="#content">Перейти к содержимому</a>
    {{template "header" .}}
    <main id="content">{{template "content" .}}</main>
    {{template "footer" .}}
  </body>
</html>
{{end}}

{{define "header"}}
<header>
  <a class="brand" href="/" aria-label="{{brand_name}} – на главную">
    
  </a>
  <nav aria-label="Главное меню">
    <a href="/"{{if nav_active "catalog"}} aria-current="true"{{end}}>{{brand_name}}</a>
  </nav>
</header>
{{end}}
```

Текущий год в общем подвале

Подвал показывает знак ©, текущий год и название магазина. `©` — HTML-сущность: браузер отображает её как ©. `{{year}}` и `{{brand_name}}` вызывают функции без аргументов; точки перед именами нет. В отличие от `{{.Title}}`, они не читают поля данных страницы. Поэтому один подвал работает и с каталогом, и позднее с библиотекой и административными страницами, даже если у них разные структуры данных.

```

{{define "footer"}}
<footer>
    <p>&copy; {{year}} {{brand_name}}</p>
    <p>Учебный интернет-магазин.</p>
</footer>
{{end}}

```

Каждая страница определяет `title` и `content`. Для неё создаётся отдельный набор: общий layout, общие partials и ровно один файл страницы. Нельзя разобрать все страницы одним шаблоном: одинаковые имена `content` начнут заменять друг друга. `ParseFS` читает файлы из встроенного `assets.Files`; шаблон пути `*.html` выбирает partials. Имя страницы задаёт наш код при запуске, а не параметр URL. Подготовленные наборы не изменяются во время запросов.

```

package web

import (
    "html/template"
    "time"

    assets "github.com/dake-edu/gopher-shop/book/examples/12-templates/web"
)

// Each page gets its own set: identical content names cannot replace another page.
// Call only during handler construction, before serving concurrent requests.
func pageTemplate(name string) *template.Template {
    active := "account"
    if name == "home" || name == "workshop" {
        active = "catalog"
    }
    if name == "cart" || name == "order" || name == "payment" {
        active = "cart"
    }
    if name == "cart" {
        name = "home"
    }
    return template.Must(template.New("base").Funcs(template.FuncMap{
        "nav_active": func(section string) bool { return section == active },
        "money":      money,
        "year":       func() int { return time.Now().Year() },
        "brand_name": func() string { return "eGopher" },
    }).ParseFS(
        assets.Files, "templates/layouts/base.html", "templates/partial/*.html",
        "templates/pages/"+name+".html",
    ))
}

var styleSource = assets.Style

```

`template.FuncMap` — таблица имён функций, доступных шаблону. Регистрируем `money`, `year` и `brand_name` через `Funcs` до `ParseFS`: иначе разбор остановится с ошибкой `function "year" not defined`. Функция `brand_name` возвращает название магазина; это значение можно изменить в одном месте для всех подвалов.

Пакет `time` входит в стандартную библиотеку. `time.Now()` читает текущее время, а метод `.Year()` возвращает год как `int`. Запись `func() int { return time.Now().Year() }` создаёт функцию без аргументов: её тело выполняется при вызове `{{year}}`, то есть при каждом формировании страницы. Мы передаём саму функцию, а не заранее вычисленный

год. Поэтому работающий сервер автоматически покажет новый год после полуночи 1 января, без пересборки и перезапуска. Уже открытая вкладка обновится при следующей загрузке страницы.

Здесь используется локальная временная зона процесса сервера. На сервере с UTC год сменится по UTC; если магазину нужен календарь определённого региона, временную зону следует задать явно. Это отдельное решение, а не свойство языка шаблонов.

Проверьте: запустите магазин и найдите © с текущим годом внизу каталога и страницы книги. В `footer_test.go` один разобранный шаблон исполняется с тестовой функцией часов сначала для 2026, затем для 2027 года. Такая подмена проверяет смену значения без ожидания Нового года и без изменения системных часов. Числа в тесте — проверочные данные; в работающем магазине год берётся только из `time.Now().Year()`.

При изменении встроенного HTML или CSS перезапустите `go run ./cmd/shop`. Встраивание не делает ресурсы секретными: браузеру передаётся результат HTML и публичный CSS. Платные файлы книги здесь хранить нельзя.

Новая запись `append([]catalog.Book(nil), books...)` создаёт копию элементов среза. `[]catalog.Book(nil)` — пустой `nil`-срез нужного типа. Три точки после `books` передают элементы среза как отдельные аргументы `append`. Это краткая форма знакомой пары `make` и `copy`, а не «продолжение пропущенного кода». Исходный и новый срез не разделяют изменяемые элементы массива; вложенные ссылки внутри элементов, если появятся, требуют отдельного внимания.

`template.FuncMap` связывает имя `money` с нашей функцией. Функции регистрируются до `Parse`, чтобы имена были известны при разборе. `template.Must` проверяет результат разбора и аварийно останавливает запуск при ошибке встроенного шаблона. Это осознанная граница: неверный шаблон — дефект поставленной программы. Ошибки пользовательской формы позже будем возвращать как обычные ответы, без аварийной остановки.

При исполнении шаблона сначала пишем в `bytes.Buffer` — буфер в памяти. Если `Execute` вернул ошибку, можно ещё честно отправить статус 500. При немедленной записи в сеть часть успешной страницы могла бы уже уйти клиенту, и поменять статус стало бы поздно. После успешного исполнения ставим `Content-Type: text/html; charset=utf-8` и переносим буфер в ответ. Ошибку последней сетевой записи уже нельзя исправить новым статусом; наблюдаемость таких ошибок добавим позднее.

`pageData` — данные конкретного представления. Мы не передаём шаблону весь сервер или будущую запись пользователя с паролем. Это делает границы страницы понятными.

Текст не должен становиться программой

`html/template` экранирует обычные строки с учётом места вставки. Если название содержит `<script>`, браузер должен увидеть текст, а не исполнить сценарий. Нельзя отключать защиту преобразованием пользовательского ввода в `template.HTML`. Такой тип означает доверенный HTML и снимает часть автоматических проверок.

Тестируем это и пустой каталог:

```
package web

import (
```

```

"net/http/httptest"
"strings"
"testing"

"github.com/dake-edu/gopher-shop/book/examples/12-templates/internal/catalog"
)

func TestEscapeAndEmptyCatalog(t *testing.T) {
    cases := [][]catalog.Book{
        nil,
        {{ID: "go", Title: "<script>alert(1)</script>", Currency: "KZT"}},
    }
    for _, books := range cases {
        w := httptest.NewRecorder()
        New(books).ServeHTTP(w, httptest.NewRequest("GET", "/", nil))
        body := w.Body.String()
        if w.Code != 200 || strings.Contains(body, "<script>") {
            t.Fatalf("unsafe response: %s", body)
        }
        if len(books) == 0 && !strings.Contains(body, "Каталог пока пуст") {
            t.Fatal("missing empty state")
        }
        if len(books) > 0 && !strings.Contains(body, "&lt;script&gt;") {
            t.Fatal("missing escaped title")
        }
    }
    if money(105) != "1.05" {
        t.Fatal("money format")
    }
}

```

&& в тесте — логическое И: обе проверки должны быть истинны. Вычисление слева направо прекращается при первом ложном условии. В `[][]catalog.Book` внешний срез хранит наборы книг, а внутренний — книги одного сценария. Вложенный литерал может опускать повтор уже известного типа; это сокращение, а не новая структура данных.

Защита от внедрения HTML не заменяет проверку цены, доступов или адресов. Она решает конкретную задачу представления текста. Опорный образ — подпись в рамке: символы остаются подписью, а не становятся указаниями монтажнику. Граница образа: безопасность зависит от контекста вставки, а не от общей «чистоты» строки.

CSS и проверка удобства

Селектор выбирает элементы, свойства внутри фигурных скобок задают оформление. `max-width` ограничивает длину строки, `margin` задаёт внешние отступы, `padding` — внутренние. `:focus-visible` обозначает видимый фокус при навигации; не убирайте обводку без понятной замены. `rem` соотносится с размером шрифта корневого элемента. В CSS запись `text-align: center`; состоит из имени свойства, двоеточия, значения и точки с запятой. Это язык оформления, а не инструкция Go.

```

*, *::before, *::after {
    box-sizing: border-box;
}

:root {
    --color-primary: #006078;
    --color-on-primary: #fff;
}

```

```

}

html {
  height: 100%;
  overflow-wrap: anywhere;
}

body {
  min-height: 100%;
  min-height: 100dvh;
  display: flex;
  flex-direction: column;
  max-width: 52rem;
  margin: 0 auto;
  padding: 0.75rem;
  font: 1rem/1.6 system-ui, sans-serif;
  color: #182f35;
  background: #fff;
}

h1 {
  font-size: clamp(1.5rem, 5vw, 2.25rem);
  line-height: 1.25;
}

h2 {
  font-size: clamp(1.2rem, 4vw, 1.6rem);
  line-height: 1.35;
}

a {
  color: var(--color-primary);
  text-underline-offset: 0.2em;
}

a:focus-visible, button:focus-visible, input:focus-visible {
  outline: 3px solid #9c3b00;
  outline-offset: 3px;
}

header, nav {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.5rem 0.75rem;
}

header a, nav a {
  display: inline-flex;
  align-items: center;
  min-height: 2.75rem;
}

header, footer {
  padding: 0.75rem 0;
}

footer {
  background: var(--color-primary);
  color: var(--color-on-primary);
}

```

```

padding: 1rem;
border-radius: 0.5rem;
margin-top: auto;
flex-shrink: 0;
text-align: center;
}

article {
  min-width: 0;
  padding: 1rem 0;
  border-bottom: 1px solid #bac7cb;
}

form {
  display: grid;
  gap: 0.75rem;
  margin: 1rem 0;
}

label {
  display: block;
  min-width: 0;
}

input, button {
  max-width: 100%;
  min-width: 0;
  min-height: 2.75rem;
  padding: 0.5rem 0.75rem;
  border: 1px solid #647a80;
  border-radius: 0.5rem;
  font: inherit;
}

input:not([type="checkbox"]):not([type="hidden"]) {
  display: block;
  width: 100%;
  background: #fff;
  color: #182f35;
}

input[type="checkbox"] {
  width: 1.25rem;
  height: 1.25rem;
  min-height: 0;
  margin: 0 0.5rem 0 0;
  vertical-align: middle;
}

label:has(input[type="checkbox"]) {
  display: flex;
  align-items: center;
  min-height: 2.75rem;
}

button {
  cursor: pointer;
  white-space: normal;
  overflow-wrap: anywhere;
  background: var(--color-primary);

```

```

        color: var(--color-on-primary);
    }

    img, svg {
        max-width: 100%;
        height: auto;
    }

    pre {
        max-width: 100%;
        overflow-x: auto;
    }

    .error, [role="alert"] {
        color: #9b1c1c;
    }

    @media (min-width: 40rem) {
        body {
            padding: 1.5rem;
            font-size: 1.1rem;
        }

        form {
            max-width: 36rem;
        }

        button {
            justify-self: start;
        }
    }

    .brand {
        display: inline-flex;
        align-items: center;
        gap: 0.5rem;
        max-width: 100%;
    }

    .brand img {
        flex: 0 0 auto;
        width: 5rem;
        height: 5rem;
        object-fit: contain;
    }

    /* Keep the logo at the left edge and let only the menu links wrap. */
    header {
        flex-wrap: nowrap;
        flex-shrink: 0;
    }

    header .brand {
        flex: 0 0 auto;
    }

    header nav {
        flex: 1 1 0;
        min-width: 0;
        margin-inline-start: auto;
    }

```

```

        justify-content: flex-end;
    }

    main {
        flex: 0 0 auto;
        min-width: 0;
    }

    @media (max-width: 39.999rem) {
        header {
            flex-direction: column;
            gap: 0.5rem;
        }

        header nav {
            flex: 0 1 auto;
            width: 100%;
            margin-inline-start: 0;
            justify-content: center;
        }
    }

    header nav a {
        padding: 0.25rem;
        border-radius: 0.5rem;
        text-decoration: none;
    }

    header nav a:hover {
        background: #e5edf0;
    }

    header nav a[aria-current="true"] {
        background: var(--color-primary);
        color: var(--color-on-primary);
    }

    @media (max-width: 39.999rem) {
        header nav {
            column-gap: 0.25rem;
        }
    }

```

Селектор `footer` выбирает подвал из `web/templates/partials/footer.html`. Правило `text-align: center` в `web/static/css/style.css` выравнивает его текст по горизонтальному центру, включая оба абзаца. Блочный подвал занимает доступную ширину; длинные строки по-прежнему переносятся на узком экране. Фиксированная ширина и абсолютное позиционирование для этого не нужны.

Чтобы подвал не поднимался к середине короткой страницы, задаём корневому `html` высоту `100%`, а `body` — `min-height: 100%`, `display: flex` и `flex-direction: column`. Следующая декларация `min-height: 100dvh` использует высоту динамической области просмотра в поддерживающих её браузерах; процентная запись остаётся запасным вариантом. `min-height`, в отличие от жёсткого ограничения высоты, позволяет странице расти вместе с содержимым.

У `footer` свойство `margin-top: auto` забирает свободное место над подвалом. На короткой странице это опускает его к нижнему краю с учётом внутренних отступов `body`; на длинной

свободного места нет, и подвал идёт после содержимого. `flex-shrink: 0` не даёт сжимать подвал. Для `main` запись `flex: 0 0 auto` сохраняет естественную высоту содержимого. Здесь не требуется `position: fixed`: закреплённый поверх страницы подвал мог бы закрыть кнопки и текст.

В шапке логотип и `nav` — два элемента горизонтальной flex-раскладки. `header` не переносит эти два блока (`flex-wrap: nowrap`), а ссылки внутри `nav` могут переноситься.

`margin-inline-start: auto` отдаёт свободное место перед меню, `justify-content: flex-end` выравнивает его ссылки к правому краю. `flex: 1 1 0` позволяет меню занять оставшуюся ширину, а `min-width: 0` — сжаться на смартфоне. На широком экране логотип остаётся слева, меню справа. В медиазапросе `@media (max-width: 39.999rem)` меняем направление шапки на `column`: логотип располагается по центру, меню под ним. Для меню сбрасываем автоматический боковой отступ, задаём `width: 100%` и `justify-content: center`. Короткая ссылка `eGopher` помогает трём пунктам помещаться в одну строку даже при ширине 320 пикселей. Перенос остаётся разрешённым для увеличенного текста или более длинного перевода: доступность ссылок важнее принудительного удержания одной строки.

Проверяйте три случая: короткую карточку книги, длинный каталог и смартфон в двух ориентациях. Подвал должен оставаться ниже содержимого, логотип — полностью видимым, а ссылки — доступными без горизонтальной прокрутки. Поведение экранной клавиатуры дополнительно проверяется на реальном устройстве: уменьшение окна настольного браузера его не воспроизводит.

Проверьте страницу без мыши: Tab должен последовательно переводить фокус по ссылкам, Enter — открывать выбранную. Увеличьте масштаб браузера до 200%, сузьте окно. Текст и цены должны оставаться читаемыми. Эта небольшая проверка не равна полному аудиту доступности, но выявляет дефекты раньше.

Самостоятельно: пустое состояние

Предскажите, что отобразится при `New(nil)`. Затем найдите проверку в `template_test.go`. Измените сообщение на «Новые книги скоро появятся» и сначала исправьте ожидаемый текст теста. Проверьте, что тест падает до изменения шаблона и проходит после него.

Подсказка: нужен блок `else` у `range`, а не дополнительная проверка длины в Go. Решение: заменить только текст этого блока и соответствующее ожидание теста. Восстановите исходный вариант перед следующей контрольной точкой.

Если компилятор сообщает `no matching files found`, проверьте дерево ресурсов рядом с `web/assets.go`. Если `ParseFS` не находит страницу, проверьте её имя в `web/templates/pages`. Если сервер завершился с ошибкой разбора шаблона, проверьте пары `range/end` и имена функций. Если пропал стиль, откройте `/assets/css/style.css`: должен прийти CSS, а не HTML с 404.

Готовность: вы различаете Go, HTML, CSS и действия шаблона, объясняете автоматическое экранирование и буфер, проверяете пустой каталог и умеете пройти страницу клавиатурой.

Адаптивность: ширина содержимого, а не название устройства

Страница должна перестраиваться при изменении доступной ширины: смартфон поворачивается, планшет открывает два приложения рядом, пользователь ноутбука увеличивает масштаб. Не определяем устройство по имени браузера и не запрещаем масштабирование. В layout уже задан `width=device-width`, `initial-scale=1` без `maximum-scale`.

Начинаем с одной колонки. `box-sizing: border-box` включает рамку и внутренние отступы в заданную ширину. Поля занимают `width: 100%`, но не выходят за родителя; `min-width: 0` разрешает им сжиматься. `overflow-wrap: anywhere` переносит даже длинный ID, почту или SHA-256. Не скрываем ошибку через `overflow-x: hidden`: иначе часть содержимого просто исчезнет.

`display: grid` у формы располагает поля по порядку, `gap` создаёт промежутки без набора отдельных отступов. Навигация использует `display: flex` и `flex-wrap: wrap`: ссылки переходят на следующую строку. `clamp` ограничивает размер заголовка снизу и сверху, а среднее значение зависит от ширины окна (`vw`). Кнопки и поля имеют высоту не меньше `2.75rem`; это выбранный для проекта удобный размер касания, а не доказательство полной доступности интерфейса.

Селектор `:not` исключает checkbox и скрытые поля из правила текстовых полей. `:has(input[type="checkbox"])` выбирает подпись, содержащую checkbox: вся подпись становится удобной областью нажатия. Без этой дополнительной раскладки обычный label всё равно остаётся связан с вложенным полем. `@media (min-width: 40rem)` расширяет отступы и ограничивает длину формы, когда места достаточно. Отдельного запрета или особой страницы для горизонтальной ориентации не требуется.

Сейчас проверьте каталог, карточку книги и длинное название; после глав 13 и 20 добавьте ошибку формы и вход при размерах 320×568, 390×844, 844×390, 768×1024, 1024×768 и 1366×768. На ноутбуке повторите проверку при масштабе 200%. Не должно быть горизонтальной прокрутки всей страницы, обрезанных полей и перекрывающихся кнопок. Вертикальная прокрутка длинной формы нормальна, в том числе при открытой экранной клавиатуре. В дальнейшем те же правила проверяем для кабинета и административных страниц.

Размер окна эмулятора не воспроизводит полностью Safari на iPhone или экранную клавиатуру Android. Перед коммерческим выпуском дополните автоматическую проверку реальными устройствами. Разбор подхода: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design.

Подвал итоговых примеров содержит динамический год, название и «Учебный интернет-магазин». В главах об оплате используется `sandbox`: настоящие деньги не списываются. Подробности режима относятся к объяснению платёжного сценария и его странице, а не к общему подвалу.

Активный раздел меню

Верхнее меню показывает, в каком разделе находится читатель. Функция шаблона `nav_active` принимает ключ раздела и сравнивает его с `active`, выбранным при подготовке шаблона. Каталог и карточка книги относятся к `catalog`; корзина, заказ и учебная оплата — к `cart`; вход, регистрация, библиотека и управление — к `account`. Более поздние страницы появятся в соответствующих главах.

В шапке условие `{{if nav_active "catalog"}}` добавляет `aria-current="true"` только активной ссылке. Значение `true` обозначает текущий пункт набора: здесь это раздел, а не обязательно точное совпадение URL с адресом ссылки. Например, карточка книги сохраняет выделение каталога. Программа чтения экрана получает то же указание, которое зрячий пользователь видит по цвету.

Селектор `header nav a[aria-current="true"]` выбирает активную ссылку. Общий бирюзовый фон `#006078` и белый текст `#fff` выделяют её без зависимости от наведения мыши. Отступы заданы у всех пунктов заранее, поэтому при смене активного пункта ширина меню не скачет. `text-decoration: none` убирает подчёркивание только у верхнего меню; обычные ссылки в содержимом остаются подчёркнутыми. Правило `:focus-visible` сохраняет рамку при переходе клавишей Tab.

`active` хранится в замыкании конкретного, заранее подготовленного шаблона. Не меняйте общую переменную «текущая страница» перед каждым запросом: одновременные посетители могли бы увидеть чужую подсветку. В следующей главе каталог и корзина используют один файл `home.html`, но получают два независимых подготовленных шаблона. Это разделяет данные запроса и неизменяемые настройки отображения.

`border-radius: 0.5rem` задаёт одинаковое небольшое скругление кнопок, полей ввода и активного пункта меню. При стандартном корневом размере шрифта 16 пикселей это 8 пикселей. Скругление меняет форму границы, а размеры полей и область нажатия сохраняются.

Подвал использует тот же бирюзовый фон `#006078` и белый текст `#fff`, что кнопки и активный пункт меню. Внутренний отступ `padding: 1rem` отделяет текст от края цветного блока; скругление `0.5rem` согласовано с элементами управления. Цвет и оформление не заменяют `margin-top: auto`: именно flex-раскладка сохраняет положение подвала на коротких страницах.

Цвета хранятся в CSS-переменных: `--color-primary` и `--color-on-primary` объявлены в `:root`, который выбирает корневой элемент документа. Имя пользовательского свойства начинается с двух дефисов. Запись `background: var(--color-primary)` подставляет значение переменной; `color: var(--color-on-primary)` задаёт цвет текста на этом фоне. Кнопки, активный пункт меню и подвал используют одну пару переменных. Чтобы сменить цветовую тему, измените два значения в начале `style.css`, затем пересоберите приложение со встроенным CSS.

Продолжение: от каталога к работающему магазину

Вы прошли ознакомительный фрагмент: создали модель книги, разделили программу на пакеты, запустили HTTP-сервер и вывели каталог через шаблоны. Проверьте себя: можете ли вы добавить поле карточки, объяснить путь запроса до шаблона и самостоятельно найти причину ошибки?

В полной книге главы 13–42 продолжают этот же проект: формы и хранение данных, регистрация и сессии, корзина и заказы, тестовая оплата и выдача файлов, административная часть, резервные копии и выпуск приложения. Вместе с книгой предоставляются контрольные точки продолжения.

[Посмотреть полную книгу, доступные форматы и условия покупки на shanraq.org](https://shanraq.org).